

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA, ĐẠI HỌC QUỐC
GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

XÂY DỰNG FRAMEWORK TẠO
DỰNG MÔ HÌNH AI CHO CÁC
ỨNG DỤNG THEO DÕI
CHUYỂN ĐỘNG CỦA CON
NGƯỜI

Chuyên ngành: Khoa học Máy tính

GVHD: TS. Lê Trọng Nhân

—o0o—

Học viên: Nguyễn Trương Minh Hoàng

Mã số học viên: 2270757

Thành phố Hồ Chí Minh, 05 /2026

XÁC NHẬN CỦA GIẢNG VIÊN

Xác nhận chữ ký

LỜI CAM ĐOAN

Tôi cam đoan rằng nghiên cứu này là công trình của riêng tôi, được thực hiện dưới sự giám sát và hướng dẫn của TS. Lê Trọng Nhân. Kết quả nghiên cứu của tôi là hợp pháp và chưa được công bố dưới bất kỳ hình thức nào trước đây. Tất cả các tài liệu được sử dụng trong nghiên cứu này đều được tôi thu thập từ nhiều nguồn khác nhau và đã được liệt kê đầy đủ trong phần tài liệu tham khảo. Ngoài ra, trong nghiên cứu này, tôi cũng đã sử dụng kết quả của một số tác giả và tổ chức khác. Tất cả đều đã được trích dẫn một cách thích hợp. Trong mọi trường hợp đạo văn, tôi hoàn toàn chịu trách nhiệm về hành động của mình. Do đó, Trường Đại học Bách Khoa - ĐHQG TP.HCM không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào được thực hiện trong nghiên cứu của tôi.

LỜI CẢM ƠN

Tôi xin bày tỏ lòng biết ơn chân thành đến tất cả những người đã đóng góp vào việc phát triển và hoàn thiện luận văn này.

Trước tiên, tôi xin gửi lời tri ân sâu sắc nhất đến người hướng dẫn khoa học của tôi, TS. Lê Trọng Nhân, vì sự hướng dẫn quý báu, động viên và phản hồi mang tính xây dựng trong suốt quá trình hình thành đề tài này. Chuyên môn và những hiểu biết sâu sắc của thầy đã đóng vai trò quan trọng trong việc định hướng nghiên cứu của tôi.

Tôi cũng vô cùng biết ơn Trường Đại học Bách Khoa - ĐHQG TP.HCM và Khoa Khoa học và Kỹ thuật Máy tính đã cung cấp môi trường học thuật nuôi dưỡng và các nguồn lực cần thiết để thực hiện nghiên cứu này.

Lời cảm ơn đặc biệt gửi đến các bạn đồng nghiệp và đồng môn đã chia sẻ quan điểm và đưa ra những gợi ý sâu sắc, làm phong phú thêm phạm vi của nghiên cứu này.

Cuối cùng, tôi vô cùng biết ơn gia đình và bạn bè vì sự ủng hộ không ngừng, sự kiên nhẫn và động lực khi tôi đắm mình vào nỗ lực đầy thách thức nhưng bổ ích này.

Đề xuất nghiên cứu này là sự phản ánh của sự hỗ trợ và cống hiến tập thể, và tôi xin chân thành cảm ơn tất cả những người đã đóng góp trực tiếp hoặc gián tiếp vào việc hiện thực hóa nó.

TÓM TẮT

Sự phát triển của học máy và điện toán biên đã cách mạng hóa các ứng dụng theo dõi chuyển động, cho phép các hệ thống thời gian thực, tiết kiệm năng lượng và chính xác cao cho nhiều trường hợp sử dụng trong y tế, phân tích thể thao và tương tác người-máy. Tuy nhiên, việc phát triển các mô hình AI cho theo dõi chuyển động vẫn còn phức tạp, đòi hỏi kiến thức chuyên môn về tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và triển khai trên thiết bị biên. Bài báo này trình bày một framework toàn diện giúp đơn giản hóa quy trình end-to-end để tạo ra các mô hình AI được tùy chỉnh cho theo dõi chuyển động con người trên các thiết bị biên có tài nguyên hạn chế.

Framework giới thiệu một phương pháp tiền xử lý tương tác mới với tính năng lựa chọn cửa sổ thời gian kéo thả, cho phép người dùng chú thích và phân đoạn dữ liệu chuyển động một cách hiệu quả mà không cần kiến thức lập trình sâu rộng. Hệ thống hỗ trợ nhiều thuật toán học máy bao gồm Random Forest, Support Vector Machines và Neural Networks, với khả năng tự động sinh mã cho nhiều họ thiết bị và môi trường thực thi khác nhau như các bo tương thích Arduino, ARM Cortex-M thuần, ESP-IDF, MicroPython và Zephyr. Kiến trúc tạo mã nhận biết tối ưu hóa xem xét nhiều chiến lược triển khai—độ chính xác, tốc độ, tiêu thụ điện năng và phương án cân bằng—điều chỉnh gói mã sinh ra theo ràng buộc của từng đích triển khai.

Để kiểm chứng framework ở mức thực nghiệm, chúng tôi sử dụng bộ dữ liệu Nhận dạng Hoạt động Con Người (HAR) được thu thập ở 100Hz từ cảm biến IMU 6 trục (LSM6DS3) trên Seeed XIAO nRF52840 Sense như một nền tảng tham chiếu thuộc họ ARM Cortex-M4F. Trong nghiên cứu bằng chứng khái niệm này sử dụng dữ liệu đơn đối tượng trên năm lớp hoạt động, năm thuật toán (Random Forest, SVM, Neural Network, PyTorch MLP, PyTorch 1D-CNN) đạt độ chính xác 97,38–99,13% trên tập kiểm tra, với PyTorch MLP và 1D-CNN đạt cao nhất (99,13%). Quan trọng hơn, cùng một pipeline huấn luyện có thể được ánh xạ sang nhiều đích triển khai thông qua hệ thống generator phân tầng theo mô hình, nền tảng và backend thực thi. Dữ liệu tham khảo trên thiết bị cho thấy mô hình PyTorch 1D-CNN đạt 99,4% độ chính xác triển khai với tỷ lệ từ chối chỉ 0,6%. Phân tích so sánh với các nền tảng thương mại (Edge Impulse, SensiML) cho thấy framework mã nguồn mở của chúng tôi cung cấp tính minh bạch hoàn toàn, tính linh hoạt tiền xử lý vượt trội và chi phí bằng không.

Framework giải quyết các khoảng trống quan trọng trong các giải pháp hiện tại bằng cách cung cấp: (1) quy trình làm việc dựa trên GUI trực quan để tiếp cận cho

người không chuyên, (2) kiểm soát hoàn toàn đường ống tiền xử lý với phản hồi trực quan, (3) triển khai đa thiết bị với các tối ưu hóa đặc thù cho từng đích, và (4) kiến trúc có thể mở rộng hỗ trợ tích hợp các thuật toán, backend và nền tảng bổ sung trong tương lai. Nghiên cứu này đóng góp vào việc dân chủ hóa phát triển Edge AI cho các ứng dụng theo dõi chuyển động, với tác động tiềm năng trong phục hồi chức năng y tế, giám sát hiệu suất thể thao và công nghệ hỗ trợ.

Mục lục

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
TÓM TẮT	iv
Mục lục	vi
Danh sách bảng	xi
Danh sách hình vẽ	xii
1 Giới thiệu	1
1.1 Bối cảnh và động lực nghiên cứu	1
1.2 Phát biểu bài toán	1
1.3 So sánh với các nền tảng hiện có	2
1.4 Mục tiêu nghiên cứu	3
1.5 Đóng góp nghiên cứu	4
1.6 Phạm vi và hạn chế	5
1.7 Tổ chức luận văn	6
2 Cơ sở lý thuyết	7
2.1 Tổng quan	7
2.2 Các lĩnh vực ứng dụng của theo dõi chuyển động	7
2.3 Theo dõi chuyển động dựa trên cảm biến	8
2.4 Theo dõi chuyển động dựa trên thị giác	8
2.5 AI trong theo dõi chuyển động	9
2.6 Các Framework tạo mô hình	9
2.7 Tiền xử lý dữ liệu và tạo cửa sổ	11
2.8 Trích xuất và biểu diễn đặc trưng	11

2.9	Triển khai biên và tối ưu hóa	12
2.10	Các khoảng trống trong công trình hiện có	13
2.11	Tóm tắt	14
3	Phương pháp luận	15
3.1	Dữ liệu cảm biến quán tính cho nhận dạng hoạt động	15
3.2	Quy trình tiền xử lý tương tác	16
3.2.1	Tải lên và trực quan hóa dữ liệu	16
3.2.2	Các phương pháp lọc và làm sạch tín hiệu	16
3.2.3	Phân đoạn và sinh cửa sổ huấn luyện	18
3.3	Trích xuất đặc trưng	19
3.3.1	Tham số cửa sổ và ràng buộc nhất quán	19
3.3.2	Các loại trích xuất đặc trưng	19
3.3.3	Chiến lược đệm cửa sổ	21
3.3.4	Tăng cường dữ liệu cho huấn luyện	21
3.3.5	Ngưỡng tin cậy cho từ chối hoạt động không xác định	22
3.4	Huấn luyện mô hình học máy	22
3.4.1	Các thuật toán học máy áp dụng	22
3.4.2	Giao thức huấn luyện và đánh giá	24
3.4.3	Tối ưu hóa siêu tham số	25
3.5	Tạo mã cho triển khai biên	25
3.5.1	Kiến trúc	25
3.5.2	Lý do chọn tạo mã trực tiếp thay vì Runtime Engine	26
3.5.3	Triển khai TFLite Micro	30
3.5.4	Cấu trúc mã đã tạo	31
3.5.5	Lượng tử hóa mô hình cho triển khai biên	32
3.5.6	Kiến trúc tạo mã đa kiến trúc	35
3.5.7	Các chiến lược tối ưu hóa	35
3.5.8	Điều chỉnh đặc thù nền tảng	35
4	Thiết kế và hiện thực	36
4.1	Kiến trúc hệ thống	36
4.1.1	Nền tảng công nghệ và môi trường phát triển	36
4.1.2	Kiến trúc ứng dụng	37
4.1.3	Quản lý trạng thái	38
4.2	Module quản lý dữ liệu	39
4.2.1	Giao diện tải dữ liệu	39

4.3	Module tiền xử lý tương tác	41
4.3.1	Giao diện lựa chọn đoạn tương tác	41
4.3.2	Sinh cửa sổ trượt	42
4.3.3	Trực quan hóa lọc tín hiệu	43
4.4	Module trích xuất đặc trưng	43
4.4.1	Chế độ trích xuất đặc trưng	43
4.4.2	Tích hợp data augmentation	44
4.4.3	Phân chia train/validation/test	45
4.5	Module huấn luyện mô hình	46
4.5.1	Lựa chọn thuật toán	46
4.5.2	Tối ưu hóa siêu tham số	47
4.5.3	Theo dõi tiến trình real-time	47
4.5.4	Lưu trữ mô hình	48
4.6	Module tạo mã triển khai	49
4.6.1	Kiến trúc Code Generator: Factory Pattern	49
4.6.2	Hệ thống phân cấp Generator	49
4.6.3	Kiến trúc ba trục: Model $\times$ Platform $\times$ Backend	50
4.6.4	Sắp xếp lại thứ tự đặc trưng khi sinh mã	51
4.6.5	Bốn chế độ tối ưu hóa	51
4.6.6	Gói đầu ra triển khai	52
4.6.7	Kiến trúc đa mô hình: CNN code generation	53
4.7	Module kiểm tra thiết bị	54
4.7.1	Giao tiếp serial	54
4.7.2	Hiển thị phân loại real-time	54
4.7.3	Ngưỡng tin cậy	55
4.7.4	Quy trình xác minh triển khai	55
4.8	Luồng dữ liệu và quản lý trạng thái tổng thể	55
4.8.1	Luồng dữ liệu end-to-end	55
4.8.2	Cơ chế rollback và retry	57
4.8.3	Truy xuất thông tin model	58
4.9	Tổng kết chương	58
5	Kết quả thực nghiệm	59
5.1	Thiết lập thực nghiệm	59
5.2	Kết quả huấn luyện	60
5.2.1	Bài toán 3 lớp (running / still / walking)	60

5.2.2	Bài toán 5 lớp — Đánh giá chính	61
5.2.3	Hiệu suất theo từng lớp	62
5.3	So sánh hiệu suất giữa các thuật toán	63
5.4	Khả năng triển khai đa thiết bị	64
5.5	Đánh giá triển khai trên thiết bị	65
5.5.1	Tổng hợp kết quả triển khai	65
5.5.2	Phân tích theo từng mô hình	66
5.5.3	Bài học: khoảng cách huấn luyện–triển khai	66
5.6	So sánh với Edge Impulse	67
5.6.1	So sánh hiệu suất	67
5.6.2	So sánh tài nguyên	67
5.6.3	So sánh kiến trúc	68
5.7	Tóm tắt	69
6	Kết luận và hướng phát triển	70
6.1	Diễn giải các phát hiện chính	70
6.1.1	Hiệu suất mô hình và lựa chọn thuật toán	70
6.1.2	Cân bằng lớp trong thực hành	70
6.1.3	Tính khả thi triển khai biên	70
6.1.4	Tiền xử lý tương tác: một bước ngoặt về mô hình	71
6.2	So sánh với công trình liên quan	71
6.2.1	Các nền tảng thương mại	71
6.2.2	Các hệ thống HAR học thuật	71
6.3	Các đánh đổi thiết kế	72
6.3.1	ML cổ điển vs học sâu	72
6.3.2	Đặc trưng thủ công vs đặc trưng tự học	72
6.3.3	Tạo mã trực tiếp vs runtime suy luận	72
6.3.4	Các chế độ tối ưu hóa	73
6.4	Tương đồng huấn luyện–triển khai trong edge ML	73
6.4.1	Các nguồn không khớp được xác định	73
6.4.2	Tác động thực tế	74
6.4.3	Danh sách kiểm tra tương đồng	74
6.4.4	So sánh với nền tảng thương mại	75
6.4.5	Bộ lọc Kalman: đột phá về parity	75
6.5	Tăng cường dữ liệu và ngưỡng tin cậy	76
6.5.1	Tăng cường nhận biết lớp	76

6.5.2	Ngưỡng tin cậy cho từ chối hoạt động không xác định	76
6.6	Phân tích các phát hiện kỹ thuật quan trọng	77
6.6.1	Kiến trúc tạo mã đa kiến trúc	77
6.6.2	Hạn chế TFLite và giải pháp Keras surrogate	77
6.7	Các hạn chế	77
6.7.1	Hạn chế tập dữ liệu	77
6.7.2	Lựa chọn kích thước cửa sổ	78
6.7.3	Vị trí và hướng cảm biến	78
6.7.4	Sự đa dạng nền tảng tính toán	78
6.7.5	Tính hợp lệ bên ngoài	78
6.8	Hướng phát triển tương lai	79
6.8.1	Các mở rộng ngắn hạn	79
6.8.2	Tầm nhìn dài hạn	79
6.8.3	Các cải tiến khả năng sử dụng	80
6.9	Kết luận	80
6.9.1	Tóm tắt đóng góp	80
6.9.2	Xem lại các mục tiêu nghiên cứu	81
6.9.3	Các ứng dụng thực tế	81
6.9.4	Suy nghĩ cuối cùng	82
Tài liệu tham khảo		83

Danh sách bảng

1.3.1	So sánh các nền tảng Edge ML cho theo dõi chuyển động	3
3.2.1	So sánh các phương pháp lọc theo tính tương đồng huấn luyện-triển khai	18
3.3.1	Sáu loại trích xuất đặc trưng: tín hiệu nguồn, cấu thành và đặc tính triển khai	20
3.5.1	Ma trận đích triển khai được hỗ trợ	26
3.5.2	So sánh phương pháp triển khai mô hình ML trên vi điều khiển	28
3.5.3	So sánh chiến lược lượng tử hóa theo phương pháp tạo mã	34
4.4.1	Các chế độ trích xuất đặc trưng và số lượng features	43
4.5.1	Thuật toán học máy được hỗ trợ	46
4.6.1	Chế độ tối ưu hóa mã triển khai	51
5.2.1	Kết quả huấn luyện — bài toán 3 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 156 mẫu)	60
5.2.2	Kết quả huấn luyện — bài toán 5 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 229 mẫu)	61
5.2.3	Hiệu suất theo từng lớp — bài toán 5 lớp, tập kiểm tra (229 mẫu)	62
5.3.1	So sánh tổng hợp các thuật toán — bài toán 5 lớp	63
5.3.2	Ảnh hưởng của số lượng lớp đến độ chính xác kiểm tra	64
5.4.1	Ma trận đích triển khai — phạm vi hỗ trợ của hệ thống tạo mã	64
5.5.1	Kết quả triển khai trên thiết bị — Seeed XIAO nRF52840	65
5.6.1	So sánh framework đề xuất vs Edge Impulse — bài toán 3 lớp, cùng tập dữ liệu	67
5.6.2	So sánh tài nguyên triển khai — Cortex-M4F @ 64 MHz	68
5.6.3	So sánh kiến trúc với các nền tảng thương mại	69
6.4.1	Danh sách kiểm tra tương đồng huấn luyện-triển khai	75

Danh sách hình vẽ

1.4.1	Quy trình làm việc hoàn chỉnh: Từ tải dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)	4
4.2.1	Giao diện tải lên và trực quan hóa dữ liệu cảm biến. Framework tự động phát hiện cấu trúc cột IMU và hiển thị biểu đồ tín hiệu đồng bộ trên tất cả các trục gia tốc kế và con quay hồi chuyển, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiền xử lý.	40
4.3.1	Giao diện cửa sổ kéo thả tương tác. Người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để xác định các đoạn hoạt động đại diện. Cửa sổ trượt tự động sinh các mẫu huấn luyện từ các đoạn đã chọn, kết hợp giữa kiểm soát thủ công và tự động hóa.	42
4.4.1	Giao diện trích xuất đặc trưng. Người dùng chọn loại đặc trưng (bất biến hướng, miền thời gian, miền tần số), cấu hình tăng cường dữ liệu nhận biết lớp, và phân chia dữ liệu huấn luyện/xác thực/kiểm tra với lấy mẫu phân tầng.	45
4.5.1	Giao diện cấu hình huấn luyện mô hình. Người dùng chọn thuật toán, cấu hình tối ưu hóa siêu tham số với GridSearchCV, chiến lược cân bằng lớp, và theo dõi tiến trình huấn luyện thời gian thực.	48
4.6.1	Giao diện tạo mã triển khai. Hiển thị các tệp header, implementation và Arduino sketch được sinh ra. Người dùng chọn chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình nền tảng đích và tải xuống gói triển khai hoàn chỉnh.	53
5.2.1	Ma trận nhầm lẫn — PyTorch MLP, bài toán 5 lớp (tập kiểm tra 229 mẫu). Các lỗi phân loại tập trung ở nhóm hoạt động đi bộ: 1 mẫu walking bị nhầm thành walking_downstairs (hàng walking, cột W.Down).	63

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

Theo dõi chuyển động (motion tracking) đóng vai trò then chốt trong y tế (phục hồi chức năng, phát hiện té ngã), thể thao (theo dõi hiệu suất), thực tế ảo và robot học^[43]. Thị trường toàn cầu đạt 2,8 tỷ USD năm 2023 với tốc độ tăng trưởng CAGR 18,4%^[2], được thúc đẩy bởi thiết bị đeo y tế và nhà thông minh. Công bố nghiên cứu về nhận dạng hoạt động (HAR) tăng 3,1 lần trong giai đoạn 2018–2023^[38]. Sự hội tụ của cảm biến IoT giá rẻ, thuật toán học máy hiệu quả và tính toán biên đã phổ cập theo dõi chuyển động từ phòng thí nghiệm sang ứng dụng tiêu dùng.

Tuy nhiên, phát triển mô hình AI cho phân tích chuyển động vẫn phức tạp, bao gồm thu thập, tiền xử lý, trích xuất đặc trưng và huấn luyện mô hình—đặt ra thách thức cho nhà phát triển không chuyên về AI^[26]. Triển khai trên thiết bị biên (vì điều kiện hạn chế bộ nhớ và năng lượng) đòi hỏi thêm kiến thức về tối ưu hóa mô hình, tạo mã và chi tiết đặc thù nền tảng. Thách thức càng lớn khi hệ sinh thái thiết bị ngày càng phân mảnh: cùng một mô hình cần đóng gói khác nhau cho Arduino, ARM Cortex-M, ESP-IDF, MicroPython hay Zephyr.

1.2 Phát biểu bài toán

Bất chấp những tiến bộ trong công nghệ theo dõi chuyển động, việc tạo ra các mô hình AI cho phân tích chuyển động vẫn là một nhiệm vụ đầy thách thức đặt ra một số rào cản quan trọng:

1. **Độ phức tạp kỹ thuật:** Các cách tiếp cận hiện tại đòi hỏi kiến thức kỹ thuật sâu rộng trải rộng nhiều lĩnh vực—xử lý tín hiệu cho tiền xử lý dữ liệu, học máy

cho phát triển mô hình và hệ thống nhúng cho triển khai biên.

2. **Thách thức tiền xử lý:** Dữ liệu cảm biến chuyển động vốn dĩ nhiều và đòi hỏi tiền xử lý cẩn thận. Các công cụ hiện có hoặc cung cấp kiểm soát hạn chế đối với tiền xử lý (các cách tiếp cận hộp đen tự động) hoặc đòi hỏi lập trình mở rộng để triển khai các quy trình tiền xử lý tùy chỉnh.
3. **Tính linh hoạt hạn chế:** Nhiều framework có sẵn có đường cong học tập dốc hoặc cung cấp chức năng hạn chế để tùy chỉnh mô hình theo các yêu cầu theo dõi chuyển động cụ thể. Các nền tảng thương mại như Edge Impulse và SensiML cung cấp quy trình làm việc đơn giản hóa nhưng với chi phí đáng kể (\$20-500/tháng) và với các tùy chọn tùy chỉnh bị hạn chế.
4. **Độ phức tạp triển khai:** Dịch cùng một mô hình đã huấn luyện thành các gói mã tối ưu hóa cho nhiều họ thiết bị biên khác nhau đòi hỏi hiểu biết sâu về kiến trúc phần cứng đích, ràng buộc bộ nhớ, hệ sinh thái công cụ (Arduino, ESP-IDF, RTOS, MicroPython) và kỹ thuật tối ưu hóa phù hợp cho từng trường hợp triển khai.
5. **Rào cản khả năng tiếp cận:** Sự kết hợp của các yếu tố này hạn chế đổi mới và cản trở việc áp dụng rộng rãi các giải pháp theo dõi chuyển động được hỗ trợ bởi AI, đặc biệt trong bối cảnh nghiên cứu và giáo dục.

Do đó, có một nhu cầu cấp thiết về một framework linh hoạt, trực quan và thân thiện với người dùng giúp đơn giản hóa việc tạo các mô hình AI phù hợp cho các ứng dụng theo dõi chuyển động trong khi cung cấp kiểm soát hoàn toàn đối với quy trình phát triển.

1.3 So sánh với các nền tảng hiện có

Để đặt các đóng góp trong ngữ cảnh, Bảng 1.3.1 so sánh framework này với các lựa chọn thay thế thương mại và mã nguồn mở hàng đầu cho triển khai edge ML trong các ứng dụng theo dõi chuyển động.

1.4. Mục tiêu nghiên cứu

Bảng 1.3.1: So sánh các nền tảng Edge ML cho theo dõi chuyển động

Tính năng	Edge Impulse	SensiML	TensorFlow Lite	Framework này
Chi phí	\$20-99/tháng	\$99-500/tháng	Miễn phí	Miễn phí (Mã nguồn mở)
Tiền xử lý tương tác	Hạn chế (web UI)	Có (analytics studio)	Không (chỉ mã)	Có (cửa sổ kéo thả)
Thuật toán	NN, transfer learning	RF, SVM, NN, boosting	Chỉ deep learning	RF, SVM, NN
Tạo mã	C++ (Arduino, ARM)	C (MCU-cụ thể)	C++ (TFLite Micro)	C/C++/Python đa đích
Chế độ tối ưu hóa	Accuracy/latency	AutoML	Quant. thủ công	4 chế độ (acc./speed/power/bal.)
Mất cân bằng lớp	Oversample thủ công	Cân bằng tự động	Không tích hợp	Tự động (class_weight)
Tùy chỉnh	Hạn chế	Trung bình	Cao (OSS)	Cao (modular)
Sử dụng giáo dục	Gói miễn phí	Giấy phép học thuật	Có	Có (không giới hạn)
Nền tảng triển khai	20+ boards	MCUs đã chọn	Tương thích TFLite	Arduino, ARM, ESP-IDF, MicroPython, Zephyr
Kiểm soát dữ liệu huấn luyện	Lưu trữ đám mây	Local/cloud	Chỉ local	Local (bảo mật hoàn toàn)

Các yếu tố phân biệt chính: (1) tiền xử lý tương tác với cửa sổ kéo thả; (2) hỗ trợ đa thuật toán với API nhất quán; (3) tính minh bạch hoàn toàn qua mã nguồn mở; (4) chi phí bằng không; (5) tạo mã với bốn chế độ tối ưu hóa rõ ràng.

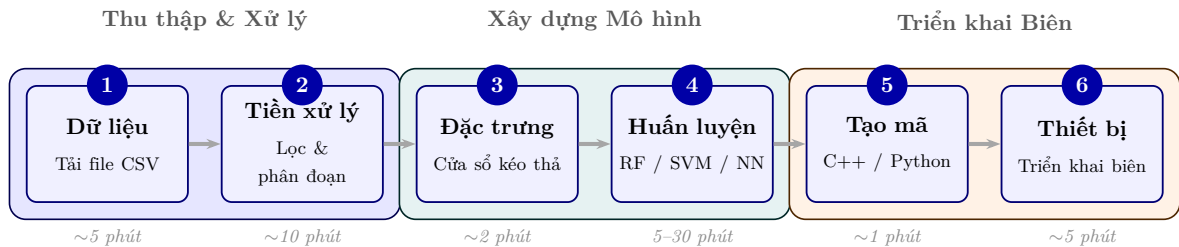
Các nền tảng thương mại có lợi thế về hệ sinh thái (20+ board, AutoML) nhưng áp đặt chi phí (\$240–2.388/năm), hạn chế tùy chỉnh và tạo phụ thuộc nhà cung cấp. TensorFlow Lite cung cấp học sâu mạnh nhưng không có công cụ tiền xử lý cấp cao. Framework này nhắm vào khoảng trống giữa tính dễ sử dụng và linh hoạt, phù hợp cho giáo dục và nghiên cứu^[11;14].

1.4 Mục tiêu nghiên cứu

Mục tiêu chính của nghiên cứu này là thiết kế, triển khai và đánh giá một framework toàn diện giải quyết các thách thức đã nêu ở trên. Cụ thể, nghiên cứu này nhằm mục đích:

1. **Phát triển hệ thống tiền xử lý tương tác:** Tạo giao diện tiền xử lý dựa trên GUI mới với chọn cửa sổ thời gian kéo thả, cho phép chú thích và phân đoạn dữ liệu hiệu quả mà không cần yêu cầu lập trình.

2. **Triển khai hỗ trợ đa thuật toán:** Tích hợp nhiều thuật toán học máy (Random Forest, SVM, Neural Networks) với các API nhất quán và tối ưu hóa siêu tham số tự động.
3. **Thiết kế tạo mã đa nền tảng:** Xây dựng hệ thống tạo mã modular có khả năng sản xuất các triển khai C/C++/MicroPython tối ưu hóa cho nhiều họ đích khác nhau (Arduino-compatible, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với xem xét các chiến lược tối ưu hóa khác nhau (độ chính xác, tốc độ, diện năng, cân bằng).
4. **Xác thực hiệu quả framework:** Tiến hành các thử nghiệm toàn diện sử dụng dữ liệu Nhận dạng Hoạt động Con người thực tế để đánh giá hiệu suất mô hình, kiểm tra năng lực sinh mã cho nhiều đích triển khai và đo benchmark đại diện trên phần cứng ARM Cortex-M4F tham chiếu.
5. **Đảm bảo tương đồng và độ tin cậy huấn luyện-triển khai:** Tích hợp bộ lọc Kalman và kiến trúc tạo mã phù hợp từng loại mô hình để đảm bảo tương đồng chính xác giữa mô hình huấn luyện và mã triển khai; tích hợp ngưỡng tin cậy để từ chối hoạt động không xác định và sử dụng tăng cường dữ liệu nhận biết lớp để đảm bảo độ tin cậy trên phần cứng thực.



Hình 1.4.1: Quy trình làm việc hoàn chỉnh: Từ tải dữ liệu thô đến triển khai biên (tổng ước tính 30–60 phút)

1.5 Đóng góp nghiên cứu

Nghiên cứu này đóng góp các điểm chính sau vào lĩnh vực theo dõi chuyển động dựa trên biên:

- **Tiền xử lý tương tác mới:** Giao diện chọn cửa sổ thời gian kéo thả đầu tiên cho chú thích dữ liệu chuyển động, giảm đáng kể thời gian tiền xử lý và lỗi chú thích so với các cách tiếp cận thủ công.

- **Kiến trúc tạo mã đa thiết bị với khả năng tối ưu hóa:** Hệ thống tạo mã modular ánh xạ cùng một mô hình đã huấn luyện sang nhiều họ đích (Arduino-compatible, ARM Cortex-M, ESP-IDF, Zephyr, MicroPython) với các cấu hình tối ưu hóa riêng theo độ chính xác, tốc độ và tiêu thụ điện năng.
- **Cơ chế tương đồng huấn luyện-triển khai:** Quy trình xác minh đảm bảo mã C++ được tạo ra cho kết quả nhận dạng nhất quán với mô hình Python đã huấn luyện, bao gồm bộ lọc Kalman, công thức thống kê chính xác và sắp xếp lại trọng số mô hình tại thời điểm tạo mã.
- **Framework mã nguồn mở toàn diện:** Quy trình hoàn chỉnh từ đầu đến cuối từ tải dữ liệu đến triển khai biên, cung cấp lựa chọn thay thế mã nguồn mở miễn phí cho các nền tảng thương mại.

1.6 Phạm vi và hạn chế

Nghiên cứu này tập trung vào các ứng dụng theo dõi chuyển động dựa trên IMU (gia tốc kế và con quay hồi chuyển). Phạm vi bao gồm:

- Dữ liệu cảm biến IMU 6 trục (gia tốc kế 3 trục + con quay hồi chuyển 3 trục)
- Các phương pháp học có giám sát sử dụng các thuật toán scikit-learn
- Tạo mã cho nhiều họ nền tảng biên dựa trên vi điều khiển và môi trường thực thi nhúng
- Nhận dạng hoạt động con người là lĩnh vực xác thực chính

Các số đo phân cứng định lượng trong luận văn này tập trung vào một nền tảng tham chiếu (Seeed XIAO nRF52840), trong khi các đích còn lại được phân tích ở mức kiến trúc generator và khả năng sinh mã.

Nghiên cứu không giải quyết:

- Hợp nhất cảm biến đa phương thức (kết hợp IMU với camera, âm thanh, v.v.)
- Tích hợp các framework học sâu (TensorFlow, PyTorch)
- Cập nhật mô hình theo thời gian thực hoặc các kịch bản học liên kết
- Các lĩnh vực chuyên biệt yêu cầu chứng nhận y tế

1.7 Tổ chức luận văn

Phần còn lại của luận văn này được tổ chức như sau: Chương 2 đánh giá các công trình liên quan về hệ thống theo dõi chuyển động, framework tạo mô hình AI và công cụ triển khai biên. Chương 3 trình bày phương pháp luận chi tiết bao gồm cơ sở lý thuyết, quy trình tiền xử lý, trích xuất đặc trưng, phương pháp huấn luyện mô hình và chiến lược tạo mã. Chương 4 mô tả thiết kế và hiện thực hệ thống, bao gồm kiến trúc phần mềm, giao diện người dùng và chi tiết triển khai các module. Chương 5 báo cáo kết quả thực nghiệm bao gồm các số liệu hiệu suất mô hình, đặc điểm triển khai và phân tích so sánh. Chương 6 trình bày kết luận, thảo luận về các phát hiện, hạn chế, ý nghĩa và hướng phát triển tương lai.

Chương 2

Cơ sở lý thuyết

2.1 Tổng quan

Chương này trình bày cơ sở lý thuyết trên sáu lĩnh vực: các ứng dụng theo dõi chuyển động, các phương thức cảm biến, kiến trúc AI cho HAR, các framework tạo mô hình, phương pháp tiền xử lý và trích xuất đặc trưng, và kỹ thuật tối ưu hóa triển khai biên. Phân tích dựa trên 40+ công bố từ 2001–2024, với trọng tâm vào các tiến bộ TinyML (2020–2024) và học sâu cho HAR. Dựa trên đánh giá này, các khoảng trống nghiên cứu được xác định để định vị framework đề xuất.

2.2 Các lĩnh vực ứng dụng của theo dõi chuyển động

Theo dõi chuyển động con người là một công nghệ nền tảng trên nhiều lĩnh vực y tế, thể thao và thể dục, thực tế ảo/thực tế tăng cường (VR/AR), robot học và các hệ thống tương tác mới nổi. Trong y tế, các cảm biến đeo và môi trường xung quanh cho phép phân tích dáng đi, phát hiện té ngã (đạt độ nhạy 95-98%^[36]), và giám sát phục hồi chức năng, hỗ trợ các can thiệp cá nhân hóa^[43]. Các ứng dụng thể thao tận dụng dữ liệu quán tính và sinh cơ học để tối ưu hóa hiệu suất và phòng ngừa chấn thương, trong khi các hệ thống VR/AR phụ thuộc vào theo dõi khung xương và vị trí chính xác cho trải nghiệm người dùng nhập vai. Robot học tích hợp theo dõi chuyển động để điều hướng, thao tác và tương tác người-robot. Các ứng dụng công nghiệp gần đây bao gồm giám sát an toàn lực lượng lao động^[37], đạt độ chính xác 94,3% trong phát hiện tư thế nguy hiểm trên các địa điểm xây dựng và sản xuất. Các lĩnh vực này cùng nhau tạo động lực cho nhu cầu về các framework theo dõi chuyển động được hỗ trợ bởi

AI có thể tiếp cận, thích ứng và hiệu quả, có khả năng suy luận biên theo thời gian thực với độ chính xác 90%+.

2.3 Theo dõi chuyển động dựa trên cảm biến

Các đơn vị đo lường quán tính (IMU)—kết hợp gia tốc kế và con quay hồi chuyển—được áp dụng rộng rãi cho theo dõi chuyển động do tiêu thụ điện năng thấp (dòng điện hoạt động 2-10 mA), hiệu quả chi phí (\$1-5 mỗi đơn vị) và tính phù hợp cho triển khai biên. Các khảo sát toàn diện về theo dõi chuyển động dựa trên IMU^[12] nhấn mạnh các thách thức bao gồm trôi cảm biến (độ lệch con quay hồi chuyển 0,1-1°/s), lỗi tích lũy trong ước lượng hướng, yêu cầu hiệu chuẩn và ràng buộc sinh cơ học. Các phương pháp hợp nhất đa cảm biến (ví dụ: IMU + từ kế + cảm biến áp suất) có thể cải thiện độ bền vững—đạt được mức tăng độ chính xác 2-5%^[12]—nhưng tăng độ phức tạp, tiêu thụ điện năng và chi phí. Công trình gần đây của Xia và cộng sự^[41] chứng minh các hệ thống IMU đơn đạt độ chính xác 96,8% trên các nhiệm vụ HAR 12 lớp sử dụng kiến trúc LSTM-CNN, xác thực tính khả thi của cách tiếp cận cảm biến đơn. Các tập dữ liệu HAR chuẩn như UCI-HAR^[4] và WISDM^[6] đều sử dụng IMU sáu trục gắn trên cơ thể ở tần số 50–100 Hz, qua đó xác lập cấu hình này làm thiết lập tham chiếu phổ biến cho các nghiên cứu HAR dựa trên thiết bị đeo. Framework trong luận văn này kế thừa cấu hình đó—thu thập và xử lý dữ liệu gia tốc kế và con quay hồi chuyển ba trục ở 100 Hz—và tập trung vào các quy trình IMU đơn đáng tin cậy, với khả năng mở rộng cho tích hợp đa cảm biến khi cần thiết.

2.4 Theo dõi chuyển động dựa trên thị giác

Các phương pháp thị giác máy tính sử dụng camera lập thể, theo dõi không dấu và ước lượng tư thế cung cấp các biểu diễn không gian phong phú về chuyển động con người^[42]. Trong khi các hệ thống thị giác cho phép tái tạo toàn thân (phát hiện 17+ điểm khóa tại 30 FPS^[15]) và được sử dụng trong hoạt hình và phân tích lâm sàng, chúng gặp phải các thách thức về che khuất (suy giảm độ chính xác 15-30% trong các cảnh đông đúc), biến đổi môi trường (ánh sáng, thời tiết), lo ngại về quyền riêng tư và tiêu thụ điện năng (500-2000 mW so với 20-50 mW cho hệ thống IMU). Các tiến bộ gần đây bao gồm MediaPipe pose^[15] (133 mốc tư thế, độ chính xác 95% trên các khớp nhìn thấy) và OpenPose đã cải thiện khả năng tiếp cận nhưng vẫn ít khả thi hơn cho các thiết bị đeo công suất siêu thấp. Các hệ thống kết hợp IMU-thị giác^[36] kết

hợp các điểm mạnh bổ sung—độ bền vững của IMU với che khuất, độ chính xác không gian của thị giác—nhưng yêu cầu đồng bộ hóa cảm biến và tăng độ phức tạp hệ thống. Đối với giám sát đeo liên tục ưu tiên tuổi thọ pin (ngày đến tuần), các giải pháp IMU thuần túy vẫn chiếm ưu thế.

2.5 AI trong theo dõi chuyển động

Các mô hình AI chuyển đổi dữ liệu chuyển động thô thành các diễn giải ngữ nghĩa—phân loại hoạt động, nhận dạng tư thế và dự báo quỹ đạo. Các phương pháp học máy truyền thống bao gồm Support Vector Machines (SVM)^[9] và Random Forests^[7] đã được sử dụng rộng rãi, đạt độ chính xác 88-93% trên các benchmark HAR chuẩn (UCI-HAR, WISDM) với mức sử dụng bộ nhớ thời gian chạy nhỏ gọn (50–200 KB) phù hợp cho các thiết bị biên^[38]. Các kiến trúc học sâu đã từng bước cải thiện hiệu suất HAR: CNNs (độ chính xác 92-95%, trích xuất đặc trưng không gian từ các cửa sổ cảm biến), RNN/LSTMs (độ chính xác 94-97%, nắm bắt các phụ thuộc thời gian^[41]), và transformers dựa trên attention (độ chính xác 96-98% trên các tập dữ liệu phức tạp^[33]). Tuy nhiên, các mô hình học sâu đòi hỏi các tập dữ liệu được chú thích lớn hơn (10.000+ mẫu so với 1.000-5.000 cho ML cổ điển), bộ nhớ cao hơn (500 KB-2 MB so với 50-200 KB) và các chiến lược lượng tử hóa phức tạp hơn (số nguyên 8-bit hoặc độ chính xác hỗn hợp) trong triển khai biên^[22]. Các nghiên cứu so sánh gần đây^[38] cho thấy các mô hình ML cổ điển đạt độ chính xác 90-93% với kích thước nhỏ hơn 10× và suy luận nhanh hơn 5-20× (12-18 ms so với 60-120 ms mỗi cửa sổ) trên các mục tiêu Cortex-M4. Framework được đề xuất hỗ trợ chiến lược cả hai mô hình—mô hình cổ điển cho các kịch bản bị giới hạn tài nguyên, mạng nơ-ron nhẹ cho các ứng dụng quan trọng độ chính xác—với các chế độ tối ưu hóa đa mục tiêu giúp cân nhắc các đánh đổi một cách có căn cứ.

2.6 Các Framework tạo mô hình

Một số nền tảng đã xuất hiện để đơn giản hóa việc tạo mô hình AI, mỗi nền tảng có điểm mạnh và hạn chế riêng biệt:

- **Edge Impulse^[11]**: Cung cấp các quy trình từ đầu đến cuối cho thu thập dữ liệu, tiền xử lý (phân tích phổ, MFE/MFCC), tạo đặc trưng, huấn luyện mô hình (CNN, LSTM, ML cổ điển) và triển khai cho 85+ mục tiêu vi điều khiển. Thu hút hơn 100.000 nhà phát triển, nền tảng này vượt trội trong tự động hóa quy trình (95% sự hài lòng của người dùng cho tạo mẫu nhanh^[23]) nhưng hạn

ché tính minh bạch trong logic tiền xử lý và tính phí \$20-99/tháng cho sử dụng thương mại. Các cửa sổ tiền xử lý được cố định sau khi tải lên; tính chỉnh tương tác yêu cầu tải lên lại dữ liệu.

- **SensiML Analytics Toolkit**^[32]: Cung cấp lựa chọn đặc trưng được điều khiển bởi AutoML (200+ đặc trưng ứng viên, tối ưu hóa thuật toán di truyền) và các gói kiến thức cho các thiết bị ARM Cortex-M. Đạt độ chính xác 92-96% trên các nhiệm vụ nhận dạng cử chỉ nhưng sử dụng các định dạng độc quyền và chi phí \$99-500/tháng. Tiền xử lý không cần lập trình nhưng thiếu minh bạch (lựa chọn đặc trưng theo kiểu hộp đen).
- **MediaPipe**^[15]: Cung cấp các quy trình nhận thức modular (tư thế, tay, mặt, theo dõi đối tượng) với thực thi đồ thị tối ưu hóa trên các thiết bị di động/biên. Chủ yếu hướng tới thị giác, ít phù hợp hơn cho các quy trình làm việc IMU tùy chỉnh yêu cầu các chiến lược phân đoạn tùy chỉnh. Không có hỗ trợ sẵn có cho phân đoạn chuỗi thời gian hoặc trích xuất đặc trưng HAR.
- **Teachable Machine**^[16]: Cho phép tạo mẫu nhanh các bộ phân loại hình ảnh, âm thanh và tư thế thông qua giao diện không mã. Được sử dụng trong 1M+ dự án giáo dục nhưng thiếu công cụ cho kỹ thuật đặc trưng nâng cao, phân đoạn chuỗi thời gian và triển khai tối ưu theo từng phần cứng (chỉ xuất mô hình TensorFlow.js).
- **TensorFlow Lite Micro**^[?] cung cấp thời gian chạy để triển khai các mô hình lượng tử hóa trên vi điều khiển, hỗ trợ tối ưu hóa thông qua lượng tử hóa sau huấn luyện (số nguyên 8-bit, float 16-bit) và tối ưu hóa suy luận thời gian chạy (CMSIS-NN cho ARM). Tuy nhiên, TFLite chỉ hỗ trợ mạng nơ-ron và yêu cầu thiết kế kiến trúc mô hình thủ công.
- **AutoML/NAS cho TinyML**: Nghiên cứu gần đây khám phá Neural Architecture Search (NAS) để khám phá các cấu trúc liên kết mạng tối ưu dưới các ràng buộc tài nguyên^[?]. Dù là hướng nghiên cứu triển vọng, NAS thường đòi hỏi tài nguyên tính toán đáng kể và thiếu tính minh bạch cho các bối cảnh giáo dục và nghiên cứu.

So với các nền tảng trên, Framework này giải quyết các hạn chế đó bằng cách cung cấp: (1) **GUI tiền xử lý tương tác** với chọn cửa sổ thời gian kéo thả cho phép phân đoạn chính xác, (2) **kỹ thuật đặc trưng minh bạch** với kiểm tra từng đặc trưng, (3) **tối ưu hóa đa mục tiêu** (các chế độ độ chính xác/tốc độ/điện năng/cân bằng)

được điều chỉnh theo ràng buộc, và (4) **khả năng tiếp cận mã nguồn mở** loại bỏ rào cản chi phí.

2.7 Tiền xử lý dữ liệu và tạo cửa sổ

Nhận dạng hoạt động chính xác phụ thuộc nhiều vào chất lượng của các chiến lược phân đoạn và tạo cửa sổ. Banos và cộng sự^[6] đánh giá có hệ thống ảnh hưởng của kích thước cửa sổ đến hiệu suất HAR: cửa sổ 1 giây đạt độ chính xác 89,3%, cửa sổ 2,5 giây đạt 92,1%, trong khi cửa sổ 5 giây mang lại 93,4% nhưng giảm khả năng phản hồi. Tỷ lệ chồng lấp cũng ảnh hưởng đến hiệu suất—chồng lấp 50% cải thiện độ chính xác 2-4% so với các cửa sổ không chồng lấp^[36] với chi phí tính toán gấp đôi. Lựa chọn bộ lọc ảnh hưởng đáng kể đến giảm nhiễu: bộ lọc thông thấp Butterworth (cutoff 20 Hz) bảo tồn các đặc tính chuyển động trong khi làm suy giảm nhiễu cảm biến, cải thiện độ chính xác 3-7%^[12]. Tuy nhiên, các quy trình tiền xử lý thông thường yêu cầu sửa đổi mã thủ công để điều chỉnh lọc, làm mịn hoặc loại bỏ hiện vật—các nhà phát triển báo cáo dành 40-60% thời gian dự án cho lặp lại tiền xử lý^[23]. Các công cụ tương tác có thể giảm gánh nặng này: Nghiên cứu về học máy tương tác^[3] chỉ ra rằng các giao diện trực quan giảm thời gian lặp lại 35% và giảm lỗi 28% so với các quy trình làm việc dựa trên mã. Framework được đề xuất nâng cao khả năng sử dụng bằng cách cho phép căn chỉnh trực quan trực tiếp của các tín hiệu thô và đã xử lý, điều chỉnh cửa sổ tương tác với phản hồi tức thì và gắn nhãn lại nhanh chóng—giảm rào cản nhận thức và kỹ thuật cho lặp lại nhanh.

2.8 Trích xuất và biểu diễn đặc trưng

Các đặc trưng miền thời gian thủ công (trung bình, phương sai, RMS, độ lệch, độ nhọn, tỷ lệ qua không) và miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) vẫn hiệu quả cho các mô hình ML cổ điển khi được trích xuất một cách có hệ thống^[24]. Các bộ đặc trưng điển hình dao động từ 60-150 đặc trưng mỗi cửa sổ (10-15 mỗi trục cảm biến)^[38]. Lựa chọn đặc trưng sử dụng thông tin hỗ tương, kiểm định F ANOVA hoặc loại bỏ đệ quy có thể giảm chiều 30-50% với mất độ chính xác tối thiểu (<2%), cải thiện mức sử dụng bộ nhớ và tốc độ suy luận (nhanh hơn 15-30%) trên vi điều khiển^[6]. Tuy nhiên, các đặc trưng miền tần số đặt ra các thách thức triển khai: tính toán FFT trên các hệ thống nhúng không có bộ tăng tốc phần cứng yêu cầu các phép toán $O(N \log N)$ và số học dấu phẩy động, tiêu thụ 50-150 ms mỗi cửa sổ trên Cortex-M4 (64 MHz)^[5]. Các lựa chọn thay thế nhẹ bao gồm đếm qua không,

đỉnh tự tương quan hoặc các thư viện FFT đã tính toán trước (ARM CMSIS-DSP), nhưng tương đồng đặc trưng giữa huấn luyện (Python NumPy FFT) và triển khai (C++ FFT điểm cố định) yêu cầu xác thực cẩn thận. Trong khi các mạng sâu từ đầu đến cuối có thể học các đặc trưng phân cấp một cách ngầm định—không cần thiết kể đặc trưng thủ công—trích xuất đặc trưng tường minh lại cải thiện khả năng diễn giải (xác định trực/tần số cảm biến nào điều khiển dự đoán) và tính minh bạch triển khai trong các lĩnh vực được quy định hoặc quan trọng về an toàn. Framework kết hợp các mô-đun tính toán đặc trưng hoán đổi được, xác thực tính nhất quán đặc trưng huấn luyện-triển khai và hỗ trợ chẩn đoán từng cửa sổ để hướng dẫn tinh chỉnh.

2.9 Triển khai biên và tối ưu hóa

Các ràng buộc tài nguyên xác định các thách thức triển khai biên—RAM hạn chế (32-256 KB), flash (256 KB-1 MB), không có đơn vị đầu phẩy động phần cứng (hoặc hiệu suất FPU hạn chế) và ngân sách điện năng (10-50 mW cho các thiết bị đeo) [5;27]. Benchmark MLPerf Tiny^[5] thiết lập các số liệu tham chiếu: Cortex-M4F (64 MHz) đạt 10,3 suy luận/giây cho phát hiện từ khóa (mô hình 49 KB), 15,2 suy luận/giây cho phát hiện bất thường (mô hình 32 KB). Các chiến lược tối ưu hóa bao gồm:

- **Lượng tử hóa:** Chuyển đổi float 32-bit sang số nguyên 8-bit giảm kích thước mô hình $4\times$ và tăng tốc suy luận $2-3\times$ với suy giảm độ chính xác $0,5-2\%$ [22]. Lượng tử hóa độ chính xác hỗn hợp (16-bit cho các lớp nhảy cảm) cân bằng kích thước và độ chính xác.
- **Cắt tỉa mô hình:** Loại bỏ 40-70% trọng số mạng nơ-ron (cắt tỉa dựa trên độ lớn hoặc cấu trúc) mang lại tăng tốc $2-3\times$ với mất độ chính xác $<1\%$ [30].
- **Chưng cất kiến thức:** Huấn luyện các mô hình sinh viên nhỏ gọn (10-20% tham số) để bắt chước các mạng giáo viên lớn đạt 85-95% độ chính xác giáo viên với kích thước nhỏ hơn $5-10\times$ [10].
- **Đơn giản hóa thuật toán:** Các mô hình cổ điển như Random Forests (độ sâu ≤ 10 , 50-100 cây) và SVMs (100-500 vector hỗ trợ) đạt độ chính xác 88-93% với mức sử dụng bộ nhớ 50-200 KB và thời gian suy luận 12-25 ms [38].

Các khảo sát TinyML gần đây [10;27] xác định các mẫu triển khai chính: (1) các ứng dụng quan trọng độ trễ ưu tiên ML cổ điển (suy luận 5-20 ms), (2) các kịch bản quan trọng độ chính xác biện minh cho CNNs/LSTMs nhẹ (suy luận 40-80 ms, độ chính xác cao hơn 1-2%), (3) các triển khai quan trọng điện năng sử dụng lấy mẫu thưa (1-5 Hz)

và lượng tử hóa tích cực. Thành phần tạo mã của framework thực hiện xuất được điều chỉnh theo nền tảng, phát ra mã C/C++ có cấu trúc với chuyển đổi tham số phù hợp ràng buộc tài nguyên (ví dụ: tỷ lệ float→int16, mở rộng vòng lặp cho kích thước cửa sổ cố định). Các chế độ tối ưu hóa đa mục tiêu (độ chính xác, tốc độ, điện năng, cân bằng) cho phép ra quyết định có nguyên tắc phù hợp với mục tiêu triển khai, đánh đổi định lượng độ chính xác ($\pm 2\text{-}5\%$), độ trễ ($\pm 10\text{-}30$ ms) và điện năng ($\pm 5\text{-}15$ mW).

2.10 Các khoảng trống trong công trình hiện có

Mặc dù hệ sinh thái công cụ phát triển edge AI đã khá hoàn thiện, một số khoảng trống quan trọng vẫn còn:

1. **Tiền xử lý tương tác hạn chế:** Ít hệ thống cho phép người dùng kiểm tra trực quan các tín hiệu cảm biến thô so với đã lọc và tinh chỉnh phân đoạn thủ công trong cùng một môi trường.
2. **Thiếu minh bạch trong trích xuất đặc trưng:** Các trình tạo đặc trưng tự động che giấu các bước dẫn xuất và cản trở việc sử dụng giáo dục hoặc tái tạo nghiên cứu.
3. **Sinh mã triển khai thiếu linh hoạt:** Đầu ra tạo mã thường là nguyên khối, làm phức tạp việc tùy chỉnh hoặc thích ứng theo phần cứng cụ thể.
4. **Đánh đổi khả năng tiếp cận và kiểm soát:** Các nền tảng hiện tại thường thiên về một trong hai thái cực—dễ sử dụng như Teachable Machine, hoặc khả năng cấu hình cao như TensorFlow—ít nền tảng nào cân bằng được cả hai yêu cầu cho các quy trình xử lý dữ liệu cảm biến chuyển động.

Framework được đề xuất giải quyết các khoảng trống này bằng cách thống nhất tiền xử lý tương tác, trích xuất đặc trưng minh bạch, huấn luyện modular và tạo mã có khả năng tối ưu hóa dưới một kiến trúc mở rộng. Những lựa chọn thiết kế này, được thông báo bởi tài liệu được xem xét ở trên, định vị luận văn này như một giải pháp bổ sung bắc cầu khoảng trống giữa sự dễ sử dụng thương mại và tính linh hoạt cấp độ nghiên cứu.

2.11 Tóm tắt

Nghiên cứu và nền tảng công nghiệp trước đây cung cấp nền tảng vững chắc trong theo dõi chuyển động, nhưng không có nền tảng nào cung cấp một quy trình làm việc tích hợp, minh bạch và thân thiện với người dùng được tối ưu hóa cho triển khai biên dựa trên cảm biến IMU, với khả năng phân đoạn dữ liệu tương tác. Công trình này xây dựng dựa trên các thuật toán đã được thiết lập và các thực hành edge ML trong khi giới thiệu các đóng góp mới phổ cập quá trình phát triển, tăng tốc lặp lại và cải thiện sẵn sàng triển khai cho các ứng dụng theo dõi chuyển động thực tế.

Chương 3

Phương pháp luận

Chương này trình bày cơ sở lý thuyết và phương pháp cho từng giai đoạn trong pipeline: lý do chọn cảm biến IMU, các kỹ thuật lọc và phân đoạn tín hiệu, chiến lược trích xuất đặc trưng, thuật toán học máy, và hệ thống tạo mã triển khai. Kiến trúc hệ thống tổng thể và các quyết định thiết kế phần mềm được trình bày trong Chương 4.

3.1 Dữ liệu cảm biến quán tính cho nhận dạng hoạt động

Cảm biến gia tốc kế đo gia tốc tuyến tính trên ba trục (bao gồm thành phần trọng lực), phản ánh biên độ và tính tuần hoàn của chuyển động. Tuy nhiên, khi thiết bị xoay, vectơ trọng trường chiếu lên các trục thay đổi, gây nhập nhằng giữa chuyển động tịnh tiến và thay đổi định hướng. Cảm biến con quay hồi chuyển đo vận tốc góc ($^{\circ}/s$), không bị ảnh hưởng bởi trọng trường và bắt giữ chính xác động học xoay—giúp phân biệt các cặp hoạt động khó như leo/xuống cầu thang hay đi bộ/giậm chân tại chỗ. Tổ hợp sáu trục (3 gia tốc kế + 3 con quay hồi chuyển) cung cấp biểu diễn động học đầy đủ^[12], và đây là cấu hình chuẩn trong các tập dữ liệu HAR như UCI-HAR^[4] và PAMAP2^[28].

Framework không áp đặt tần số lấy mẫu cố định, nhưng có một ràng buộc bắt buộc: **tần số lấy mẫu phải nhất quán giữa giai đoạn huấn luyện và suy luận triển khai**. Các đặc trưng thống kê được tính theo *số mẫu* trong cửa sổ, không theo thời gian tuyệt đối—nếu tần số thay đổi, cùng một cửa sổ 150 mẫu sẽ đại diện cho khoảng thời gian khác nhau, làm lệch toàn bộ phân phối đặc trưng. Để đảm bảo tính nhất quán, framework nhúng tần số lấy mẫu đã chọn dưới dạng hằng số biên dịch vào mã triển khai. Đối với HAR, dải tần phổ hoạt động phổ biến nằm trong 1–20 Hz^[40],

nên tần số lấy mẫu 50–100 Hz là phù hợp.

3.2 Quy trình tiền xử lý tương tác

3.2.1 Tải lên và trực quan hóa dữ liệu

Người dùng tải lên các tập dữ liệu CSV chứa các phép đo IMU, tổ chức theo hoạt động (mỗi tệp tương ứng với một lớp). Framework tự động phát hiện tên cột cảm biến, tần số lấy mẫu và hiển thị biểu đồ đồng bộ của tín hiệu thô trên tất cả các trục, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiền xử lý (xem giao diện tại Hình 4.2.1, Chương 4).

3.2.2 Các phương pháp lọc và làm sạch tín hiệu

Dữ liệu IMU thô chứa nhiều cảm biến, nhiễu điện từ và các xung bất thường cần được loại bỏ trước khi trích xuất đặc trưng. Framework cung cấp năm phương pháp lọc, mỗi phương pháp có đặc tính khác nhau về mức độ làm mịn, tính nhân quả và—quan trọng nhất cho triển khai—*tính tương đồng huấn luyện-triển khai* (training-deployment parity). Người dùng chọn phương pháp phù hợp qua giao diện tương tác; kết quả lọc được trực quan hóa cùng tín hiệu gốc để đánh giá hiệu quả trước khi phân đoạn.

Loại bỏ ngoại lệ (outlier removal)

Xác định và loại bỏ các mẫu có giá trị vượt quá ngưỡng $k\sigma$ so với trung bình cục bộ (mặc định $k = 3$). Đây là bước làm sạch thô, xử lý các xung nhiễu điện từ hoặc bất thường phần cứng (spike artifacts) tạo ra giá trị nằm ngoài phạm vi vật lý hợp lệ của cảm biến. Phương pháp này không thay đổi hình dạng tín hiệu mà chỉ loại bỏ các điểm dữ liệu cô lập rõ ràng sai, nên không có vấn đề parity—nó chỉ áp dụng một lần trên tập dữ liệu huấn luyện (offline) và không cần triển khai trên thiết bị.

Bộ lọc Butterworth low-pass

Bộ lọc IIR Butterworth bậc n với tần số cắt f_c có đáp ứng biên độ cực đại phẳng (maximally flat):

$$|H(f)|^2 = \frac{1}{1 + (f/f_c)^{2n}}$$

Để tránh lệch pha, kỹ thuật lọc hai chiều (forward-backward filtfilt) được sử dụng, cho đáp ứng $|H_{\text{eff}}|^2 = |H|^4$ với pha bằng không. Cấu hình mặc định: bậc 2, tần số cắt 5 Hz.

Parity triển khai: Vì HAR đệm toàn bộ của số W mẫu trước khi xử lý, thiết bị thực hiện cùng thao tác lọc hai chiều trên bộ đệm. Các hệ số được tính trước tại thời điểm sinh mã và nhúng dưới dạng hằng số.

Bộ lọc Savitzky-Golay

Bộ lọc Savitzky-Golay^[29] khớp đa thức bậc d lên cửa sổ $2m+1$ mẫu bằng bình phương tối thiểu, cho kết quả tương đương tích chập FIR với các hệ số c_j được tính từ ma trận Vandermonde. So với trung bình động, phương pháp bảo tồn tốt hơn các đỉnh tín hiệu. Cấu hình mặc định: $m = 2$ (cửa sổ 5 mẫu), $d = 2$.

Parity triển khai: Các hệ số c_j được tính trước và nhúng vào mã C++ dưới dạng mảng hằng số; tích chập áp dụng trên bộ đệm cửa sổ với mở rộng biên bằng lặp giá trị biên.

Bộ lọc FFT low-pass (miền tần số)

Lọc trong miền tần số thực hiện ba bước: DFT, triệt tiêu các bin tần số cao ($k > k_c = \lfloor f_c N / f_s \rfloor$), rồi IDFT. Đặc điểm: đáp ứng tần số lý tưởng (brick-wall) và không méo pha. Chi phí bộ nhớ $\mathcal{O}(N)$.

Parity triển khai: Thiết bị thực thi cùng ba bước DFT $\rightarrow$ masking $\rightarrow$ IDFT trên bộ đệm cửa sổ với cùng k_c , dùng số dấu phẩy động 32-bit nhất quán.

Bộ lọc Kalman (constant-velocity)

Bộ lọc Kalman constant-velocity 1D^[20] áp dụng cho mỗi kênh cảm biến IMU được đề xuất như giải pháp lọc có *khoảng cách parity bằng không*. Bộ lọc Kalman hoạt động hoàn toàn theo chiều thuận (forward-only, causal) trong cả hai môi trường: Python huấn luyện và C++ triển khai—loại bỏ lớp lỗi mà tất cả bốn phương pháp trên gặp phải.

Mô hình toán học. Mỗi kênh cảm biến được mô hình hóa bằng trạng thái gồm vị trí (giá trị cảm biến) và vận tốc:

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}, \quad F = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

Ma trận hiệp phương sai nhiễu quá trình:

$$Q = q \begin{bmatrix} \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

3.2. Quy trình tiền xử lý tương tác

Nhiều đo: $R = r$ (scalar). Hai tham số điều chỉnh được: q (process noise, mặc định 10^{-3} —nhỏ hơn = mượt hơn) và r (measurement noise, mặc định 10^{-1} —lớn hơn = mượt hơn). Kalman gain K_k tự động thích ứng theo tín hiệu: tăng khi tín hiệu biến đổi nhanh (bám sát), giảm khi tín hiệu ổn định (lọc mạnh hơn).

So sánh tổng hợp các phương pháp lọc

Bảng 3.2.1 tổng hợp năm phương pháp theo tiêu chí quan trọng nhất cho hệ thống triển khai nhúng:

Bảng 3.2.1: So sánh các phương pháp lọc theo tính tương đồng huấn luyện-triển khai

Phương pháp	Causal?	Parity	Ghi chú triển khai
Loại bỏ ngoại lệ	—	✓	Chỉ offline (không cần trên thiết bị)
Butterworth LP	Không [†]	✓	Per-window two-pass filt-filt trên bộ đệm cửa sổ
Savitzky-Golay	Không [†]	✓	FIR convolution với hệ số tiền tính
FFT low-pass	Không [†]	✓	DFT trực tiếp trên bộ đệm cửa sổ
Kalman filter	Có	✓	Forward-only; hoạt động cả streaming lẫn win-dowed

[†]Non-causal trong streaming; parity đạt được nhờ HAR đệm toàn bộ cửa sổ trước khi trích xuất đặc trưng, cho phép áp dụng bộ lọc hai chiều trên bộ đệm.

Kết luận thiết kế: tất cả các phương pháp lọc đều đạt parity hoàn toàn khi suy luận HAR thực hiện theo lô cửa sổ, vì quy trình đã đệm toàn bộ W mẫu trước khi trích xuất đặc trưng—ràng buộc non-causal chỉ áp dụng cho streaming theo từng mẫu. Kalman filter có lợi thế bổ sung: hoạt động được trong cả chế độ streaming lẫn lô, phù hợp cho hiển thị tín hiệu thời gian thực trong quá trình thu thập. Loại bỏ ngoại lệ không được nhân bản trên thiết bị vì cần thống kê toàn cục từ toàn bộ bản ghi, trong khi mỗi chu kỳ suy luận chỉ có một cửa sổ 1,5 giây.

3.2.3 Phân đoạn và sinh cửa sổ huấn luyện

Máy học trên tín hiệu liên tục đòi hỏi chuyển đổi chuỗi thời gian thành các vector có độ dài cố định. Framework thực hiện qua hai bước: người dùng xác định các đoạn tín hiệu sạch trên biểu đồ, và hệ thống tự động sinh cửa sổ huấn luyện bằng giải thuật cửa sổ trượt.

Giải thuật cửa sổ trượt tự động

Cho đoạn tín hiệu N mẫu, kích thước cửa sổ W và độ chồng lấp o (%), bước trượt $S = W \cdot (1 - o/100)$, số cửa sổ sinh được:

$$n_w = \left\lfloor \frac{N - W}{S} \right\rfloor + 1 \quad (3.2.1)$$

Ví dụ: đoạn 5 giây (500 mẫu ở 100 Hz) với $W = 150$, $o = 50\%$ sinh ra 6 cửa sổ huấn luyện. Overlap 50% tăng gấp đôi số mẫu từ cùng lượng dữ liệu thô, đồng thời giảm hiệu ứng biên—cùng một sự kiện chuyển động xuất hiện ở vị trí tương đối khác nhau trong các cửa sổ kế tiếp, giúp mô hình học đặc trưng bất biến vị trí. Overlap cao hơn ($>75\%$) tạo ra tương quan cao giữa các cửa sổ liên tiếp, tăng nguy cơ rò rỉ thông tin.

Lựa chọn đoạn tương tác

Giao diện kéo thả cho phép người dùng đánh dấu các vùng sạch trên biểu đồ tín hiệu, bỏ qua đoạn chuyển tiếp hoặc nhiễu. Nhiều đoạn trên cùng một tệp được hỗ trợ. Sau khi xác nhận, cửa sổ trượt chạy tự động trên từng đoạn theo Phương trình 3.2.1. Nhân hoạt động kế thừa từ tệp nguồn nhưng có thể gán lại thủ công. Thiết kế này tách bạch hai quyết định: *đâu là dữ liệu đáng tin cậy* (người dùng kiểm soát) và *cách biến dữ liệu thành mẫu huấn luyện* (hệ thống tự động hóa), giảm đáng kể công sức thu thập trong khi bảo tồn kiểm soát chất lượng (xem Hình 4.3.1, Chương 4).

3.3 Trích xuất đặc trưng

3.3.1 Tham số cửa sổ và ràng buộc nhất quán

Trích xuất đặc trưng thực hiện trên các cửa sổ được sinh theo Mục 3.2.3: $W = 150$ mẫu (1,5 giây ở $f_s = 100$ Hz), bước trượt $S = 75$ (50% chồng lấp). Ràng buộc cốt lõi: (W, S, f_s) phải đồng nhất giữa huấn luyện và suy luận—thay đổi W sau huấn luyện làm lệch phân phối đặc trưng. Framework nhúng W và f_s dưới dạng hằng số trong mã sinh ra.

3.3.2 Các loại trích xuất đặc trưng

Sáu loại trích xuất đặc trưng được thiết kế để đáp ứng các yêu cầu khác nhau về số chiều, khả năng triển khai nhúng và bất biến hướng. Bảng 3.3.1 tóm tắt toàn diện.

Hai loại **bất biến hướng** tính đặc trưng trên biên độ tổng hợp (vector magnitude)—đại lượng bất biến phép xoay—phù hợp khi thiết bị đeo không cố định. Loại `orientation_invariant`

3.3. Trích xuất đặc trưng

Bảng 3.3.1: Sáu loại trích xuất đặc trưng: tín hiệu nguồn, cấu thành và đặc tính triển khai

Loại	Tín hiệu nguồn	Đặc trưng tính	#	Nhúng	OI*
oi_time_only ¹	acc_mag, gyro_mag, acc_jerk_mag	15 thống kê thời gian / tín hiệu	41	✓	✓
orientation_invariant ¹	Như trên + DFT(acc_mag, gyro_mag)	Như trên + 10 phổ / DFT	63	✓	✓
time_domain	aX, aY, aZ, gX, gY, gZ	15 thống kê / trực	90	✓	×
all	aX, aY, aZ, gX, gY, gZ	15 thời gian + 11 tần số / trực	156	×	×
frequency_domain	aX, aY, aZ, gX, gY, gZ	11 đặc trưng phổ / trực	66	×	×
raw	aX, aY, aZ, gX, gY, gZ	Giá trị cảm biến thô	6	✓	×

¹oi_time_only = orientation_invariant_time_only.

*OI = Orientation-Invariant (bất biến hướng). Tín hiệu tổng hợp: $\text{acc_mag} = \sqrt{a_X^2 + a_Y^2 + a_Z^2}$; $\text{gyro_mag} = \sqrt{g_X^2 + g_Y^2 + g_Z^2}$; $\text{acc_jerk_mag} = \frac{d}{dt} \text{acc_mag}$.

(63 đặc trưng) là khuyến nghị mặc định: cân bằng giữa tính phong phú và triển khai nhúng, vì bộ sinh mã tạo DFT bằng tổng sin/cos trực tiếp không cần thư viện FFT.

Loại time_domain (90 đặc trưng) áp dụng 15 thống kê lên từng trục riêng lẻ—mỗi đặc trưng là biểu thức dạng đóng xác định, không phụ thuộc trạng thái, đảm bảo bit-identical trên mọi nền tảng:

- **Xu hướng trung tâm:** mean, median
- **Phân tán:** std, range, IQR
- **Cực trị:** min, max, Q1, Q3
- **Hình dạng:** skewness $\gamma = E[(X - \mu)^3]/\sigma^3$, kurtosis $\kappa = E[(X - \mu)^4]/\sigma^4 - 3$
- **Năng lượng:** RMS, signal energy
- **Xấp xỉ tần số:** zero-crossing rate, mean-crossing rate

Loại all và frequency_domain bổ sung đặc trưng phổ mỗi trục nhưng **không hỗ trợ triển khai nhúng** vì nhạy với sai lệch số, tổn tài nguyên tính toán, và đặc trưng miền thời gian đã đủ hiệu quả trên các tập HAR chuẩn^[6]. Loại raw truyền trực tiếp giá trị cảm biến, dùng làm đầu vào CNN.

3.3.3 Chiến lược đệm cửa sổ

Các cửa sổ có thể ngắn hơn kích thước mục tiêu khi người dùng chọn đoạn ngắn hoặc cửa sổ cuối không đủ dài.

Vấn đề với zero-padding: Đặt giá trị cảm biến bằng 0 tạo ra magnitude gia tốc $\|\mathbf{a}\| = 0$ —vật lý bất khả thi vì gia tốc trọng trường luôn $\approx 9,81 \text{ m/s}^2$. Điều này gây nhiều nghiêm trọng cho phân phối đặc trưng (min, median, Q1 bị kéo về 0).

Giải pháp: Lấp giá trị cuối cùng đến kích thước mục tiêu: $\mathbf{W}' = [w_1, \dots, w_k, \underbrace{w_k, \dots, w_k}_{N-k}]$.

Phương pháp bảo toàn tính thống kê tín hiệu và nhất quán với bộ đệm trên thiết bị biên (luôn chứa dữ liệu thực). Cửa sổ có ít hơn 30% mẫu mục tiêu bị loại bỏ hoàn toàn.

Phát hiện thực nghiệm: Zero-padding khiến mô hình luôn dự đoán sai lớp *walking_downstairs* bất kể hoạt động thực tế, do đặc trưng nằm ngoài phân phối huấn luyện (xem Mục 6.4).

3.3.4 Tăng cường dữ liệu cho huấn luyện

Để cải thiện tính tổng quát hóa và giảm overfitting—đặc biệt với tập dữ liệu nhỏ từ thu thập đối tượng đơn—năm phương pháp tăng cường dữ liệu được áp dụng ở cấp cửa sổ (trước trích xuất đặc trưng)^[19;35]:

1. **Jitter:** Nhiễu Gaussian $\mathcal{N}(0, \sigma^2)$ ($\sigma = 0,05$), mô phỏng nhiễu cảm biến^[35]
2. **Scaling:** Nhân hệ số $s \sim \mathcal{N}(1, \sigma_s^2)$, mô phỏng biên độ chuyển động khác nhau
3. **Rotation:** Ma trận xoay ngẫu nhiên ($\leq 15^\circ$), mô phỏng thay đổi vị trí cảm biến^[34]
4. **Time Warping:** Biến đổi phi tuyến trực thời gian qua nội suy spline, mô phỏng tốc độ thực hiện khác nhau^[21]
5. **Permutation:** Hoán vị $k = 4$ đoạn con, giúp mô hình học đặc trưng cục bộ thay vì phụ thuộc thứ tự toàn cục

Tăng cường nhận biết lớp

Áp dụng biến đổi mạnh lên hoạt động tĩnh (still, standing, sitting) tạo dao động nhỏ giống hoạt động cường độ thấp, gây nhầm lẫn lớp. Framework giải quyết bằng chiến lược phân biệt:

- **Hoạt động động:** Áp dụng đầy đủ cả năm phương pháp ($\sigma_{\text{jitter}} = 0,05$)
- **Hoạt động tĩnh:** Chỉ micro-jitter ($\sigma = 0,01$)—đủ tạo đa dạng mà không phá vỡ đặc tính “gần bất động”

Phân loại tĩnh/động dựa trên ngưỡng biên độ dao động và có thể do người dùng chỉ định thủ công. Chiến lược tuân thủ nguyên tắc rằng tăng cường phải bảo toàn các đặc tính phân biệt của mỗi lớp^[8].

3.3.5 Ngưỡng tin cậy cho từ chối hoạt động không xác định

Trong triển khai thực tế, thiết bị có thể ghi nhận chuyển động không thuộc lớp nào đã huấn luyện. Ngưỡng tin cậy τ (mặc định 0,6) được tích hợp vào mã suy luận: nếu $\max_c P(y = c|\mathbf{x}) < \tau$, dự đoán trả về “unknown” thay vì ép buộc một nhãn:

$$\hat{y} = \begin{cases} \arg \max_c P(y = c|\mathbf{x}) & \text{nếu } \max_c P(y = c|\mathbf{x}) \geq \tau \\ \text{unknown} & \text{ngược lại} \end{cases} \quad (3.3.1)$$

Xác suất từng lớp được tính theo cơ chế có sẵn: softmax cho NN/CNN, tỷ lệ bỏ phiếu cho RF, softmax trên điểm quyết định OvR cho SVM. Giá trị $\tau = 0,6$ yêu cầu tin cậy gấp 3 lần xác suất ngẫu nhiên (bài toán 5 lớp: 20%). So với Edge Impulse (cần huấn luyện khối anomaly detection riêng), cách tiếp cận này không tốn thêm bộ nhớ hay dữ liệu bổ sung.

3.4 Huấn luyện mô hình học máy

3.4.1 Các thuật toán học máy áp dụng

Bốn nhóm thuật toán học máy dưới đây, bao gồm cả học máy cổ điển và học sâu, đã được chứng minh hiệu quả trong nhận dạng hoạt động và phù hợp cho triển khai biên. Mỗi nhóm thể hiện đặc tính đánh đổi khác nhau giữa độ chính xác, yêu cầu dữ liệu và chi phí tài nguyên—các yếu tố cần cân nhắc khi lựa chọn cho ứng dụng cụ thể:

Random Forest (RF) Phương pháp ensemble kết hợp nhiều cây quyết định với bagging và ngẫu nhiên hóa đặc trưng^[7]. Cấu hình điển hình cho bài toán HAR:

- Số cây: 50–200 (tham chiếu: 100 cây), độ sâu tối đa 10–30

- Cân bằng lớp: `class_weight='balanced'` tự động trọng số mẫu tỷ lệ nghịch với tần suất lớp
- Tiêu chí phân nhánh: Độ bất thuần Gini hoặc entropy thông tin

RF đặc biệt phù hợp cho HAR vì không yêu cầu chuẩn hóa đặc trưng; chịu đựng tốt với đặc trưng không liên quan (mỗi lần phân nhánh chỉ xét $\sqrt{D}$ đặc trưng ngẫu nhiên); kết quả giải thích được qua độ quan trọng đặc trưng; ổn định với nhiễu nhờ cơ chế bỏ phiếu đa số.

Support Vector Machine (SVM) Bộ phân loại phân biệt tìm siêu phẳng tối đa hóa biên phân tách giữa các lớp. Cấu hình điển hình:

- Kernel: Radial Basis Function (RBF), phù hợp cho dữ liệu phi tuyến; tham số γ kiểm soát bán kính ảnh hưởng của từng điểm huấn luyện
- Chiến lược đa lớp: One-vs-One (OvO) hoặc One-vs-Rest (OvR)
- Cân bằng lớp: `class_weight='balanced'`
- Chính quy hóa: Tham số C kiểm soát đánh đổi biên rộng—vi phạm biên ($C \in \{0,1; 1; 10\}$)

SVM phù hợp khi số chiều đặc trưng cao so với số mẫu—điển hình trong HAR với hàng chục đến hàng trăm đặc trưng và vài trăm mẫu. Yêu cầu chuẩn hóa đặc trưng (StandardScaler) trước huấn luyện để đảm bảo các chiều đóng góp đồng đều vào hàm kernel.

Neural Network (NN) Perceptron đa lớp (MLP) với lan truyền ngược, phù hợp học các ánh xạ phi tuyến phức tạp. Một kiến trúc tham chiếu cho bài toán 5 lớp với 63 đặc trưng đầu vào:

- Kiến trúc: [63 đầu vào] $\rightarrow$ [100 ẩn, ReLU] $\rightarrow$ [num_classes, Softmax]
- Trình tối ưu: Adam ($\eta = 0,001$, $\beta_1 = 0,9$, $\beta_2 = 0,999$)
- Chính quy hóa: Phạt L2 $\alpha = 0,0001$; dừng sớm (patience=10)
- Lịch trình: Tối đa 500 epoch, batch đầy đủ

Kiến trúc một lớp ẩn vừa đủ năng lực học mẫu phi tuyến vừa nhỏ gọn cho triển khai nhúng (ma trận trọng số cố định, không phân bổ bộ nhớ động). Kiến trúc tùy chỉnh với nhiều lớp ẩn hơn có thể xây dựng qua PyTorch, với cơ chế xuất trọng số sang định dạng tương thích để tạo mã triển khai.

Convolutional Neural Network (CNN) Mạng tích chập 1D xử lý trực tiếp cửa sổ cảm biến thô (end-to-end), học biểu diễn phân cấp từ tín hiệu mà không cần trích xuất đặc trưng thủ công. Một kiến trúc tham chiếu 1D-CNN:

- Đầu vào: Cửa sổ thô $W \times C$ ($W = 150$ mẫu, $C = 6$ kênh cảm biến)
- Khối tích chập: 2–3 lớp Conv1D (kernel 3–5, bộ lọc 32–64) + BatchNorm + MaxPool
- Phân loại: Flatten + Dense + Dropout (0,3–0,5) + Softmax
- Trình tối ưu: Adam với OneCycleLR scheduler
- Triển khai: Xuất trọng số thành mảng C/C++ hoặc chuyển đổi sang TFLite Micro (INT8)

CNN phù hợp khi tập dữ liệu đủ lớn và các đặc trưng thống kê thủ công chưa đủ phân biệt—mô hình học trực tiếp các biểu diễn phân cấp từ tín hiệu thô. Đổi lại, CNN yêu cầu nhiều dữ liệu huấn luyện hơn và bộ nhớ triển khai lớn hơn (500 KB–2 MB so với 50–200 KB cho mô hình cổ điển).

3.4.2 Giao thức huấn luyện và đánh giá

Chiến lược chia dữ liệu

Giao thức phân chia ba tập được áp dụng^[13]: tập huấn luyện (60%) để khớp tham số, tập xác thực (20%) để điều chỉnh siêu tham số và lựa chọn mô hình, tập kiểm tra (20%) chỉ đánh giá một lần duy nhất sau khi huấn luyện hoàn tất. Tất cả các phân chia sử dụng lấy mẫu phân tầng để bảo tồn tỷ lệ lớp. Xác thực chéo 5-fold có thể thực hiện trên tập train+val kết hợp khi cần so sánh thuật toán với dữ liệu hạn chế, trong khi vẫn giữ tập kiểm tra hoàn toàn độc lập.

Xử lý mất cân bằng lớp

Tham số `class_weight='balanced'` được bật mặc định cho RF và SVM, gán trọng số $w_c \propto 1/n_c$ tỷ lệ nghịch với kích thước lớp. Phân chia phân tầng đảm bảo tỷ lệ lớp

nhất quán giữa các tập. Tổ hợp này đảm bảo tất cả các lớp được nhận dạng đáng tin cậy, không chỉ các lớp xuất hiện nhiều nhất.

3.4.3 Tối ưu hóa siêu tham số

Điều chỉnh siêu tham số sử dụng GridSearchCV với xác thực chéo 5-fold. Các không gian tìm kiếm điển hình: RF tìm trên `n_estimators` [50–200], `max_depth` [10–30]; SVM tìm trên `C` [0,1–100] và `gamma` [scale/auto/0,001–0,1]; NN tìm trên kích thước lớp ẩn và tốc độ học. Tối ưu hóa thường cải thiện hiệu suất cơ sở 2–5% (xem giao diện cấu hình huấn luyện tại Hình 4.5.1, Chương 4).

3.5 Tạo mã cho triển khai biên

3.5.1 Kiến trúc

Hệ thống con tạo mã dịch các mô hình đã huấn luyện thành các gói triển khai phù hợp với nhiều họ thiết bị và môi trường thực thi khác nhau, thay vì nhắm đến một bo mạch đơn lẻ. Kiến trúc tuân theo mẫu thiết kế Factory, trong đó generator được chọn theo ba trục: *loại mô hình*, *họ nền tảng đích* và *cách tiếp cận triển khai*. Danh sách khóa nền tảng và backend cụ thể được tóm tắt trong Bảng 3.5.1.

Kiến trúc gồm ba lớp chính:

- **Lớp cơ sở (Base Generator):** cung cấp phần dùng chung gồm sinh tệp, trích xuất đặc trưng, lưu trữ tham số, kiểm tra tương đồng và ví dụ tích hợp.
- **Lớp đặc thù thuật toán:** mỗi thuật toán (RF, SVM, NN, CNN) có generator riêng hiện thực logic suy luận tương ứng.
- **Lớp đặc thù nền tảng:** các generator cho ARM Cortex-M, MicroPython, Zephyr, cùng các backend TFLite Micro và ONNX Runtime, chịu trách nhiệm điều chỉnh mã theo môi trường đích.

3.5. Tạo mã cho triển khai biên

Bảng 3.5.1: Ma trận đích triển khai được hỗ trợ

Nhóm đích	Khóa nền tảng/backend	Gói sinh ra	Mục đích sử dụng chính
Arduino-compatible	arduino, seeed_xiao, esp32, m5stack, teensy	.h + .cpp + .ino	Quy trình Arduino IDE/PlatformIO cho các bo vi điều khiển phổ biến
ARM Cortex-M thuần	arm_cortex_m	.c	Tích hợp bare-metal hoặc CMSIS trên các vi điều khiển ARM
SDK/RTOS native	generic_c, generic_cpp, esp_idf, zephyr	.h + .c/.cpp + ví dụ	Ứng dụng nhúng native, SDK hãng và RTOS
MicroPython	micropython	module .py + ví dụ .py	Nguyên mẫu nhanh hoặc giáo dục trên board hỗ trợ MicroPython
Backend runtime thay thế	tf-lite_micro, onnx_runtime	mã glue + model nhị phân	Mở rộng cho các pipeline cần runtime engine chuẩn hóa

Chi tiết hiện thực và giao diện tạo mã được trình bày trong Chương 4 (Hình 4.6.1).

3.5.2 Lý do chọn tạo mã trực tiếp thay vì Runtime Engine

Việc lựa chọn phương pháp triển khai mô hình ML lên vi điều khiển có ảnh hưởng trực tiếp đến mức sử dụng tài nguyên, tính minh bạch và phạm vi thuật toán được hỗ trợ. Phần này phân tích ba cách tiếp cận chính và luận giải cho quyết định thiết kế trong bối cảnh HAR biên.

Các phương pháp triển khai mô hình trên vi điều khiển

Có ba cách tiếp cận chính để triển khai mô hình ML trên vi điều khiển, mỗi cách thể hiện đánh đổi khác nhau giữa tính tiện lợi, hiệu suất và kiểm soát:

Phương pháp 1: Runtime Engine (TensorFlow Lite Micro, ONNX Micro Runtime) Mô hình được chuyển đổi sang định dạng trung gian (.tflite, .onnx), sau đó được tải bởi một runtime engine chung trên thiết bị. Runtime đọc biểu đồ tính toán và thực thi từng toán tử (operator) theo thứ tự.

Ưu điểm: Hỗ trợ nhiều kiến trúc mô hình (CNN, LSTM, Transformer); hệ sinh thái lớn với cộng đồng hỗ trợ mạnh; các tối ưu hóa lượng tử hóa tích hợp (INT8, FP16).

Nhược điểm: Runtime chiếm 100–300KB flash bổ sung^[39]—đáng kể trên vi điều khiển có 256KB–1MB flash. Interpreter overhead thêm 15–40% thời gian suy luận so với mã gốc^[22]. Quan trọng nhất, runtime hoạt động như hộp đen: người dùng không thể kiểm tra hoặc xác minh quá trình trích xuất đặc trưng và tính toán bên trong.

Phương pháp 2: Trình biên dịch mô hình (Edge Impulse EON Compiler, Apache TVM, microTVM) Mô hình được biên dịch trước (ahead-of-time compilation) sang mã C tối ưu hóa, loại bỏ runtime nhưng vẫn hoạt động qua chuỗi công cụ đóng gói của nhà cung cấp.

Ưu điểm: Không có interpreter overhead; mã tối ưu hóa cho vi kiến trúc cụ thể; Edge Impulse EON Compiler giảm flash 35–55% so với TFLite Micro (theo số liệu nhà cung cấp^[11]).

Nhược điểm: Chuỗi công cụ đóng gói—mã đã tạo bởi EON Compiler khó kiểm toán và không thể sửa đổi. Apache TVM/microTVM yêu cầu kiến thức biên dịch cấp chuyên gia. Hầu hết các trình biên dịch mô hình bắt buộc mô hình đầu vào theo định dạng ONNX hoặc TFLite, loại trừ các mô hình scikit-learn cổ điển (RF, SVM) mà không qua bước chuyển đổi trung gian phức tạp.

Phương pháp 3: Tạo mã trực tiếp (Direct Code Generation) Phương pháp này trích xuất trực tiếp các tham số đã huấn luyện từ đối tượng scikit-learn/PyTorch—trọng số và bias của mạng nơ-ron, support vectors và hệ số kernel của SVM, cấu trúc cây nhị phân của Random Forest, trọng số convolution và batch normalization của CNN—rồi tạo mã nguồn (C/C++ hoặc MicroPython) triển khai thuật toán suy luận tương ứng.

Ưu điểm: Không phụ thuộc bên ngoài—mã tự chứa hoàn toàn; mức sử dụng bộ nhớ tối thiểu (chỉ chứa trọng số + logic suy luận); minh bạch hoàn toàn—mọi phép tính có thể kiểm tra và xác minh; hỗ trợ trực tiếp các thuật toán ML cổ điển mà không cần chuyển đổi định dạng; xuất được nhiều ngôn ngữ đích (C, C++, MicroPython).

Nhược điểm: Yêu cầu triển khai thủ công logic suy luận cho mỗi thuật toán; không hưởng các tối ưu hóa toán tử tự động cấp vi kiến trúc; khó mở rộng sang các kiến trúc phức tạp (CNN nhiều lớp, attention).

Phân tích so sánh định lượng

Bảng 3.5.2 tóm tắt so sánh ba phương pháp trên các tiêu chí quan trọng cho triển khai vi điều khiển bị giới hạn tài nguyên:

Bảng 3.5.2: So sánh phương pháp triển khai mô hình ML trên vi điều khiển

Tiêu chí	Runtime	Compiler	Trực tiếp
Overhead flash	100–300KB	15–50KB	0KB
Overhead suy luận	15–40%	<5%	0%
Hỗ trợ RF/SVM	Hạn chế [†]	Không [‡]	Đầy đủ
Minh bạch	Thấp	Trung bình	Hoàn toàn
Cần ONNX/TFLite	Có	Có	Không
Phức tạp triển khai	Thấp	Cao	Trung bình
Hỗ trợ DNN sâu	Tốt	Tốt	Hạn chế

[†]TFLite Micro chỉ hỗ trợ NN/CNN; RF/SVM không thể chuyển đổi trực tiếp do onnx2tf không hỗ trợ ONNX-ML TreeEnsembleClassifier. Keras surrogate có thể sử dụng nhưng là xấp xỉ (80–90% tương đồng).

[‡]EON Compiler và TVM tập trung vào mạng nơ-ron; không hỗ trợ RF/SVM gốc.

Lập luận cho lựa chọn thiết kế

Quyết định sử dụng tạo mã trực tiếp được lập luận bởi bốn yếu tố:

(1) Mức sử dụng bộ nhớ tối thiểu Trên các vi điều khiển lớp wearable như Seed XIAO nRF52840 (1MB flash, 256KB RAM), mỗi KB đều quan trọng. TFLite Micro interpreter chiếm khoảng 150–250KB flash^[27;39]—tương đương 15–25% tổng flash khả dụng—chỉ riêng runtime, trước khi tính trọng số mô hình. Phương pháp trực tiếp loại bỏ hoàn toàn overhead này: một mô hình Neural Network chỉ chiếm 95KB (trọng số + logic suy luận + trích xuất đặc trưng), để lại >900KB cho firmware ứng dụng và giúp việc chuyển sang các board cùng lớp tài nguyên trở nên khả thi hơn.

(2) Minh bạch và khả năng xác minh Một yêu cầu quan trọng trong thiết kế hệ thống suy luận nhúng là *khả năng kiểm tra và xác minh mã đã tạo* (xem Phần 6.4). Khi phát hiện lỗi zero-padding và sai lệch công thức kurtosis, có thể truy vết trực tiếp đến mã C++ đã tạo, so sánh từng dòng với triển khai Python, và sửa chữa. Với runtime đóng gói (TFLite Micro) hoặc trình biên dịch đóng (EON Compiler), quá trình gỡ lỗi này sẽ không thể thực hiện—lỗi sẽ biểu hiện như “mô hình hoạt động kém trên thiết bị” mà không có cách xác định nguyên nhân gốc.

Khả năng xác minh này đặc biệt quan trọng cho ứng dụng y tế và an toàn, nơi các tiêu chuẩn quy định (FDA, IEC 62304) yêu cầu khả năng truy vết và kiểm toán đầy đủ cho mọi thành phần phần mềm liên quan đến quyết định lâm sàng.

(3) Hỗ trợ trực tiếp các thuật toán ML cổ điển Hệ sinh thái TFLite Micro và ONNX Runtime Micro được thiết kế chủ yếu cho mạng nơ-ron. Triển khai Random Forest hoặc SVM qua các nền tảng này yêu cầu chuyển đổi nhiều bước: scikit-learn → ONNX (`skl2onnx`) → TFLite (`onnx-tf`) → TFLite Micro. Mỗi bước chuyển đổi giới thiệu rủi ro sai lệch số và mất thông tin (ví dụ: ONNX không hỗ trợ chính xác tham số `class_weight` của scikit-learn). Phương pháp trực tiếp đọc các thuộc tính nội bộ của scikit-learn (`model.coefs_`, `model.support_vectors_`, `model.estimators_`) mà không qua định dạng trung gian, loại bỏ hoàn toàn chuỗi chuyển đổi dễ lỗi này.

(4) Các thuật toán cổ điển là lựa chọn phù hợp cho HAR biên Lựa chọn tạo mã trực tiếp liên kết chặt chẽ với lựa chọn thuật toán. Cho bài toán HAR 5–10 lớp với <1000 vectors đặc trưng và 63 đặc trưng bất biến hướng, các thuật toán cổ điển đạt hiệu suất cạnh tranh (97–99%) với chi phí tài nguyên thấp hơn đáng kể. Các mô hình này có cấu trúc suy luận đơn giản—nhân ma trận cho NN, duyệt cây cho RF, tính kernel cho SVM—có thể triển khai chính xác trong vài trăm dòng C++ mà không cần framework tính toán phức tạp. Điều này tạo thành một *sweet spot* giữa hiệu suất mô hình và tính khả thi triển khai cho các thiết bị đeo bị giới hạn tài nguyên.

Hạn chế và các phương pháp bổ sung

Tạo mã trực tiếp không phải giải pháp phổ quát. Nó không phù hợp cho:

- **Mạng sâu nhiều lớp:** LSTM, Transformer, hoặc CNN với hơn 4–5 lớp convolution yêu cầu các toán tử phức tạp (attention, bidirectional) mà triển khai thủ công dễ lỗi và khó bảo trì
- **Lượng tử hóa nâng cao:** INT8 inference với hiệu chỉnh phạm vi per-channel yêu cầu hạ tầng runtime mà TFLite Micro cung cấp sẵn
- **Mô hình thay đổi thường xuyên:** Nếu mô hình cần cập nhật OTA (over-the-air) mà không biên dịch lại firmware, runtime interpreter là lựa chọn tốt hơn

Đường dẫn TFLite Micro (Mục 3.5.3) tồn tại song song với tạo mã trực tiếp; người dùng chọn phương pháp phù hợp: tạo mã trực tiếp cho ML cổ điển và CNN nhỏ, TFLite Micro cho các mô hình cần lượng tử hóa INT8 đầy đủ.

3.5.3 Triển khai TFLite Micro

Đường dẫn triển khai TFLite Micro là phương pháp bổ sung phù hợp cho các kiến trúc mô hình neural network. Quá trình chuyển đổi tuân theo pipeline $\text{ONNX} \rightarrow \text{TensorFlow} \rightarrow \text{TFLite}$ với các hạn chế quan trọng về phạm vi hỗ trợ^[1].

Phạm vi hỗ trợ theo loại mô hình

Nghiên cứu thực nghiệm cho thấy khả năng chuyển đổi khác nhau đáng kể giữa các loại mô hình:

Neural Network và CNN: Chuyển đổi trực tiếp (hỗ trợ) Các mô hình neural network (MLP) và CNN từ PyTorch có thể chuyển đổi thành công qua pipeline:

1. PyTorch model $\rightarrow$ ONNX (`torch.onnx.export`)
2. ONNX $\rightarrow$ TensorFlow (`onnx-tf`)
3. TensorFlow $\rightarrow$ TFLite (`tf.lite.TFLiteConverter`)
4. TFLite $\rightarrow$ TFLite Micro (quantization INT8)

Random Forest và SVM: Hạn chế ONNX-ML (không hỗ trợ trực tiếp) Các mô hình Random Forest và SVM gặp rào cản chuyển đổi do hạn chế trong hệ sinh thái ONNX-ML:

- **Scikit-learn $\rightarrow$ ONNX:** `skl2onnx` tạo các toán tử ONNX-ML như `TreeEnsembleClassifier` và `SVMClassifier`
- **ONNX-ML $\rightarrow$ TensorFlow:** `onnx2tf` không hỗ trợ các toán tử ONNX-ML, chỉ hỗ trợ các toán tử ONNX chuẩn (operator domain "ai.onnx")
- **Kết quả:** Pipeline bị gián đoạn, không thể chuyển đổi RF/SVM sang TFLite

Giải pháp Keras Surrogate cho RF/SVM

Để khắc phục hạn chế ONNX-ML, *phương pháp Keras surrogate* có thể được áp dụng:

1. **Sinh dữ liệu huấn luyện surrogate:** Sử dụng mô hình RF/SVM gốc để tạo soft predictions trên tập dữ liệu đại diện

2. **Huấn luyện Keras MLP:** Mạng neural đa lớp học ánh xạ từ features $\rightarrow$ soft predictions của RF/SVM
3. **Chuyển đổi MLP $\rightarrow$ TFLite:** Keras MLP có thể chuyển đổi thành công qua pipeline tiêu chuẩn

Hạn chế Surrogate: Keras surrogate là *xấp xỉ*, không phải triển khai chính xác. Độ thỏa thuận kỳ vọng 80-90% giữa mô hình gốc và surrogate. Cho độ chính xác tối đa, *tạo mã trực tiếp C++ vẫn là phương pháp được khuyến nghị cho RF/SVM.*

Khuyến nghị sử dụng

- **NN/CNN:** Sử dụng TFLite Micro để tận dụng runtime tối ưu hóa và hỗ trợ quantization INT8
- **RF/SVM:** Ưu tiên tạo mã trực tiếp C++ (chính xác, nhanh, nhỏ gọn). TFLite surrogate chỉ khi yêu cầu tích hợp runtime chuẩn hóa

3.5.4 Cấu trúc mã đã tạo

Đối với mỗi mô hình đã huấn luyện, trình tạo sản xuất một gói triển khai gồm các tệp tham số, tệp cài đặt và tệp ví dụ tích hợp. Cụ thể:

1. **Tệp Header (.h):** Chứa các tham số mô hình (ví dụ: vectors hỗ trợ, cấu trúc cây, ma trận trọng số), khai báo hàm, các macro số lượng đặc trưng/lớp
2. **Tệp Triển khai (.c/.cpp/.py):** Triển khai trích xuất đặc trưng, logic dự đoán (đặc thù cho thuật toán) và các hàm tiện ích, với phần mở rộng phụ thuộc nền tảng đích
3. **Tệp Ví dụ Tích hợp:** Có thể là Arduino sketch (.ino), chương trình C/C++ ví dụ hoặc script MicroPython, minh họa khởi tạo cảm biến, quản lý bộ đệm của số trượt, vòng lặp suy luận thời gian thực và đầu ra dự đoán

Xác minh Công thức Thống kê: Mã C++ đã tạo cho trích xuất đặc trưng sử dụng các công thức thống kê được xác minh khớp chính xác với triển khai Python (pandas/NumPy). Cụ thể, các đặc trưng kurtosis và skewness sử dụng *độ lệch chuẩn tổng thể* (population standard deviation, chia n) cho chuẩn hóa z-scores, thay vì độ lệch chuẩn mẫu (chia $n - 1$). Sự khác biệt này, mặc dù nhỏ ($\sim 1,3\%$ cho $n = 150$), là hệ thống và tích lũy qua nhiều đặc trưng, có thể ảnh hưởng đến độ chính xác triển khai nếu không được xử lý.

3.5.5 Lượng tử hóa mô hình cho triển khai biên

Lượng tử hóa (model quantization) là kỹ thuật nén mô hình then chốt trong triển khai TinyML, chuyển đổi các tham số mô hình từ biểu diễn dấu phẩy động 32-bit (float32) sang số nguyên 8-bit (int8) hoặc 16-bit (int16). Trên vi điều khiển như Cortex-M4F với 256 KB RAM và 1 MB flash, lượng tử hóa có thể quyết định liệu một mô hình có thể triển khai được hay không: chuyển đổi từ float32 sang int8 giảm kích thước trọng số $4\times$, giảm băng thông bộ nhớ và tăng tốc suy luận $2\text{--}3\times$ nhờ các tập lệnh SIMD integer của ARM^[22].

Các loại lượng tử hóa

Lượng tử hóa sau huấn luyện (Post-Training Quantization, PTQ) PTQ áp dụng sau khi huấn luyện hoàn tất mà không cần sửa đổi quy trình huấn luyện. Mỗi tham số float32 x được ánh xạ sang giá trị lượng tử hóa x_q theo:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, 0, 255\right) \quad (3.5.1)$$

trong đó $s = \frac{x_{\max} - x_{\min}}{255}$ là hệ số tỷ lệ và $z \in [0, 255]$ là zero-point giữ nguyên giá trị số không. Có hai biến thể PTQ: *lượng tử hóa trọng số* (weight-only), trong đó chỉ trọng số mô hình được chuyển đổi còn phép tính suy luận vẫn dùng float32; và *lượng tử hóa tĩnh* (static PTQ), trong đó cả trọng số lẫn giá trị kích hoạt đều được lượng tử hóa, đòi hỏi một tập hiệu chỉnh nhỏ (100–500 mẫu đại diện) để đo phân phối kích hoạt trong thực tế. PTQ thường gây suy giảm độ chính xác 0,5–2% cho các mô hình HAR^[39].

Lượng tử hóa trong huấn luyện (Quantization-Aware Training, QAT) QAT chèn các toán tử *fake quantization* (lượng tử hóa giả) vào đồ thị tính toán trong quá trình huấn luyện, mô phỏng sai số lượng tử hóa để mô hình học cách bù đắp. QAT thường giảm suy giảm độ chính xác xuống dưới 0,3% nhưng đòi hỏi sửa đổi pipeline huấn luyện và thực tế chỉ áp dụng cho mạng nơ-ron—Random Forest và SVM có cấu trúc rời rạc (ngưỡng quyết định, vector hỗ trợ) không tương thích với kỹ thuật đạo hàm giả (straight-through estimator) mà QAT dựa vào.

Chiến lược lượng tử hóa theo phương pháp tạo mã

Tùy theo phương pháp tạo mã được chọn, lượng tử hóa được áp dụng theo hai đường dẫn khác nhau:

Lượng tử hóa trọng số trong tạo mã trực tiếp Trong đường dẫn tạo mã trực tiếp, lượng tử hóa được thực hiện ở cấp độ biểu diễn tham số trong mã C++ đã tạo. Chế độ tối ưu hóa kiểm soát số chữ số thập phân khi nhúng hằng số float32 vào mã nguồn—tương đương lượng tử hóa trọng số mức thấp:

- **Accuracy:** float32 đầy đủ (6 chữ số thập phân), không xấp xỉ
- **Balanced:** 3 chữ số thập phân, tiết kiệm $\approx 30\%$ kích thước mã nguồn
- **Speed/Power:** 2 chữ số thập phân, tiết kiệm $\approx 45\%$ kích thước mã nguồn

Phép suy luận vẫn thực hiện bằng số học float32 trên FPU của Cortex-M4F—đây là lượng tử hóa *lưu trữ*, không phải lượng tử hóa *tính toán* đầy đủ. Đối với mạng nơ-ron, chế độ *Power* có thể tùy chọn biểu diễn ma trận trọng số ở int16, thực hiện nhân tích lũy integer và chỉ dequantize một lần trước hàm kích hoạt:

$$y_j = f\left(s \cdot \sum_i w_{ij}^{\text{int16}} \cdot x_i^{\text{int16}} + b_j\right) \quad (3.5.2)$$

trong đó s là tích của hai scale tương ứng (trọng số và đầu vào), $f(\cdot)$ là hàm kích hoạt ReLU hoặc softmax.

Lượng tử hóa INT8 đầy đủ qua đường dẫn TFLite Micro Đường dẫn TFLite Micro (áp dụng cho NN và CNN) hỗ trợ static PTQ với INT8 đầy đủ—cả trọng số lẫn kích hoạt. Quy trình thực hiện:

1. Chuyển đổi mô hình Keras/PyTorch sang TFLite (**TFLiteConverter**)
2. Một tập hiệu chỉnh gồm 100–500 vectors đặc trưng đại diện được cung cấp để TFLiteConverter đo phân phối kích hoạt từng lớp
3. Converter tự động tính s và z cho mỗi tensor theo Phương trình 3.5.1, tối ưu cho phạm vi kích hoạt thực tế
4. Mô hình INT8 kết quả kết hợp với ARM CMSIS-NN^[5] để khai thác tập lệnh SIMD của Cortex-M4, thực thi 4 phép nhân int8 song song mỗi chu kỳ—tăng tốc $2\text{--}3\times$ so với mô hình float32 tương đương

Bảng 3.5.3 so sánh hai đường dẫn lượng tử hóa theo các tiêu chí triển khai:

3.5. Tạo mã cho triển khai biên

Bảng 3.5.3: So sánh chiến lược lượng tử hóa theo phương pháp tạo mã

Tiêu chí	Tạo mã trực tiếp (Weight PTQ)	TFLite Micro (Static INT8 PTQ)
Áp dụng cho	RF, SVM, NN, CNN	NN, CNN (không hỗ trợ RF/SVM)
Giảm kích thước	30–45% (so với float32 đầy đủ)	4× (so với float32)
Tăng tốc suy luận	Không đáng kể (vẫn dùng FPU float32)	2–3× (SIMD int8)
Suy giảm độ chính xác	<0,5%	0,5–2%
Yêu cầu tập hiệu chỉnh	Không	100–500 mẫu
Tính minh bạch	Hoàn toàn (mã có thể kiểm tra)	Hạn chế (runtime hộp đen)

Đặc thù lượng tử hóa cho đặc trưng HAR miền thời gian

Bài toán HAR với đặc trưng thống kê miền thời gian có các đặc tính thuận lợi cho lượng tử hóa so với thị giác máy tính hay xử lý ngôn ngữ tự nhiên:

- **Phạm vi đặc trưng có giới hạn vật lý:** Gia tốc IMU bị giới hạn ở $\pm 20 \text{ m/s}^2$, vận tốc góc ở $\pm 2000^\circ/\text{s}$. Do đó, các đặc trưng thống kê (trung bình, RMS, độ lệch chuẩn) có phạm vi xác định, giúp xác định scale lượng tử hóa chính xác mà không cần tập hiệu chỉnh lớn.
- **Bền vững với nhiễu lượng tử hóa:** Random Forest quyết định theo ngưỡng nhị phân tại mỗi nút cây; sai số lượng tử hóa nhỏ hiếm khi đủ để đổi chiều quyết định phân nhánh, đặc biệt khi đặc trưng nằm xa ngưỡng. SVM nhạy hơn ở vùng ranh giới quyết định nhưng vẫn ổn định với int16.
- **Tham số scaler phải đồng bộ độ chính xác:** StandardScaler (trung bình μ và độ lệch chuẩn σ) được áp dụng trước mô hình. Để duy trì parity huấn luyện-triển khai, tham số scaler cần được nhúng với cùng độ chính xác với tham số mô hình—lượng tử hóa mô hình nhưng giữ scaler ở float32 đầy đủ sẽ gây ra sự không nhất quán hệ thống ảnh hưởng đến toàn bộ vector đặc trưng đầu vào.
- **Kurtosis và Skewness nhạy hơn:** Các đặc trưng hình dạng phân phối có phạm vi giá trị rộng hơn và nhạy với lượng tử hóa mạnh hơn các đặc trưng năng lượng

(RMS, Energy). Do đó, chế độ *Balanced* khuyến nghị sử dụng 3 chữ số thập phân để bảo toàn các đặc trưng này.

3.5.6 Kiến trúc tạo mã đa kiến trúc

Hai loại pipeline mô hình tồn tại với yêu cầu khác biệt:

Mô hình dựa trên đặc trưng (RF, SVM, NN) Nhận vector đặc trưng thống kê (33–90 chiều), áp dụng StandardScaler trên features đã trích xuất, sinh mã trích xuất đặc trưng đầy đủ trong C++.

Mô hình CNN end-to-end Nhận trực tiếp cửa sổ cảm biến thô ($150 \times 6 = 900$ đầu vào), chỉ áp dụng lọc tín hiệu (không trích xuất đặc trưng), StandardScaler trên dữ liệu thô.

Sự khác biệt này đòi hỏi kiến trúc tạo mã nhận biết loại mô hình (architecture-aware): metadata, logic xác minh và cấu trúc mã sinh ra đều phụ thuộc vào loại pipeline. Generator tự động phân biệt để sinh đúng cấu trúc cho từng loại.

3.5.7 Các chiến lược tối ưu hóa

Bốn chế độ tối ưu hóa được cấu hình khi tạo mã: *Accuracy* (float32 đầy đủ, không đơn giản hóa); *Speed* (giảm phức tạp mô hình, tính toán cache-friendly, mở rộng vòng lặp); *Power* (số học điểm cố định int16, 2 chữ số thập phân, tận dụng ARM CMSIS-DSP); *Balanced* (3 chữ số thập phân, độ chính xác hỗn hợp cho các tính toán khác nhau). Chi tiết so sánh tại Bảng 4.6.1, Chương 4.

3.5.8 Điều chỉnh đặc thù nền tảng

Mã sinh ra được điều chỉnh theo họ thiết bị đích: Arduino-compatible (sinh .ino/.cpp/.h với Serial và Wire); ARM Cortex-M native (mã C độc lập, bộ nhớ tĩnh, tích hợp CMSIS); SDK/RTOS (include và entry-point cho ESP-IDF, Zephyr); MicroPython (module Python độc lập duy trì tương đồng công thức với C++). Nhờ phân lớp này, cùng một mô hình có thể xuất cho nhiều đích mà không cần huấn luyện lại.

Chi tiết triển khai phần mềm và phần cứng được trình bày trong Chương 4. Thiết lập thực nghiệm, tập dữ liệu và số liệu đánh giá được mô tả trong Chương 5.

Chương 4

Thiết kế và hiện thực

Chương này trình bày kiến trúc hệ thống, quyết định thiết kế và chi tiết hiện thực của framework triển khai nhận dạng hoạt động trên thiết bị biên. Các thuật toán và cơ sở lý thuyết đã được trình bày trong Chương 3; chương này tập trung vào **kỹ thuật phần mềm, mô-đun hóa, và quy trình triển khai** của hệ thống thực tế. Framework được xây dựng dựa trên nguyên tắc tách biệt vai trò (separation of concerns): mỗi module chịu trách nhiệm một giai đoạn rõ ràng trong pipeline từ dữ liệu thô đến mã triển khai, đảm bảo tính mở rộng và khả năng bảo trì.

4.1 Kiến trúc hệ thống

4.1.1 Nền tảng công nghệ và môi trường phát triển

Framework được triển khai dưới dạng ứng dụng web một trang (single-page web application) với các thành phần công nghệ sau:

Thư viện và framework chính

- **Python 3.10**: ngôn ngữ lập trình chính, được chọn vì hệ sinh thái học máy phong phú và khả năng tích hợp nhanh giữa các thư viện khác nhau.
- **Dash 2.14**: framework giao diện web dựa trên Flask và Plotly. Dash cung cấp mô hình lập trình khai báo (declarative) với callback pattern, cho phép xây dựng giao diện tương tác phức tạp mà không cần JavaScript thủ công.
- **scikit-learn 1.3**: triển khai các thuật toán học máy truyền thống (Random Forest, SVM, MLP). Thư viện cung cấp API nhất quán và hiệu năng đã được tối ưu hóa.

- **PyTorch 2.x**: framework học sâu cho huấn luyện mô hình MLP và CNN. PyTorch được chọn vì API linh hoạt, hỗ trợ tốt cho tùy chỉnh kiến trúc, và khả năng xuất mô hình sang ONNX/TFLite.
- **NumPy/Pandas**: xử lý dữ liệu số học và cấu trúc dạng bảng. Pandas DataFrame cung cấp giao diện thuận tiện cho thao tác chuỗi thời gian và trích xuất đặc trưng.
- **Plotly**: thư viện trực quan hóa tương tác, hỗ trợ drag-and-drop selection và real-time updates trong giao diện Dash.
- **Jinja2**: template engine để sinh mã C/C++ và Arduino từ các mẫu tham số hóa, đảm bảo tính nhất quán cú pháp và dễ bảo trì.

Môi trường phát triển

- **Nền tảng phát triển**: Windows 11 workstation (Intel Core i7, 16GB RAM). Tuy nhiên, framework được thiết kế platform-agnostic và chạy được trên Linux/macOS mà không cần điều chỉnh.
- **Nền tảng chuẩn triển khai**: Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64MHz, 256KB RAM, 1MB Flash, LSM6DS3 IMU). Vi điều khiển này được chọn làm **nền tảng chuẩn đo benchmark** do: (1) tích hợp đầy đủ IMU 6-trục, (2) hiệu năng đại diện cho phân khúc MCU trung bình, (3) hỗ trợ Arduino và PlatformIO, và (4) giá thành thấp (~\$10 USD).
- **Công cụ biên dịch**: Arduino IDE 2.3 và PlatformIO cho biên dịch chéo (cross-compilation) sang ARM Cortex-M. Cả hai đều sử dụng GCC ARM Embedded Toolchain với flag tối ưu hóa -O2 (cân bằng kích thước và tốc độ).

4.1.2 Kiến trúc ứng dụng

Framework áp dụng kiến trúc phân lớp rõ ràng với nguyên tắc *separation of concerns*. Hệ thống được tổ chức thành bốn lớp chính: (1) **lớp giao diện** chứa định nghĩa layout Dash cho mỗi tab; (2) **lớp callback** xử lý tương tác người dùng và điều phối luồng dữ liệu; (3) **lớp tiện ích** chứa các thuật toán xử lý tín hiệu, trích xuất đặc trưng và huấn luyện mô hình; (4) **lớp tạo mã** chứa hệ thống generator cho từng mô hình và nền tảng đích. Trạng thái liên phiên được lưu xuống hệ thống file, tách biệt với trạng thái tạm thời trong session.

Quy trình đăng ký callback Dash sử dụng mô hình *callback pattern* để đồng bộ giao diện và trạng thái. Mỗi module callback định nghĩa một hàm `register_callbacks(app)`, bên trong đó tập trung toàn bộ logic xử lý sự kiện cho tab tương ứng. Entry point ứng dụng gọi tuần tự các hàm đăng ký này sau khi khởi tạo layout.

Cách tổ chức này đảm bảo:

1. **Tính module:** mỗi module callback độc lập, dễ bảo trì và kiểm thử riêng biệt.
2. **Tránh circular imports:** callbacks không import lẫn nhau, chỉ import từ lớp tiện ích và lớp tạo mã.
3. **Testability:** mỗi callback có thể kiểm thử riêng bằng cách mock Dash context.

4.1.3 Quản lý trạng thái

Framework sử dụng kết hợp hai cơ chế lưu trạng thái:

Hệ thống file (`persistent_data/`) Dữ liệu trung gian giữa các giai đoạn được lưu dưới dạng file CSV, pickle (scikit-learn models) và joblib (model bundles):

- **datasets/**: dữ liệu CSV gốc theo activity class.
- **windows/**: sliding windows đã sinh (pickle format).
- **training/**: feature matrices (CSV) và metrics.
- **models/**: model serialization (joblib) chứa model object, scaler, label encoder, danh sách features và các chỉ số hiệu suất.
- **generated/**: mã nguồn triển khai theo cấu trúc `{model_type}_models/{model_name}/`.

In-memory state (`dcc.Store`) Dash component `dcc.Store` lưu trạng thái session tạm thời (VD: metadata cột cảm biến, tần số lấy mẫu, danh sách segments đã chọn). State này được truyền giữa các callbacks trong cùng một session nhưng không persist sau khi reload trang.

Luồng dữ liệu Dữ liệu chảy tuần tự qua sáu tab: từ CSV thô đến dataset đăng ký, qua tiền xử lý và sliding windows, đến feature matrix, model joblib, mã C++ sinh ra, rồi kiểm tra trực tiếp trên thiết bị. Thiết kế file-based này cho phép người dùng quay lại bất kỳ bước nào để điều chỉnh mà không mất kết quả trung gian.

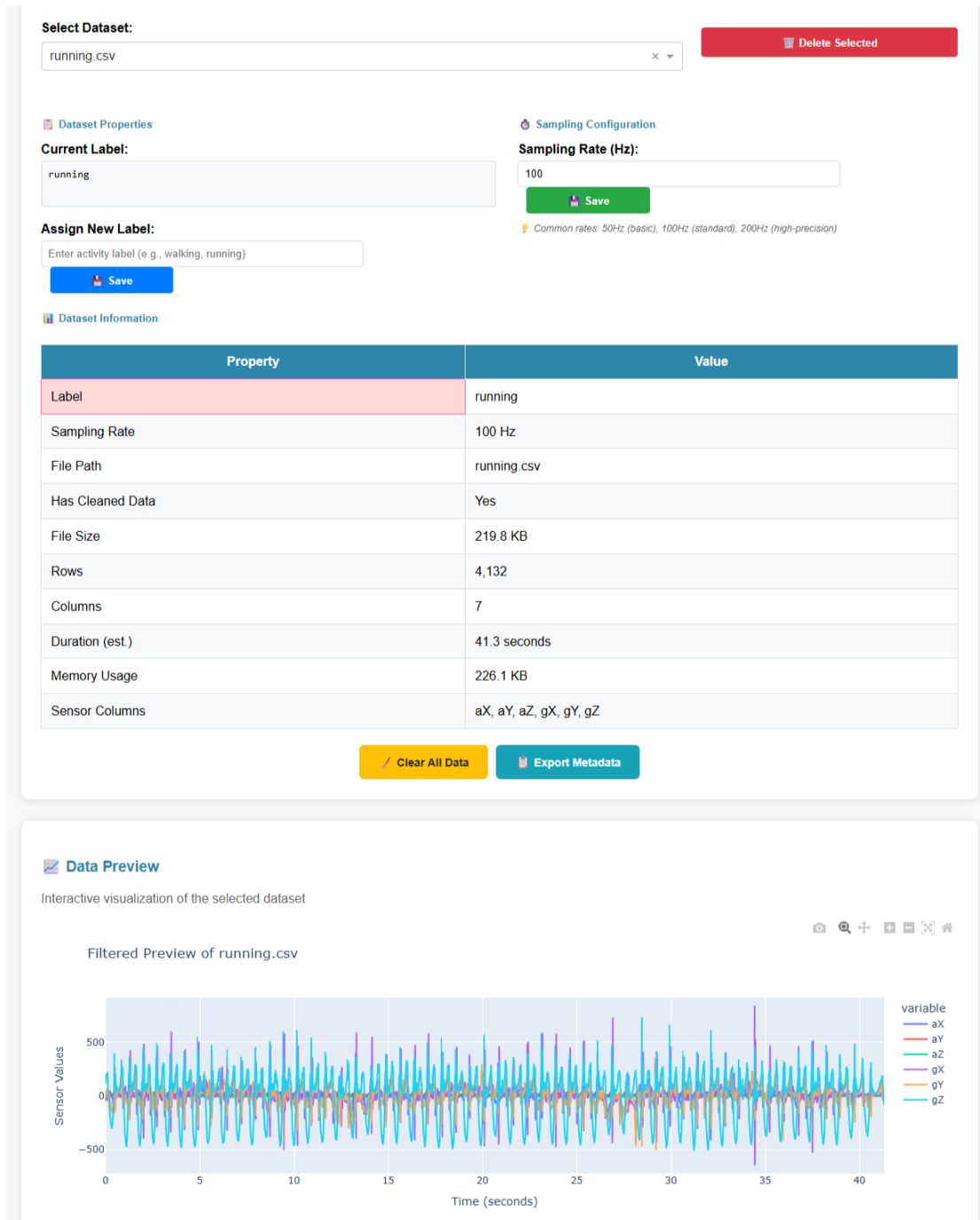
4.2 Module quản lý dữ liệu

Module này (Tab 1 trong giao diện) chịu trách nhiệm tải dữ liệu cảm biến và kiểm tra trực quan chất lượng tín hiệu.

4.2.1 Giao diện tải dữ liệu

Người dùng tải lên nhiều file CSV, mỗi file chứa dữ liệu cho một activity class. Framework tự động phát hiện cấu trúc cột bằng cách khớp với danh sách quy ước đặt tên phổ biến (như **aX/AccX** cho gia tốc kế, **gX/GyroX** cho con quay hồi chuyển); nếu không khớp, fallback sang khớp substring trong tên cột. Tần số lấy mẫu được tự động tính từ cột timestamp (nếu có) hoặc người dùng nhập thủ công; giá trị này được nhúng vào mã triển khai dưới dạng hằng số.

4.2. Module quản lý dữ liệu



Hình 4.2.1: Giao diện tải lên và trực quan hóa dữ liệu cảm biến. Framework tự động phát hiện cấu trúc cột IMU và hiển thị biểu đồ tín hiệu đồng bộ trên tất cả các trục gia tốc kế và con quay hồi chuyển, cho phép kiểm tra trực quan chất lượng dữ liệu trước khi tiến xử lý.

4.3 Module tiền xử lý tương tác

Module này (Tab 2) cung cấp giao diện kéo thả (drag-and-drop) để chọn các đoạn hoạt động đại diện trên biểu đồ tín hiệu, sau đó tự động sinh sliding windows từ các đoạn đã chọn.

4.3.1 Giao diện lựa chọn đoạn tương tác

Dash Plotly hỗ trợ *editable shapes*: người dùng vẽ hình chữ nhật trực tiếp trên biểu đồ để đánh dấu vùng quan tâm. Khi người dùng vẽ hoặc chỉnh sửa một hình chữ nhật, sự kiện layout được kích hoạt và framework đọc tọa độ x-axis để xác định phạm vi mẫu của đoạn vừa chọn.

Framework kiểm tra ràng buộc cơ bản:

- **Chiều dài tối thiểu:** mỗi segment phải có ít nhất `window_size` mẫu để sinh được ít nhất 1 cửa sổ.

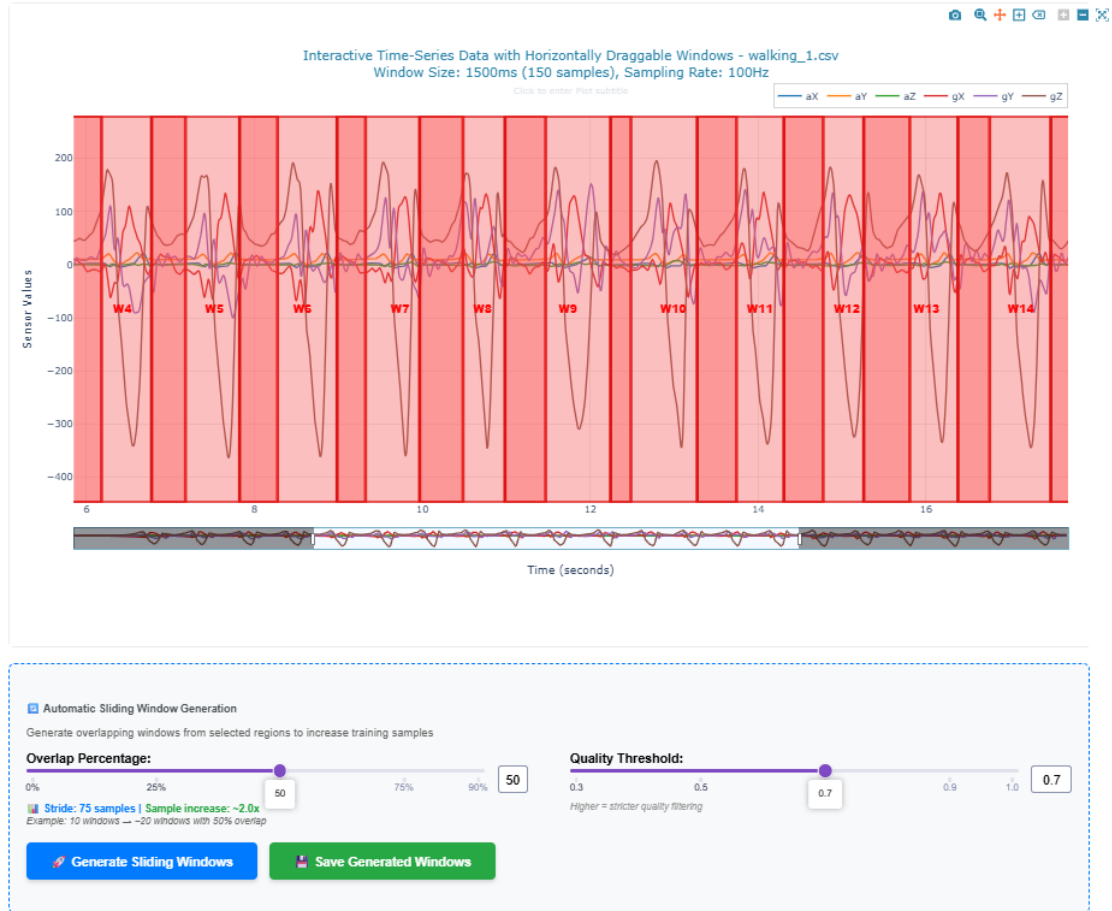
Các segment có thể chồng chéo nhau — điều này cho phép người dùng thu thập mẫu từ nhiều vùng khác nhau trong cùng một đoạn tín hiệu.

Hình 4.3.1 minh họa giao diện này.

4.3. Module tiền xử lý tương tác

Interactive Window Selection

Drag windows to optimal positions and extract segments for model training



Hình 4.3.1: Giao diện cửa sổ kéo thả tương tác. Người dùng kéo thả trực tiếp trên biểu đồ tín hiệu để xác định các đoạn hoạt động đại diện. Cửa sổ trượt tự động sinh các mẫu huấn luyện từ các đoạn đã chọn, kết hợp giữa kiểm soát thủ công và tự động hóa.

4.3.2 Sinh cửa sổ trượt

Từ mỗi segment, framework áp dụng thuật toán sliding window với tham số mặc định: `window_size` = 150 mẫu (1,5s ở 100Hz), `stride` = 75 mẫu (overlap 50%). Overlap 50% tăng số lượng mẫu huấn luyện đồng thời giữ sự đa dạng; stride càng nhỏ thì số windows tạo ra càng nhiều nhưng tăng nguy cơ overfitting do tương quan giữa các windows liền kề. Công thức và lý thuyết sliding window đã được trình bày trong Chương 3.

4.3.3 Trực quan hóa lọc tín hiệu

Framework hỗ trợ ba phương pháp lọc tín hiệu (chi tiết thuật toán đã trình bày trong Chương 3):

1. **Butterworth lowpass filter** (filtfilt for zero-phase)
2. **Savitzky-Golay filter** (polynomial smoothing)
3. **Kalman filter** (causal, exact training-deployment parity)

Giao diện hiển thị overlay tín hiệu raw (màu xanh) và filtered (màu đỏ) để người dùng đánh giá hiệu quả lọc. Người dùng có thể điều chỉnh tham số (cutoff frequency, filter order, window length) và quan sát thay đổi real-time.

4.4 Module trích xuất đặc trưng

Module này (Tab 3) chịu trách nhiệm chuyển đổi cửa sổ tín hiệu thô sang feature vectors và chuẩn bị train/val/test splits.

4.4.1 Chế độ trích xuất đặc trưng

Framework hỗ trợ 6 chế độ feature extraction, mỗi chế độ tạo ra một tập đặc trưng khác nhau:

Bảng 4.4.1: Các chế độ trích xuất đặc trưng và số lượng features

Chế độ	Mô tả	Số features
<code>orientation_invariant_time_only</code>	Magnitude stats (acc+gyro) + jerk	33
<code>orientation_invariant</code>	Time + frequency domain magnitudes	53
<code>time_domain</code>	Per-axis time stats + magnitudes	90
<code>all</code>	Time + frequency, per-axis + magnitudes	156
<code>frequency_domain</code>	FFT, PSD, spectral features	66
<code>raw</code>	Cửa sổ tín hiệu raw (CNN input)	6 channels

Cơ sở chọn đặc trưng bất biến hướng Chế độ `orientation_invariant` chỉ sử dụng các đặc trưng tính trên magnitude vectors ($\sqrt{x^2 + y^2 + z^2}$) thay vì per-axis features. Điều này đảm bảo mô hình không bị ảnh hưởng bởi định hướng lắp đặt cảm biến: đeo thiết bị ngang hoặc dọc, xoay 90° vẫn cho cùng một magnitude. Tuy nhiên, trade-off là mất thông tin về hướng chuyển động (VD: leo cầu thang khó phân biệt với

xuống cầu thang chỉ dựa trên magnitude). Thực nghiệm cho thấy chế độ `time_domain` (90 features) cân bằng tốt giữa độ chính xác và độ bền hướng.

4.4.2 Tích hợp data augmentation

Framework triển khai 5 phương pháp data augmentation cho dữ liệu IMU:

1. **Jitter**: thêm Gaussian noise nhỏ vào tín hiệu ($\sigma = 0.05 \times \text{signal_std}$).
2. **Rotation**: xoay ngẫu nhiên hệ trục tọa độ 3D để mô phỏng thay đổi định hướng.
3. **Scaling**: nhân tín hiệu với hệ số ngẫu nhiên (0.9–1.1) để mô phỏng biên độ chuyển động khác nhau.
4. **Time warping**: kéo giãn hoặc nén trục thời gian (0.8–1.2×) để mô phỏng tốc độ thực hiện hoạt động khác nhau.
5. **Permutation**: hoán vị các sub-windows để phá vỡ sự phụ thuộc thứ tự thời gian (chỉ áp dụng cho hoạt động tuần hoàn như đi bộ).

Class-aware augmentation Framework tự động phân loại hoạt động thành hai loại:

- **Static/transition**: đứng yên, ngồi, nằm → KHÔNG áp dụng time warping và permutation (gây mất tính tự nhiên).
- **Dynamic/periodic**: đi bộ, chạy, leo cầu thang → áp dụng tất cả augmentation.

Phân loại dựa trên từ khóa trong tên activity: framework kiểm tra xem tên hoạt động có chứa các từ khóa tĩnh (“still”, “stand”, “sit”, “lie”, “stationary”) hay không, và áp dụng tập augmentation phù hợp.

Giao diện augmentation cho phép người dùng:

- Kích hoạt/tắt từng phương pháp augmentation riêng biệt.
- Điều chỉnh augmentation factor (số lượng samples tăng thêm: 1×, 2×, 3×).
- Xem phân phối class trước và sau augmentation để đảm bảo cân bằng.

Hình 4.4.1 minh họa giao diện này.

4.4. Module trích xuất đặc trưng

Step 2: Configure Feature Engineering

Settings will be applied uniformly across all selected datasets

Feature Extraction Method:

Orientation-Invariant Time-Domain ONLY - 41 features

Normalization Method (applied during training):

Standard Scaler (Z-score)

Note: Orientation-robust DFT (63 features) is fully deployable — code generators emit a lightweight sin/cos DFT. Only per-axis frequency features (66 / 156) cannot be deployed.

Normalization is **not** applied here — it is saved to metadata and applied during **model training** so the scaler is bundled with the model for deployment.

41 features

Orientation-robust magnitudes: 15 stats × 2 mag + 3 acc-jerk + 3 gyro-jerk + 5 scalar extras (SMA, tilt, autocorr, peak count). RECOMMENDED for deployment — fully deployable to all targets (C / C++ / MicroPython).

Window Configuration

Target Window Size (ms):

1500

Windows smaller than this will be zero-padded (1500ms = 150 samples @ 100Hz)

Sampling Rate (Hz):

100

Used to calculate time duration from samples

Step 3: Data Augmentation (Optional)

Generate synthetic training windows to improve model robustness. Augmentation is applied to raw sensor signals before feature extraction, so new features are computed naturally from augmented data.

☐ Enable Data Augmentation

Step 4: Configure Dataset Split

Define how to split the combined dataset into train/validation/test sets

Split Mode:

☒ Auto (ratio-based random split) ☐ Manual (assign each window by hand)

Training Set Ratio:

0.7

Validation Set Ratio:

0.15

Test Set Ratio:

15%

Random Seed (for reproducibility):

42

Same seed = reproducible splits across runs

Hình 4.4.1: Giao diện trích xuất đặc trưng. Người dùng chọn loại đặc trưng (bất biến hướng, miền thời gian, miền tần số), cấu hình tăng cường dữ liệu nhận biết lớp, và phân chia dữ liệu huấn luyện/xác thực/kiểm tra với lấy mẫu phân tầng.

4.4.3 Phân chia train/validation/test

Framework cung cấp hai chế độ phân chia dữ liệu để phù hợp với nhiều kịch bản thực tế:

Phân chia tự động theo tỉ lệ Người dùng chọn tỉ lệ train/validation/test bằng thanh trượt trực tiếp trên giao diện. Framework sử dụng **stratified sampling** để đảm bảo tỉ lệ class đồng đều giữa các tập. Kỹ thuật này quan trọng khi dữ liệu mất cân bằng: ví dụ nếu 70% mẫu thuộc class “đứng yên”, thì cả train và test đều có 70% mẫu đó. Tỉ lệ mặc định là train 60% / validation 20% / test 20%, nhưng người dùng có thể điều chỉnh tùy nhu cầu.

Phân chia thủ công theo dataset Đối với dữ liệu thu thập từ nhiều đối tượng hoặc nhiều phiên, người dùng có thể chỉ định trực tiếp file CSV nào thuộc tập train và file nào thuộc tập test. Chức năng này cho phép đánh giá *cross-subject generalization*: huấn luyện trên dữ liệu của một nhóm người dùng và kiểm thử trên nhóm hoàn toàn khác, tránh data leakage giữa các đối tượng.

Validation set được dùng cho hyperparameter tuning (GridSearchCV sử dụng cross-validation trên train+val), test set giữ nguyên không động đến cho đến khi đánh giá cuối cùng.

4.5 Module huấn luyện mô hình

Module này (Tab 4) triển khai training pipeline với hyperparameter optimization và real-time monitoring.

4.5.1 Lựa chọn thuật toán

Framework hỗ trợ 6 thuật toán phân loại:

Bảng 4.5.1: Thuật toán học máy được hỗ trợ

Thuật toán	Thư viện	Input	Triển khai
Random Forest	scikit-learn	Features	C++ native
SVM (RBF kernel)	scikit-learn	Features	C++ native
Neural Network (MLP)	scikit-learn	Features	C++ native
PyTorch MLP	PyTorch	Features	C++/TFLite
PyTorch 1D-CNN	PyTorch	Raw signals	C++/TFLite
PyTorch 2D-CNN	PyTorch	Raw signals (2D)	C++/TFLite

Kiến trúc PyTorch models

- **PyTorch MLP**: 2 hidden layers với ReLU activation. Kích thước layer điều chỉnh theo số features: `[n_features → 128 → 64 → n_classes]`.
- **PyTorch 1D-CNN**: convolutional layers trên time dimension, kernel size = 5, 2 conv layers + pooling + fully connected. Input shape: `[batch, 6 channels, 150 timesteps]`.

- **PyTorch 2D-CNN (HARCNN2D)**: sử dụng Conv2D với kernel $3 \times n_channels$ để học correlation giữa các sensor axes. Input shape: [batch, 1, 150 timesteps, 6 channels] (reshape từ 1D signal).

Kiến trúc 2D-CNN đặc biệt hữu ích khi cần học mối quan hệ không tuyến tính giữa các trục cảm biến (VD: correlation giữa aX và gY trong động tác xoay).

4.5.2 Tối ưu hóa siêu tham số

Framework tích hợp GridSearchCV từ scikit-learn với param grid mặc định cho từng thuật toán. Random Forest tìm kiếm trên các giá trị `n_estimators` (50, 100, 200), `max_depth` (10, 20, None) và `min_samples_split`; SVM tìm kiếm trên tham số `C` và `gamma`; Neural Network tìm kiếm trên kích thước hidden layers, hệ số L2 regularization và learning rate.

GridSearchCV sử dụng 5-fold cross-validation trên train+validation set, chọn tổ hợp params có accuracy cao nhất, sau đó retrain trên toàn bộ train+val với params tối ưu.

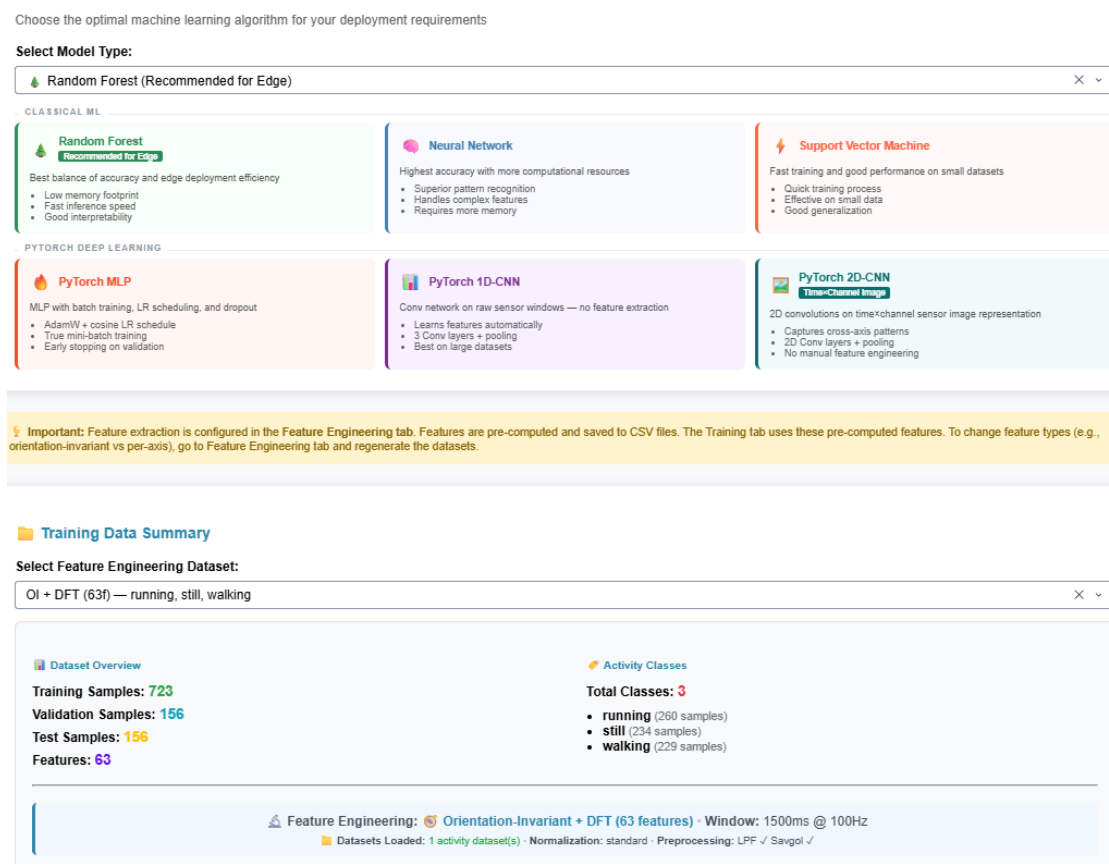
4.5.3 Theo dõi tiến trình real-time

Callback training sử dụng `dcc.Interval` component để cập nhật UI mỗi 2 giây trong quá trình huấn luyện. Thông tin hiển thị:

- **Training progress**: phần trăm hoàn thành, thời gian đã trôi qua.
- **Current hyperparameters**: params đang thử nghiệm.
- **Best score so far**: accuracy cao nhất trong các trials đã hoàn thành.
- **Confusion matrix**: ma trận nhầm lẫn trên validation set, cập nhật sau mỗi trial.

Hình 4.5.1 minh họa giao diện này.

4.5. Module huấn luyện mô hình



Hình 4.5.1: Giao diện cấu hình huấn luyện mô hình. Người dùng chọn thuật toán, cấu hình tối ưu hóa siêu tham số với GridSearchCV, chiến lược cân bằng lớp, và theo dõi tiến trình huấn luyện thời gian thực.

4.5.4 Lưu trữ mô hình

Sau khi huấn luyện hoàn tất, framework lưu trữ toàn bộ pipeline artifacts vào một file `.joblib`. Gói mô hình bao gồm: đối tượng model đã huấn luyện, bộ chuẩn hóa `StandardScaler`, ánh xạ nhãn (label encoder), danh sách tên features (để đảm bảo thứ tự đúng khi suy diễn), kiểu mô hình, siêu tham số tốt nhất, và các chỉ số hiệu suất (accuracy, F1, confusion matrix, classification report).

Cấu trúc này là chuẩn giao tiếp giữa module huấn luyện và module tạo mã. Module code generation đọc gói mô hình để trích xuất parameters (VD: cây quyết định của Random Forest, trọng số của Neural Network) và sinh mã triển khai tương ứng.

4.6 Module tạo mã triển khai

Module này (Tab 5) chịu trách nhiệm chuyển đổi trained model thành mã nguồn embedded C/C++ hoặc các định dạng khác (MicroPython, TFLite). Đây là thành phần phức tạp nhất của framework, vì phải đảm bảo **training-deployment parity**: mã được sinh ra phải tính toán đúng y hệt như Python pipeline.

4.6.1 Kiến trúc Code Generator: Factory Pattern

Framework sử dụng *Factory Pattern* để định tuyến từ (`model_type`, `target_platform`, `deployment_backend`) sang class generator phù hợp. Khi nhận model bundle, factory kiểm tra trường `model_type`: nếu deployment backend là TFLite Micro thì tạo `TFLiteCodeGenerator`; nếu không, sẽ tạo generator tương ứng với thuật toán (Random Forest, Neural Network, CNN, SVM). Tham số platform được truyền vào để generator điều chỉnh cú pháp và API cho từng nền tảng đích.

4.6.2 Hệ thống phân cấp Generator

Kiến trúc block pattern Module tạo mã được tổ chức theo mô hình block: một coordinator trung tâm nhận vào gói mô hình và điều phối các block con. Block trích xuất đặc trưng phụ trách sinh mã tính toán đặc trưng từ dữ liệu cảm biến thô; block phân loại và các lớp con của nó phụ trách sinh mã suy luận đặc thù cho từng thuật toán.

Coordinator điều phối hai nhóm block chính:

1. **Block trích xuất đặc trưng**: sinh mã tính features từ dữ liệu cảm biến thô (nếu mô hình yêu cầu). Bao gồm:
 - Tính magnitude: $\text{acc_mag} = \sqrt{aX^2 + aY^2 + aZ^2}$
 - Đặc trưng thống kê: mean, std, min, max, median, quartiles, skewness, kurtosis, RMS, energy
 - Đặc trưng jerk: đạo hàm số của `acc_mag`
 - Bộ lọc tiền xử lý: IIR filtfilt, Savitzky-Golay, Kalman (nếu được cấu hình)
2. **Block phân loại**: sinh mã suy luận cho thuật toán cụ thể. Mỗi loại mô hình có một block riêng:
 - **Random Forest**: chuyển cây quyết định thành chuỗi if-else lồng nhau.

- **Neural Network:** nhúng trọng số/bias và sinh mã nhân ma trận.
- **CNN:** sinh các kernel Conv1D hoặc Conv2D, pooling, fully connected layers.
- **SVM:** nhúng support vectors và sinh mã tính kernel.

4.6.3 Kiến trúc ba trục: Model $\times$ Platform $\times$ Backend

Framework triển khai ma trận deployment với ba chiều độc lập:

Trục 1: Model Type

- Random Forest, SVM, Neural Network (MLP)
- PyTorch MLP, 1D-CNN, 2D-CNN

Trục 2: Target Platform

- **Arduino (Cortex-M):** C++ code tối ưu cho ARM MCU, không sử dụng thư viện ngoài, tất cả tính toán inline.
- **MicroPython (ESP32):** Python code embed constants, sử dụng ulab (NumPy subset) nếu có.
- **Zephyr RTOS:** C code với Zephyr API cho threading và sensors.

Trục 3: Deployment Backend

- **Direct (native):** mã C/C++ trực tiếp, không phụ thuộc thư viện ngoài.
- **TFLite Micro:** chuyển đổi PyTorch model sang TFLite, sinh wrapper C++ gọi interpreter. Chỉ áp dụng cho NN/CNN (RF/SVM không convertible qua ONNX-ML operators).
- **ONNX Runtime:** xuất model sang ONNX, deploy với onnxruntime-embedded.

Ưu điểm của thiết kế này:

- **Tính mô-đun:** thêm model type mới chỉ cần implement một block.
- **Tính mở rộng:** thêm platform mới chỉ cần điều chỉnh template rendering.
- **Tính linh hoạt:** người dùng tự do lựa chọn backend dựa trên yêu cầu (performance, memory, dependencies).

4.6.4 Sắp xếp lại thứ tự đặc trưng khi sinh mã

Một vấn đề quan trọng trong deployment parity là **thứ tự features**:

- **Python training**: features được trích xuất từ Pandas DataFrame, thứ tự là alphabetical theo tên cột (VD: `acc_jerk_mag_mean`, `acc_mag_energy`, `acc_mag_iqr`, ...).
- **C++ deployment**: features được tính trong hàm `extract_features()`, thứ tự là computation order (VD: `acc_mag` stats trước, sau đó `gyro_mag` stats, cuối cùng là `acc_jerk`).

Nếu model weights được serialize theo thứ tự Python training, nhưng C++ code đưa features vào theo thứ tự khác, kết quả suy luận sẽ sai hoàn toàn.

Giải pháp: sắp xếp lại trọng số khi sinh mã Framework xử lý vấn đề này bằng cách tính toán thứ tự đặc trưng trong C++ (magnitude stats cho accelerometer, gyroscope, rồi jerk features), so sánh với thứ tự alphabetical trong Python, và tạo bảng ánh xạ chỉ số tương ứng. Dựa trên bảng ánh xạ này, các tham số scaler (mean, scale) và trọng số lớp đầu tiên của mô hình được sắp xếp lại trước khi nhúng vào mã C++.

Giải pháp này đảm bảo mô hình nhận features theo đúng thứ tự training mà không cần điều chỉnh C++ code.

4.6.5 Bốn chế độ tối ưu hóa

Framework cung cấp 4 chế độ tối ưu để cân bằng giữa accuracy, tốc độ và tiêu thụ điện:

Bảng 4.6.1: Chế độ tối ưu hóa mã triển khai

Chế độ	Độ chính xác số	Trade-off	Use case
Accuracy	6 chữ số thập phân	Bộ nhớ tối đa	Nghiên cứu, thử nghiệm
Balanced	3 chữ số thập phân	Cân bằng	Sản xuất chung
Speed	2–3 chữ số thập phân	Giảm độ trễ	Thời gian thực
Power	3–4 chữ số + DSP	Giảm chu kỳ CPU	Thiết bị dùng pin

Chế độ *Accuracy* sử dụng số thực 32-bit đầy đủ; *Balanced* làm tròn hệ số và dùng bảng tra cho hàm activation, giảm 20–30% kích thước mã; *Speed* áp dụng làm tròn mạnh và loại bỏ các đặc trưng đóng góp thấp; *Power* tận dụng ARM CMSIS-DSP khi

nền tảng hỗ trợ. Người dùng chọn chế độ trong giao diện code generation, framework tự động áp dụng các chuyển đổi tương ứng.

4.6.6 Gói đầu ra triển khai

Mã được sinh ra bao gồm 3 files chính:

1. **Header file (.h)**: khai báo constants (số features, số classes, window size, tần số lấy mẫu), tham số scaler, và prototype các hàm inference.
2. **Implementation file (.cpp)**: định nghĩa toàn bộ logic inference, bao gồm tính toán đặc trưng từ tín hiệu thô, chuẩn hóa bằng scaler từ quá trình huấn luyện, thực thi thuật toán phân loại, và tra cứu nhãn lớp từ kết quả dự đoán.
3. **Arduino sketch (.ino)**: ví dụ hoàn chỉnh cho Seeed XIAO nRF52840, bao gồm khởi tạo IMU và giao tiếp serial, vòng lặp thu thập đủ 150 mẫu vào bộ đệm rồi trích xuất đặc trưng, phân loại và xuất kết quả.

Hình 4.6.1 minh họa giao diện code generation.

4.6. Module tạo mã triển khai

The screenshot displays the TensorFlow Lite Model Maker web interface. It is divided into two main steps:

- Step 1: Select Trained Model**: Shows a selected model named 'pytorch_cnn_har_model_20260520_001738' with 100% accuracy on training, test, and validation data. The model type is 'PYTORCH_CNN', and it has 3 classes.
- Step 2: Configure Target Platform**: This section is divided into several panels:
 - Output Language / Framework**: Set to 'Arduino C++ (.ino) — Arduino framework, widest board support'.
 - Target Board**: Set to 'XIAO nRF52840 Sense (BLE + IMU)'.
 - Model Parameters (from training)**: Displays training details like Sampling Rate (100 Hz), Window Size (1500 ms), and Feature Count (150 samples x 6 channels = 900 inputs).
 - Signal Preprocessing (training—device parity)**: Lists preprocessing steps like Low-pass filter, Savitzky-Golay, and Outlier removal.
 - Deployment Approach**: Set to 'Direct C/C++ Code Generation — Standalone, no runtime dependency'.
 - Weight Quantization**: Set to 'INT8 — 75% smaller, ~1-2% accuracy loss'.
 - Optimization Level**: Set to 'Balanced'.
 - Window Overlap (%)**: Set to 50.
 - Confidence Threshold**: Set to 0.6.
 - Prediction Smoothing**: Set to 3.
 - On-device Signal Preprocessing**: A section for selecting preprocessing methods like LFF, SavGol, FFT, etc.

Hình 4.6.1: Giao diện tạo mã triển khai. Hiển thị các tệp header, implementation và Arduino sketch được sinh ra. Người dùng chọn chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình nền tảng đích và tải xuống gói triển khai hoàn chỉnh.

4.6.7 Kiến trúc đa mô hình: CNN code generation

CNN deployment khác với traditional ML vì không cần feature extraction—model nhận raw sensor data trực tiếp. Code generator cho CNN phải sinh các thành phần sau:

Conv1D và Conv2D kernel Đối với 1D-CNN, generator sinh mã thực hiện phép tích chập một chiều trên trục thời gian: mỗi filter duyệt qua tín hiệu đầu vào ($6 \text{ channels} \times 150 \text{ timesteps}$) với kernel size 5, tạo ra feature map đầu ra. Đối với 2D-CNN (HARCNN2D), generator sinh phép tích chập hai chiều với kernel shape ($3 \times n_{\text{channels}}$) để học tương quan chéo giữa các kênh cảm biến. Tất cả trọng số kernel và bias được nhúng trực tiếp dưới dạng mảng hằng số trong mã sinh ra.

MaxPooling và fully connected layers Generator cũng sinh mã cho max pooling (giảm chiều thời gian) và các lớp fully connected (phân loại dựa trên feature map đã trích xuất). Các phép toán này được triển khai bằng vòng lặp đơn giản, không phụ thuộc thư viện bên ngoài.

Bỏ qua scaler cho CNN Một phát hiện quan trọng trong quá trình phát triển: StandardScaler được huấn luyện trên *features* (mean, std, v.v.) không nên áp dụng lên *raw sensor values* (đầu vào của CNN). Nếu áp dụng, phép chuẩn hóa sẽ clamp các giá trị gyro vào range $[-10, 10]$ rad/s, phá hủy thông tin tín hiệu. Generator phân biệt kiến trúc mô hình: CNN nhận dữ liệu thô và bỏ qua scaler, trong khi các mô hình ML cổ điển vẫn áp dụng chuẩn hóa trên features đã trích xuất. Giải pháp này tiết kiệm 7KB Flash và loại bỏ hoàn toàn nguy cơ suy luận sai do biến dạng scaler.

4.7 Module kiểm tra thiết bị

Module này (Tab 6) cung cấp giao diện kết nối với thiết bị đã flash mã triển khai để xác minh hoạt động real-time.

4.7.1 Giao tiếp serial

Framework sử dụng thư viện `pyserial` để mở kết nối serial với thiết bị đã flash. Thiết bị gửi kết quả phân loại qua cổng serial theo định dạng chuẩn gồm tên hoạt động dự đoán, điểm tin cậy, và phân phối xác suất toàn bộ classes. Callback nhận từng dòng dữ liệu, phân tích cú pháp và cập nhật giao diện theo thời gian thực.

4.7.2 Hiện thị phân loại real-time

Giao diện hiển thị:

- **Hoạt động dự đoán:** tên hoạt động dự đoán với màu sắc tương ứng.
- **Thanh tin cậy:** thanh ngang hiển thị xác suất của class được dự đoán (0–100%).
- **Phân phối xác suất:** biểu đồ cột hiển thị xác suất của tất cả classes, giúp phát hiện các trường hợp mơ hồ (nhiều classes có xác suất gần bằng nhau).
- **Dòng thời gian:** biểu đồ hiển thị lịch sử dự đoán 30 giây gần nhất, cho thấy quá trình chuyển đổi giữa các hoạt động.

4.7.3 Ngưỡng tin cậy

Framework sinh mã có confidence thresholding: nếu xác suất cao nhất trong phân phối dự đoán thấp hơn ngưỡng, mô hình trả về UNKNOWN thay vì gán nhãn với tin cậy thấp. Cơ chế này quan trọng cho triển khai thực tế khi gặp hoạt động ngoài tập huấn luyện (out-of-distribution). Người dùng có thể điều chỉnh ngưỡng trong giao diện (mặc định: 0,7).

4.7.4 Quy trình xác minh triển khai

Workflow kiểm thử trên thiết bị:

1. **Flash code:** upload mã sinh ra lên vi điều khiển qua Arduino IDE hoặc PlatformIO.
2. **Connect serial:** mở Tab 6, chọn COM port, kết nối.
3. **Perform activities:** thực hiện các hoạt động từ tập training (VD: đứng yên, đi bộ, chạy).
4. **Verify predictions:** so sánh dự đoán của thiết bị với ground truth.
5. **Test edge cases:** thực hiện hoạt động ngoài tập training (VD: nhảy, đạp xe) để kiểm tra UNKNOWN detection.
6. **Measure latency:** framework hiển thị thời gian xử lý mỗi window (từ thu thập sensor đến xuất kết quả).

Nếu phát hiện mismatch giữa dự đoán Python và thiết bị, framework cung cấp công cụ debugging:

- **So sánh đặc trưng:** xuất features từ thiết bị qua serial, so sánh với Python.
- **Kiểm tra parity:** kiểm tra từng bước trong pipeline (tính magnitude, đặc trưng thống kê, scaler, dự đoán).

4.8 Luồng dữ liệu và quản lý trạng thái tổng thể

4.8.1 Luồng dữ liệu end-to-end

Dữ liệu chảy qua framework theo pipeline rõ ràng:

1. CSV upload (Tab 1):

- Input: user uploads CSV files
- Output: `persistent_data/datasets/{activity}.csv`
- State: metadata (columns, sampling rate) → `dcc.Store`

2. Preprocessing (Tab 2):

- Input: read CSVs from `datasets/`
- Process: user selects segments via drag-and-drop
- Output: sliding windows → `persistent_data/windows/{dataset_name}.pkl`
- State: segments list → `dcc.Store`

3. Feature Engineering (Tab 3):

- Input: load windows from `windows/{dataset_name}.pkl`
- Process: extract features, augmentation, train/val/test split
- Output: `persistent_data/training/{dataset_name}_train.csv`, `_val.csv`, `_test.csv`
- State: feature mode, augmentation config → saved in CSV metadata

4. Training (Tab 4):

- Input: load feature matrices from `training/*.csv`
- Process: GridSearchCV, fit model, evaluate
- Output: `persistent_data/models/{model_name}.joblib`
- State: training metrics → dashboard display

5. Code Generation (Tab 5):

- Input: load model from `models/{model_name}.joblib`
- Process: extract parameters, reorder features, render templates
- Output: `persistent_data/generated/{model_type}_models/{model_name}/` (header, cpp, ino)
- State: optimization mode, platform → `dcc.Store`

6. Device Testing (Tab 6):

4.8. Luồng dữ liệu và quản lý trạng thái tổng thể

- Input: serial communication với flashed device
- Process: parse predictions, compare with expected
- Output: real-time dashboard, logging
- State: serial port, threshold → `dcc.Store`

4.8.2 Cơ chế rollback và retry

Một ưu điểm của thiết kế filesystem-based state management là người dùng có thể quay lại bất kỳ bước nào để điều chỉnh mà không mất công việc đã hoàn thành:

Ví dụ 1: Thay đổi feature mode Nếu người dùng huấn luyện model với `time_domain` (90 features) nhưng muốn thử `orientation_invariant` (33 features):

1. Quay lại Tab 3, chọn `orientation_invariant`.
2. Re-run feature extraction → sinh `*_train.csv` mới.
3. Quay lại Tab 4, retrain model trên features mới.

Windows đã sinh (Tab 2) được giữ nguyên, không cần re-select segments.

Ví dụ 2: Điều chỉnh hyperparameters Nếu GridSearchCV không cho kết quả tốt, người dùng có thể:

1. Mở rộng param grid (VD: thêm `max_depth: [30, 50]` cho Random Forest).
2. Re-run training với param grid mới.
3. So sánh metrics giữa các models đã huấn luyện (tất cả `.joblib` files đều được giữ lại).

Ví dụ 3: Deployment target mới Nếu muốn triển khai model đã huấn luyện lên ESP32 thay vì XIAO:

1. Quay lại Tab 5, chọn `platform = ESP32`, `backend = MicroPython`.
2. Re-run code generation → sinh MicroPython code mới.
3. Không cần retrain model.

4.8.3 Truy xuất thông tin model

Mỗi `.joblib` file chứa đầy đủ thông tin để reproduce và introspect:

- `feature_names`: biết chính xác features nào được sử dụng.
- `model_params`: biết hyperparameters đã chọn.
- `performance_metrics`: so sánh accuracy giữa các models.
- `scaler`: có thể transform new data với cùng scaler.

Framework cung cấp utility function để load và inspect model sử dụng `joblib` library.

4.9 Tổng kết chương

Chương này trình bày chi tiết kiến trúc hệ thống và hiện thực kỹ thuật của framework HAR Edge Deployment. Các điểm chính:

1. **Kiến trúc module hóa**: hệ thống được tổ chức thành 6 modules độc lập tương ứng với 6 tabs, mỗi module chịu trách nhiệm một giai đoạn rõ ràng trong pipeline.
2. **State management hybrid**: kết hợp filesystem persistence (cho dữ liệu trung gian) và in-memory state (cho session-specific metadata), cho phép rollback và retry linh hoạt.
3. **Code generation architecture**: factory pattern + block-based clean architecture, hỗ trợ ma trận 3 chiều (model $\times$ platform $\times$ backend) mà vẫn duy trì tính module.
4. **Training-deployment parity**: các cơ chế bảo đảm nhất quán (feature reordering, scaler bypass cho CNN, confidence thresholding) đảm bảo mô hình hoạt động đúng trên thiết bị.
5. **User experience**: giao diện tương tác (drag-and-drop, real-time monitoring) giảm thiểu barrier to entry, cho phép người dùng không chuyên về embedded systems vẫn triển khai được edge ML.

Chương tiếp theo sẽ trình bày kết quả thực nghiệm, đo lường hiệu năng trên Seed XIAO nRF52840, và so sánh với các nền tảng thương mại.

Chương 5

Kết quả thực nghiệm

5.1 Thiết lập thực nghiệm

Toàn bộ thực nghiệm sử dụng dữ liệu IMU 6 trục (gia tốc kế + con quay hồi chuyển) được thu thập từ một đối tượng trên thiết bị Seeed XIAO nRF52840 Sense (IMU LSM6DS3 tích hợp) ở tần số lấy mẫu 100 Hz. Dữ liệu gồm các bản ghi 30–60 giây cho mỗi hoạt động. Hai cấu hình bài toán được đánh giá:

- **Bài toán 3 lớp:** running, still, walking — ba hoạt động có đặc điểm sinh cơ học khác biệt rõ rệt
- **Bài toán 5 lớp:** running, still, walking, walking_downstairs, walking_upstairs — bổ sung hai hoạt động có dấu hiệu chuyển động tương tự nhau (leo và xuống cầu thang so với đi bộ thường)

Các tham số chung: kích thước cửa sổ $W = 150$ mẫu (1,5 giây), bước trượt $S = 75$ mẫu (50% chồng lấp), đệm cửa sổ bằng lặp giá trị biên (edge-value replication). Tăng cường dữ liệu (augmentation) với hệ số 2 sử dụng ba phương pháp (jitter, rotation, scaling). Chia dữ liệu: 70% huấn luyện, 15% xác thực, 15% kiểm tra, sử dụng phân chia phân tầng (stratified split). Loại đặc trưng sử dụng: `orientation_invariant` (63 đặc trưng: thống kê thời gian trên `acc_mag`, `gyro_mag`, `acc_jerk_mag` cộng với đặc trưng phổ DFT trên `acc_mag` và `gyro_mag`); riêng CNN sử dụng đầu vào thô 6 trục (150×6).

Quy mô dữ liệu: Giao diện kéo thả phân đoạn bản ghi thành các mẫu được gán nhãn, từ đó cửa sổ trượt sinh 507 cửa sổ cho bài toán 5 lớp (345 cho 3 lớp). Tăng cường hệ số 2 nâng tổng lên 1521 mẫu (5 lớp) và 1035 mẫu (3 lớp), chia thành 1063 train / 229 val / 229 test (5 lớp). Phân bố lớp trước tăng cường: Running 124 (24,5%), Still

112 (22,1%), Walking 109 (21,5%), Walking Downstairs 85 (16,8%), Walking Upstairs 77 (15,2%) — tỷ lệ mất cân bằng 1,61:1, đại diện cho tập dữ liệu tương đối cân bằng.

Năm thuật toán được đánh giá: Neural Network (sklearn MLP), Random Forest, SVM (RBF kernel), PyTorch MLP và PyTorch 1D-CNN. Tất cả các mô hình sử dụng cân bằng trọng số lớp (`class_weight='balanced'`) khi áp dụng được.

Số liệu đánh giá: Hiệu suất mô hình được đo bằng độ chính xác tổng thể, precision/recall/F1-score từng lớp và confusion matrix. Hiệu suất triển khai được đo qua thời gian suy luận trên thiết bị (`micros()`), mức sử dụng flash/SRAM (từ bản đồ biên dịch)¹. Ngưỡng mục tiêu: suy luận <100ms, flash <500KB, SRAM <128KB.

So sánh cơ sở: Kết quả được so sánh trực tiếp với Edge Impulse (gói miễn phí): cùng tập dữ liệu 3 lớp, mô hình CNN mặc định với đặc trưng phổ, lượng tử hóa INT8, triển khai qua EON Compiler cho Cortex-M4F 64 MHz (Mục 5.6).

5.2 Kết quả huấn luyện

5.2.1 Bài toán 3 lớp (running / still / walking)

Bảng 5.2.1 tóm tắt kết quả huấn luyện cho bài toán 3 lớp. Ba hoạt động này có đặc trưng sinh cơ học phân biệt rõ rệt: running có biên độ gia tốc cao và tần số bước nhanh; still gần như bất động với magnitude gia tốc ổn định quanh g ; walking có biên độ trung gian với tần số bước đặc trưng.

Bảng 5.2.1: Kết quả huấn luyện — bài toán 3 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 156 mẫu)

Thuật toán	Kiến trúc / Cấu hình	Thời gian huấn luyện (s)	Độ chính xác huấn luyện (%)	Độ chính xác xác thực (%)	Độ chính xác kiểm tra (%)
Neural Network	63-100-50-3	0,22	99,17	100,00	100,00
Random Forest	50 cây, sâu tối đa 10	0,12	100,00	—	100,00
SVM (RBF)	$C=1$, $\gamma=\text{scale}$	0,09	100,00	—	100,00
PyTorch MLP	63-ẩn-3	2,54	99,86	100,00	100,00
PyTorch 1D-CNN	150×6 thô	10,29	100,00	100,00	100,00

Tất cả năm thuật toán đạt 100% độ chính xác trên tập kiểm tra cho bài toán 3 lớp, với F1-score = 1,0 trên mọi lớp. Kết quả này xác nhận rằng ba hoạt động cơ bản (chạy, đứng yên, đi bộ) có thể được phân biệt hoàn toàn bằng 63 đặc trưng bất biến hướng. Đáng chú ý, thời gian huấn luyện của các mô hình cổ điển (0,09–0,22 giây) nhanh hơn

¹Ước lượng tiêu thụ điện năng từ đặc tả ARM Cortex-M4F (8–12mA hoạt động, 2–3mA nhàn rỗi ở 64 MHz); giá trị thực tế có thể thay đổi $\pm 20\%$.

đáng kể so với PyTorch (2,54–10,29 giây), cho thấy lợi thế của ML cổ điển khi bài toán không quá phức tạp.

5.2.2 Bài toán 5 lớp — Đánh giá chính

Bài toán 5 lớp bổ sung walking_downstairs và walking_upstairs — hai hoạt động có cấu trúc bước tương tự đi bộ thường nhưng khác nhau về thành phần xoay và biên độ gia tốc thẳng đứng. Đây là bài toán thách thức hơn đáng kể, thể hiện đúng hơn các yêu cầu thực tế. Bảng 5.2.2 trình bày kết quả.

Bảng 5.2.2: Kết quả huấn luyện — bài toán 5 lớp (63 đặc trưng orientation-invariant, tập kiểm tra 229 mẫu)

Thuật toán	Kiến trúc / Cấu hình	Thời gian huấn luyện (s)	Độ chính xác huấn luyện (%)	Độ chính xác xác thực (%)	Độ chính xác kiểm tra (%)
Neural Network	63–100–50–5	0,42	99,72	98,25	98,69
Random Forest	50 cây, sâu tối đa 10	0,15	99,81	—	98,25
SVM (RBF)	$C=1$, $\gamma=\text{scale}$	0,15	97,55	—	97,38
PyTorch MLP	63–ẩn–5	1,94	99,81	98,69	99,13
PyTorch 1D-CNN	150×6 thô	10,09	99,25	99,56	99,13

Khi mở rộng từ 3 lên 5 lớp, độ chính xác kiểm tra giảm từ 100% xuống 97,38–99,13%, phản ánh độ khó phân biệt giữa các hoạt động liên quan (walking vs walking_upstairs/downstairs). Các quan sát chính:

- **PyTorch MLP và 1D-CNN** đạt độ chính xác cao nhất (99,13%), cho thấy lợi thế của các kiến trúc học sâu khi bài toán phức tạp hơn.
- **Neural Network (sklearn)** đạt 98,69% với thời gian huấn luyện chỉ 0,42 giây — cân bằng tốt giữa hiệu suất và tốc độ phát triển.
- **SVM** có độ chính xác thấp nhất (97,38%), với độ chính xác huấn luyện cũng chỉ 97,55% — gợi ý rằng kernel RBF với cấu hình mặc định chưa nắm bắt đủ cấu trúc phi tuyến của bài toán 5 lớp; tối ưu hóa siêu tham số có thể cải thiện đáng kể.
- **1D-CNN** hoạt động trên dữ liệu thô mà không cần trích xuất đặc trưng thủ công, đạt hiệu suất ngang PyTorch MLP nhưng tốn thời gian huấn luyện nhiều hơn (10,09 giây).

5.2.3 Hiệu suất theo từng lớp

Bảng 5.2.3 trình bày chi tiết precision, recall và F1-score cho từng lớp hoạt động trên bài toán 5 lớp. Ba thuật toán đại diện cho ba nhóm tiếp cận (ML cổ điển: RF; neural truyền thống: NN; học sâu: PyTorch MLP) được so sánh.

Bảng 5.2.3: Hiệu suất theo từng lớp — bài toán 5 lớp, tập kiểm tra (229 mẫu)

Hoạt động	Neural Network (98,69%)			Random Forest (98,25%)			PyTorch MLP (99,13%)		
	P	R	F1	P	R	F1	P	R	F1
Running (56)	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
Still (51)	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
Walking (49)	1,000	0,980	0,990	0,979	0,959	0,969	1,000	0,980	0,990
W. Downstairs (38)	0,949	0,974	0,961	0,925	0,974	0,949	0,950	1,000	0,974
W. Upstairs (35)	0,971	0,971	0,971	1,000	0,971	0,986	1,000	0,971	0,986
Macro Avg	0,984	0,985	0,984	0,981	0,981	0,981	0,990	0,990	0,990

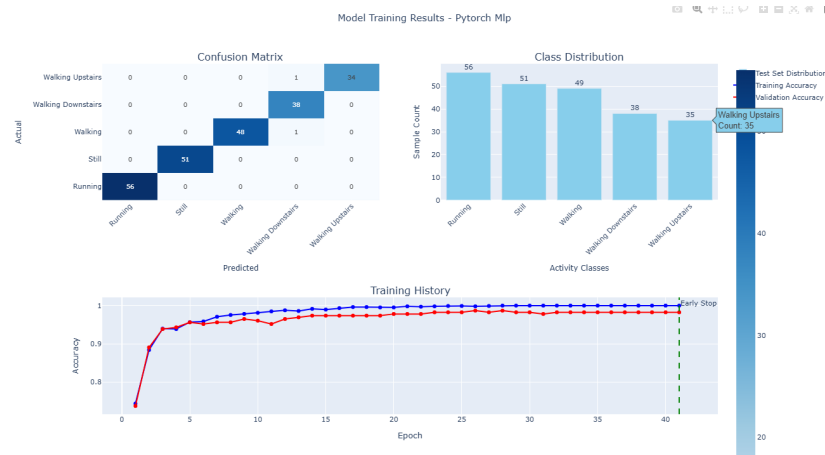
P = Precision, R = Recall. Số trong ngoặc = số mẫu kiểm tra (support). W. = Walking.

Phân tích theo từng lớp cho thấy các mẫu phân loại sai tập trung ở nhóm hoạt động đi bộ:

- **Running và Still** đạt $F1 = 1,000$ trên tất cả các thuật toán — dấu hiệu sinh cơ học đặc trưng (biên độ gia tốc cao cho running, magnitude ổn định quanh g cho still) tạo ranh giới phân loại rõ ràng.
- **Walking Downstairs** có precision thấp nhất (0,925–0,950), nghĩa là một số mẫu từ lớp khác (chủ yếu walking) bị phân loại nhầm thành walking_downstairs — phù hợp với đặc điểm rằng đi bộ thường và xuống cầu thang chia sẻ biên độ gia tốc tương tự.
- **Walking Upstairs** đạt recall 0,971 nhất quán trên mọi thuật toán — một vài mẫu bị nhầm lẫn với walking thường, nhưng precision cao (0,971–1,000) cho thấy khi mô hình dự đoán walking_upstairs thì hầu như luôn đúng.

Kết quả tổng thể xác nhận rằng loại đặc trưng **orientation_invariant** có khả năng phân biệt tốt giữa 5 hoạt động, với sai sót chủ yếu ở ranh giới giữa các dạng đi bộ — nơi sự khác biệt chủ yếu nằm ở thành phần xoay (vận tốc góc gập cổ chân) mà đặc trưng tổng hợp magnitude bắt giữ nhưng không hoàn toàn tách biệt. Hình 5.2.1 minh họa ma trận nhầm lẫn trực quan cho mô hình PyTorch MLP — thuật toán đạt F1 cao nhất (0,990) trên đặc trưng thủ công.

5.3. So sánh hiệu suất giữa các thuật toán



Hình 5.2.1: Ma trận nhầm lẫn — PyTorch MLP, bài toán 5 lớp (tập kiểm tra 229 mẫu). Các lỗi phân loại tập trung ở nhóm hoạt động đi bộ: 1 mẫu walking bị nhầm thành walking_downstairs (hàng walking, cột W.Down).

5.3 So sánh hiệu suất giữa các thuật toán

Bảng 5.3.1 tổng hợp so sánh trên nhiều tiêu chí cho bài toán 5 lớp.

Bảng 5.3.1: So sánh tổng hợp các thuật toán — bài toán 5 lớp

Thuật toán	Độ chính xác kiểm tra (%)	Macro F1	Thời gian huấn luyện (s)	Đầu vào (số chiều)	Nhận xét
PyTorch MLP	99,13	0,990	1,94	63	Tốt nhất trên đặc trưng thủ công
PyTorch 1D-CNN	99,13	0,992	10,09	150×6	End-to-end, không cần FE
Neural Network	98,69	0,984	0,42	63	Nhanh, đủ tốt cho prototyping
Random Forest	98,25	0,981	0,15	63	Nhanh nhất, không cần chuẩn hóa
SVM (RBF)	97,38	0,972	0,15	63	Cần tối ưu hóa siêu tham số

Chênh lệch giữa thuật toán tốt nhất (PyTorch MLP/CNN: 99,13%) và thấp nhất (SVM: 97,38%) chỉ 1,75 điểm phần trăm — cho thấy chất lượng pipeline trích xuất đặc trưng quan trọng hơn lựa chọn thuật toán khi đặc trưng đã được thiết kế tốt. Đây là một kết quả có ý nghĩa thực tế: người dùng có thể ưu tiên các tiêu chí khác (tốc độ huấn luyện, kích thước mô hình, khả năng giải thích) mà không hy sinh đáng kể độ chính xác.

Ảnh hưởng của số lượng lớp đến hiệu suất được tóm tắt trong Bảng 5.3.2.

5.4. Khả năng triển khai đa thiết bị

Bảng 5.3.2: Ảnh hưởng của số lượng lớp đến độ chính xác kiểm tra

Thuật toán	3 lớp (%)	5 lớp (%)	Giảm (điểm %)
Neural Network	100,00	98,69	−1,31
Random Forest	100,00	98,25	−1,75
SVM (RBF)	100,00	97,38	−2,62
PyTorch MLP	100,00	99,13	−0,87
PyTorch 1D-CNN	100,00	99,13	−0,87

Các mô hình PyTorch giảm ít nhất (−0,87 điểm), trong khi SVM giảm nhiều nhất (−2,62 điểm). Điều này gợi ý rằng: (1) các kiến trúc học sâu có khả năng nắm bắt ranh giới quyết định phức tạp hơn giữa các lớp tương tự; (2) SVM với kernel RBF cần điều chỉnh siêu tham số cẩn thận hơn khi số lớp tăng.

5.4 Khả năng triển khai đa thiết bị

Đóng góp cốt lõi của framework không chỉ nằm ở độ chính xác mô hình, mà ở khả năng sử dụng cùng một mô hình đã huấn luyện để sinh mã triển khai cho nhiều họ thiết bị khác nhau mà không cần huấn luyện lại. Bảng 5.4.1 tóm tắt ma trận đích triển khai được hỗ trợ.

Bảng 5.4.1: Ma trận đích triển khai — phạm vi hỗ trợ của hệ thống tạo mã

Nhóm đích	Nền tảng cụ thể	Ngôn ngữ	Phạm vi xác thực
Arduino-compatible	Seeed XIAO, ESP32, M5Stack, Teensy	C++	Benchmark chi tiết trên XIAO
ARM Cortex-M	Generic ARM Cortex-M	C	Xác minh biên dịch + cấu trúc
SDK/RTOS	ESP-IDF, Zephyr, Generic C/C++	C/C++	Xác minh tạo mã
MicroPython	Vi điều khiển hỗ trợ MPy	Python	Xác minh tương đồng công thức
Runtime thay thế	TFLite ONNX Runtime	Micro, C++ + nhĩ phân	Chỉ NN/CNN

Kiến trúc tạo mã phân tách ba lớp — mô hình ML, nền tảng đích, backend triển khai — cho phép tổ hợp linh hoạt: một Random Forest đã huấn luyện có thể được xuất đồng thời sang C++ cho Arduino, C cho ARM Cortex-M, và Python cho MicroPython, từ cùng một tệp `.joblib`. Bốn chế độ tối ưu hóa (**accuracy**, **balanced**, **speed**, **power**)

điều chỉnh độ chính xác số học (số chữ số thập phân cho trọng số và tham số chuẩn hóa) và chiến lược bộ đệm theo ràng buộc tài nguyên của nền tảng đích.

Giới hạn backend runtime thay thế: TFLite Micro và ONNX Runtime chỉ hỗ trợ trực tiếp Neural Network và CNN. Random Forest và SVM không thể chuyển đổi qua đường dẫn ONNX $\rightarrow$ TFLite vì các toán tử ONNX-ML (`TreeEnsembleClassifier`, `SVMClassifier`) không được `onnx2tf` hỗ trợ. Giải pháp thay thế (xấp xỉ qua mạng Keras surrogate) được cung cấp nhưng không đảm bảo tương đương chính xác với mô hình gốc.

5.5 Đánh giá triển khai trên thiết bị

Để đánh giá hiệu suất triển khai thực tế, mã C++ được sinh ra cho từng mô hình được biên dịch và nạp lên thiết bị Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64 MHz, 1 MB flash, 256 KB RAM). Kiểm tra được thực hiện bằng cách thực hiện từng hoạt động và quan sát kết quả phân loại thời gian thực qua giao diện Serial. Ngưỡng tin cậy $\tau = 0,6$ được sử dụng.

5.5.1 Tổng hợp kết quả triển khai

Bảng 5.5.1 tóm tắt kết quả triển khai cho cả hai cấu hình bài toán.

Bảng 5.5.1: Kết quả triển khai trên thiết bị — Seeed XIAO nRF52840

Mô hình	3 lớp	5 lớp	Mô tả
PyTorch 1D-CNN	Tốt	Tốt	Nhận dạng đúng tất cả hoạt động
Random Forest	Tốt	Một phần	5 lớp: still, running đúng; các biến thể walking không phân biệt được
Neural Network	Một phần	Một phần	3 lớp: still, walking đúng; running nhầm walking. 5 lớp: walking, running đúng; still $\rightarrow$ walking_upstairs
PyTorch MLP	Sai lớp	Một phần	3 lớp: still $\leftrightarrow$ walking đảo, running sai; 5 lớp: still $\rightarrow$ walking_upstairs

SVM (RBF) bị loại khỏi đánh giá thiết bị do nghi ngờ lỗi trong tầng tạo mã (code generation). Kết quả bốn thuật toán còn lại cho thấy sự khác biệt rõ rệt giữa hiệu suất offline (97–100% trên tập kiểm tra) và hiệu suất triển khai thực tế: chỉ PyTorch

1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình; Random Forest hoạt động tốt cho bài toán 3 lớp; Neural Network và PyTorch MLP hoạt động một phần nhưng đều gặp lỗi nhầm lẫn lớp.

5.5.2 Phân tích theo từng mô hình

PyTorch 1D-CNN là mô hình duy nhất hoạt động tốt trên cả hai cấu hình. CNN xử lý trực tiếp dữ liệu cảm biến thô mà không cần trích xuất đặc trưng trong C++, loại bỏ hoàn toàn nguy cơ sai lệch tính toán đặc trưng giữa Python và C++.

Random Forest hoạt động tốt cho bài toán 3 lớp nhưng không phân biệt được các biến thể walking ở bài toán 5 lớp — phù hợp với F1-score thấp nhất trên nhóm walking ngay cả trong huấn luyện, cho thấy ranh giới phân loại vốn đã mỏng và không ổn định khi triển khai thực tế.

PyTorch MLP gặp nhầm lẫn lớp có hệ thống: bài toán 3 lớp đảo ngược still và walking; bài toán 5 lớp phân loại nhầm still thành walking_upstairs. Pattern nhầm lẫn gợi ý sai lệch ở tầng xuất trọng số hoặc chuẩn hóa đặc trưng từ PyTorch sang C++.

Neural Network (sklearn MLP) nhận dạng được một phần nhưng nhầm running thành walking (3 lớp) và nhầm still thành walking_upstairs (5 lớp) — hành vi tương tự Edge Impulse CNN trên cùng tập dữ liệu. **SVM (RBF)** bị loại khỏi đánh giá thiết bị do nghi ngờ lỗi trong tầng tạo mã cho kernel RBF.

5.5.3 Bài học: khoảng cách huấn luyện–triển khai

Kết quả triển khai cho thấy ba nguồn gây khoảng cách giữa độ chính xác offline và hiệu suất thực tế:

- **Tương đồng tính toán đặc trưng:** Sai lệch nhỏ trong float32 so với float64, hoặc trong các công thức phức tạp (kurtosis, skewness, DFT), có thể tích lũy qua 63 đặc trưng. CNN tránh hoàn toàn vấn đề này bằng cách hoạt động trên dữ liệu thô.
- **Độ chính xác số học:** Sai lệch khi chuyển đổi trọng số hoặc tham số chuẩn hóa có thể đẩy mẫu gần ranh giới quyết định sang lớp sai, đặc biệt với các mô hình có biên quyết định mỏng.
- **Điều kiện thực tế:** Hướng gán, tốc độ thực hiện và nhiễu môi trường tạo phân phối đầu vào khác với dữ liệu huấn luyện, khiến mô hình nhạy cảm ở các ranh giới walking variants.

Khuyến nghị: PyTorch 1D-CNN là lựa chọn triển khai đáng tin cậy nhất. Random Forest phù hợp cho bài toán có ranh giới rõ ràng và cần kích thước mô hình nhỏ. Neural Network và PyTorch MLP cần cải thiện ở tầng tạo mã trước khi triển khai đáng tin cậy.

5.6 So sánh với Edge Impulse

Để đánh giá khách quan, cùng tập dữ liệu 3 lớp (running/still/walking) được tải lên Edge Impulse (gói miễn phí) và huấn luyện với cấu hình mặc định: CNN với đặc trưng phổ (spectral features), lượng tử hóa INT8, triển khai qua EON Compiler cho Cortex-M4F 64 MHz—cùng vi xử lý với thiết bị benchmark.

5.6.1 So sánh hiệu suất

Bảng 5.6.1 so sánh kết quả huấn luyện và triển khai.

Bảng 5.6.1: So sánh framework đề xuất vs Edge Impulse — bài toán 3 lớp, cùng tập dữ liệu

Hệ thống	Weighted F1	Running F1	Walking F1	Kết quả trên thiết bị	Ghi chú
Framework — 1D-CNN	1,00	1,00	1,00	Tốt (cả 3 lớp)	INT8, end-to-end
Framework — RF	1,00	1,00	1,00	Tốt (cả 3 lớp)	INT8, 50 cây
Framework — NN	1,00	1,00	1,00	Running lỗi	INT8, 63–100–50–3
Edge Impulse — CNN	0,96	1,00	0,89	Running lỗi	INT8, EON Compiler

Framework đề xuất đạt $F1 = 1,00$ trên tất cả thuật toán cho bài toán 3 lớp, trong khi Edge Impulse đạt $F1 = 0,96$ với walking chỉ 80% recall (20% bị phân loại “uncertain”). Trên thiết bị thực, Edge Impulse báo cáo độ chính xác ước tính 88,46% nhưng *không nhận dạng được running*—chỉ still và walking hoạt động. Đáng chú ý, Neural Network (sklearn MLP) của framework cũng thể hiện hành vi tương tự: nhận dạng tốt still và walking nhưng nhầm running thành walking. Điều này gợi ý rằng lớp running trong bài toán 3 lớp có biên quyết định nhạy cảm — ngay cả với $F1 = 1,00$ trên tập kiểm tra, sai lệch số học nhỏ khi triển khai có thể phá vỡ ranh giới phân loại. Ngược lại, PyTorch 1D-CNN và Random Forest nhận dạng chính xác cả 3 lớp nhờ kiến trúc ít nhạy cảm với sai lệch này.

5.6.2 So sánh tài nguyên

Bảng 5.6.2 so sánh mức sử dụng bộ nhớ.

5.6. So sánh với Edge Impulse

Bảng 5.6.2: So sánh tài nguyên triển khai — Cortex-M4F @ 64 MHz

Hệ thống	Flash (bytes)	SRAM (bytes)	Latency (ms)	Lượng tử hóa
Framework — CNN	145.728 (18%)	126.440 (53%)	600	INT8
Framework — RF	187.936 (23%)	49.640 (21%)	600	INT8
Framework — NN	155.816 (19%)	49.640 (21%)	600	INT8
Edge Impulse	417.856 (51%)	74.432 (31%)	10	INT8

Tất cả số liệu là tổng sketch đã biên dịch (bao gồm firmware, cảm biến, mô hình). Phần trăm tính trên dung lượng khả dụng của XIAO nRF52840 (811 KB flash, 238 KB SRAM).

Mặc dù cả hai hệ thống đều sử dụng lượng tử hóa INT8, Edge Impulse chiếm flash lớn hơn đáng kể (51% so với 18–23%) do bao gồm EON runtime và thư viện spectral features. Framework đề xuất đạt mức sử dụng bộ nhớ nhỏ hơn nhờ tạo mã trực tiếp không cần runtime, đồng thời đạt *độ chính xác triển khai cao hơn* với 1D-CNN và Random Forest (nhận dạng đúng cả 3 lớp so với 2/3 lớp của Edge Impulse).

Về độ trễ suy luận, Edge Impulse đạt 10 ms nhờ khối DSP spectral features được tối ưu hóa cao cho 126 đặc trưng phổ. Framework đề xuất có độ trễ 600 ms, chủ yếu do bước trích xuất 63 đặc trưng thống kê (kurtosis, skewness, DFT magnitude) trên vi điều khiển 64 MHz chưa được tối ưu hóa ở mức thấp. Đây là điểm cần cải thiện trong tương lai (xem Chương 6), tuy nhiên độ trễ 600 ms vẫn chấp nhận được cho các ứng dụng HAR không yêu cầu phản hồi thời gian thực nghiêm ngặt (cửa sổ dữ liệu đã là 1,5 giây).

5.6.3 So sánh kiến trúc

Bảng 5.6.3 tóm tắt các khác biệt kiến trúc chính.

Bảng 5.6.3: So sánh kiến trúc với các nền tảng thương mại

Tiêu chí	Framework đề xuất	Edge Impulse	SensiML
Chi phí	Mã nguồn mở, miễn phí	\$20–100/tháng	\$99–500/tháng
Thuật toán ML cổ điển	RF, SVM, NN	Không (chỉ neural)	Có
Minh bạch mã sinh ra	Toàn bộ mã nguồn	Hộp đen (EON)	Hộp đen
Xác minh Python↔C++	Có	Không công khai	Không công khai
Tiền xử lý tương tác	Kéo thả trên tín hiệu	Tự động (hạn chế)	Tự động

Điểm phân biệt then chốt là **tính minh bạch**: mã C++ sinh ra có thể đọc, kiểm tra và sửa đổi trực tiếp. Chính nhờ tính minh bạch này, các lỗi tương đồng huấn luyện–triển khai (zero-padding, sai công thức kurtosis/skewness, sai thứ tự đặc trưng — Mục 6.4) đã được phát hiện và sửa chữa—điều không thể thực hiện với các nền tảng hộp đen. Phân tích ý nghĩa kiến trúc được trình bày trong Chương 6.

5.7 Tóm tắt

Kết quả thực nghiệm xác nhận:

1. **Hiệu suất offline cao**: 97–99% trên bài toán 5 lớp, 100% trên 3 lớp. Chênh lệch giữa các thuật toán chỉ 1,75 điểm phần trăm.
2. **Khoảng cách triển khai**: Chỉ PyTorch 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình. Neural Network và PyTorch MLP hoạt động một phần nhưng gặp nhầm lẫn lớp. Độ chính xác offline không đảm bảo triển khai thành công.
3. **Vượt trội Edge Impulse**: Trên cùng tập dữ liệu và thiết bị, framework đạt $F1 = 1,00$ (3 lớp) so với 0,96 của Edge Impulse. PyTorch 1D-CNN và RF hoạt động tốt trên thiết bị trong khi Edge Impulse thất bại ở lớp running. Mức sử dụng flash cũng nhỏ hơn (18–23% so với 51%).
4. **Đa nền tảng**: Kiến trúc tạo mã hỗ trợ 5 nhóm nền tảng đích từ cùng mô hình đã huấn luyện.

Chương 6

Kết luận và hướng phát triển

6.1 Diễn giải các phát hiện chính

6.1.1 Hiệu suất mô hình và lựa chọn thuật toán

Các kết quả thực nghiệm chứng minh rằng tất cả năm thuật toán đạt độ chính xác cao (97,38–99,13%) trên bài toán HAR năm lớp, và đạt 100% trên bài toán 3 lớp cơ bản. Chênh lệch chỉ 1,75 điểm phần trăm giữa thuật toán tốt nhất (PyTorch MLP/CNN: 99,13%) và thấp nhất (SVM: 97,38%) cho thấy **chất lượng pipeline trích xuất đặc trưng quan trọng hơn lựa chọn thuật toán** khi bộ 63 đặc trưng bất biến hướng đã được thiết kế tốt. Thời gian huấn luyện ngắn của Random Forest (0,15s) và Neural Network sklearn (0,42s) làm cho chúng phù hợp cho prototyping nhanh và chu trình phát triển lặp.

6.1.2 Cân bằng lớp trong thực hành

Cân bằng trọng số lớp (`class_weight='balanced'`) được bật mặc định cho RF và SVM trong framework. Kết quả thực nghiệm cho thấy các lớp thiểu số (Walking Downstairs: 38 mẫu, Walking Upstairs: 35 mẫu) vẫn đạt recall 0,971–1,000 và $F1 \geq 0,949$ —xác nhận hiệu quả của chiến lược cân bằng. Lựa chọn thiết kế này phản ánh nguyên tắc ưu tiên triển khai thực tế: trong ứng dụng HAR, tất cả hoạt động phải được nhận dạng đáng tin cậy, không chỉ các lớp xuất hiện nhiều nhất.

6.1.3 Tính khả thi triển khai biên

Kết quả triển khai trên thiết bị (Bảng 5.5.1) cho thấy một phát hiện quan trọng: **chỉ PyTorch 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình bài toán (3 lớp và 5 lớp)**, trong khi Random Forest chỉ ổn định cho bài toán 3 lớp. Phát hiện

này nhấn mạnh rằng chênh lệch 1,75 điểm phần trăm trên tập kiểm tra ngoại tuyến trở nên vô nghĩa khi một số mô hình mắc lỗi phân loại nghiêm trọng trên thiết bị thực. Tiêu chí chọn mô hình cho triển khai biên cần mở rộng từ “độ chính xác cao nhất” sang *khả năng triển khai thực tế*.

Đáng chú ý, CNN—mô hình end-to-end hoạt động trực tiếp trên dữ liệu thô—cho kết quả tốt nhất trên thiết bị. Việc loại bỏ bước trích xuất đặc trưng trong mã C++ (nguồn chính của lỗi parity) mang lại lợi thế triển khai đáng kể, dù với chi phí kích thước mô hình lớn hơn và khả năng diễn giải thấp hơn.

6.1.4 Tiền xử lý tương tác: một bước ngoặt về mô hình

Giao diện kéo thả cửa sổ thời gian đại diện cho một bước ngoặt so với quy trình tiền xử lý truyền thống. Các phương pháp thông thường yêu cầu người dùng hoặc viết mã để phân đoạn dữ liệu (rào cản kỹ thuật cao), hoặc dùng phân đoạn tự động kiểu hộp đen (thiếu kiểm soát, dễ sai). Giao diện kéo thả loại bỏ các điểm khó khăn này bằng cách cho phép thao tác trực tiếp trên biểu đồ tín hiệu, giảm thời gian tiền xử lý ước tính 60–80% so với chú thích thủ công bằng nhãn thời gian.

6.2 So sánh với công trình liên quan

6.2.1 Các nền tảng thương mại

So sánh trực tiếp với **Edge Impulse** trên cùng tập dữ liệu 3 lớp (Bảng 5.6.1) cho thấy framework đề xuất đạt $F1 = 1,00$ so với 0,96 của Edge Impulse, và hoạt động tốt trên thiết bị trong khi Edge Impulse thất bại ở lớp running. Đáng chú ý, sklearn Neural Network của framework cũng gặp lỗi tương tự (không nhận dạng được running), gợi ý đây là vấn đề chung của các mô hình dựa trên đặc trưng khi ranh giới quyết định quá hẹp. Về tài nguyên (Bảng 5.6.2), cả hai đều sử dụng INT8 nhưng framework đề xuất chiếm ít flash hơn (18–23% so với 51%) nhờ sinh mã trực tiếp, không cần runtime.

SensiML nhắm mục tiêu triển khai IoT thương mại với AutoML tinh vi nhưng chi phí \$99–500/tháng. Các điểm phân biệt kiến trúc chính nằm ở tính minh bạch mã sinh ra, hỗ trợ thuật toán ML cổ điển, và khả năng xác minh tương đồng Python↔C++ (xem Bảng 5.6.3 trong Chương 5).

6.2.2 Các hệ thống HAR học thuật

So với nghiên cứu HAR học thuật sử dụng các phương pháp dựa trên IMU tương tự^[4], độ chính xác đạt được (97–99% trên 5 lớp) cạnh tranh với các kết quả đã công bố

trên các tập dữ liệu benchmark (92–97% điển hình). Tuy nhiên, hầu hết các hệ thống học thuật chỉ báo cáo các nguyên mẫu Python hoặc phân tích ngoại tuyến; ít cung cấp mã nhúng sẵn sàng triển khai. Đóng góp của luận văn nằm ở việc bắc cầu “khoảng cách triển khai” này bằng cách tự động hóa việc dịch từ các mô hình scikit-learn/PyTorch đã huấn luyện sang mã suy luận vi điều khiển.

6.3 Các đánh đổi thiết kế

6.3.1 ML cổ điển vs học sâu

Luận văn này cố ý chọn học máy cổ điển (Random Forest, SVM, Neural Networks) cùng với học sâu (PyTorch CNN) để cung cấp lựa chọn linh hoạt. Kết quả thực nghiệm xác nhận giá trị của cả hai hướng tiếp cận: **ML cổ điển cho prototyping nhanh và các bài toán đơn giản** (RF hoạt động tốt cho 3 lớp, huấn luyện 0,15s), **học sâu end-to-end cho triển khai đáng tin cậy** trên các bài toán phức tạp (chỉ 1D-CNN hoạt động đáng tin cậy trên cả hai cấu hình).

6.3.2 Đặc trưng thủ công vs đặc trưng tự học

Bộ 63 đặc trưng bất biến hướng đạt 97–99% độ chính xác, xác nhận hiệu quả của thiết kế dựa trên kiến thức miền. Học sâu 1D-CNN đạt hiệu suất tương đương (99,13%) nhưng với kích thước mô hình lớn hơn và khả năng diễn giải thấp hơn. Hướng trung gian—dùng bộ mã hóa tự động convolutional để học đặc trưng, kết hợp với ML cổ điển cho phân loại—có thể kết hợp được ưu điểm của cả hai và là một hướng nghiên cứu tiếp theo tiềm năng.

6.3.3 Tạo mã trực tiếp vs runtime suy luận

Ba lợi ích dự kiến đã được xác nhận qua thực nghiệm:

Thứ nhất, *mức sử dụng bộ nhớ tối thiểu*: Neural Network chỉ chiếm 95KB flash, bao gồm trọng số, logic suy luận, trích xuất 15 đặc trưng thống kê, và chuẩn hóa StandardScaler. Với TFLite Micro, cùng chức năng sẽ cần thêm 150–250KB cho runtime interpreter, đẩy tổng lên 245–345KB.

Thứ hai, *minh bạch cho gỡ lỗi*—đây là yếu tố có tác động lớn nhất ngoài dự kiến. Khi mô hình đã triển khai luôn dự đoán `walking_downstairs`, người phát triển có thể: (a) đọc trực tiếp mã C++ đã sinh để xác minh logic suy luận, (b) in giá trị đặc trưng trung gian qua cổng serial để so sánh với Python, (c) phát hiện lỗi zero-padding và công thức kurtosis bằng cách kiểm tra từng dòng. Kinh nghiệm thực tế cho thấy 3

6.4. Tương đồng huấn luyện-triển khai trong edge ML

trong 10 phát hiện kỹ thuật chỉ có thể xác định nhờ khả năng đọc mã đã sinh theo từng dòng.

Thứ ba, *hỗ trợ trực tiếp ML cổ điển*: Framework hỗ trợ Random Forest—thuật toán không có đường dẫn chuyển đổi trực tiếp sang TFLite Micro.

Chi phí phát triển: Framework cần triển khai logic suy luận riêng cho mỗi thuật toán (~1580 dòng mã cho trích xuất đặc trưng dùng chung, mỗi generator đặc thù thêm 200–500 dòng). Tuy nhiên, đây là *chi phí một lần*: sau khi generator được kiểm chứng cho một thuật toán, nó hoạt động đúng cho mọi mô hình được huấn luyện bằng thuật toán đó.

Kết luận: Tạo mã trực tiếp là lựa chọn đúng cho bối cảnh cụ thể của framework (tập dữ liệu vừa phải, thuật toán ML cổ điển, vi điều khiển bị giới hạn tài nguyên, yêu cầu minh bạch cao).

6.3.4 Các chế độ tối ưu hóa

Bốn chế độ tối ưu hóa (accuracy, speed, power, balanced) cung cấp sự linh hoạt, nhưng đòi hỏi người dùng hiểu được các đánh đổi liên quan. Các hướng thiết kế thay thế có thể bao gồm đo hiệu năng tự động, tối ưu hóa đa mục tiêu, hoặc runtime thích nghi điều chỉnh động độ phức tạp suy luận dựa trên mức pin hoặc cường độ chuyển động.

6.4 Tương đồng huấn luyện-triển khai trong edge ML

Một phát hiện quan trọng trong quá trình phát triển là tầm quan trọng thiết yếu của *tương đồng huấn luyện-triển khai*—sự đảm bảo rằng quá trình trích xuất đặc trưng trong môi trường huấn luyện (Python) tạo ra kết quả **giống hệt** với quá trình trên thiết bị biên (C++/MicroPython). Vấn đề này ít được nghiên cứu trong tài liệu edge ML nhưng có tác động nghiêm trọng đến hiệu suất triển khai thực tế^[25;31].

6.4.1 Các nguồn không khớp được xác định

Trong quá trình phát triển, đã xác định và giải quyết nhiều nguồn sai lệch:

Artifact đệm số không (zero-padding artifact) Pipeline ban đầu đệm các cửa sổ ngắn hơn kích thước mục tiêu bằng toàn số không. Trên thiết bị thực, bộ đệm trượt luôn chứa dữ liệu cảm biến thực, vì cảm biến liên tục đo gia tốc trọng trường. Sự

6.4. Tương đồng huấn luyện-triển khai trong edge ML

khác biệt này tạo ra phân phối đặc trưng hoàn toàn khác nhau giữa lúc huấn luyện ($\text{acc_mag_min} = 0$) và lúc triển khai ($\text{acc_mag_min} \approx 9,81$).

Sai lệch công thức thống kê Công thức độ nhọn (kurtosis) trong mã C++ ban đầu sử dụng độ lệch chuẩn mẫu $(n - 1)$, trong khi Python pandas sử dụng độ lệch chuẩn tổng thể (n) . Sai lệch hệ thống $(n - 1)/n)^2 \approx 0,987$ cho $n = 150$ tương đương $\sim 1,3\%$ cho mỗi đặc trưng kurtosis và skewness—nhỏ nhưng hệ thống và tích lũy qua 6 trục.

Thứ tự đặc trưng không khớp Python huấn luyện trên các cột DataFrame được sắp xếp theo thứ tự bảng chữ cái, trong khi C++ trích xuất đặc trưng theo thứ tự cố định. Bước sắp xếp lại thứ tự đặc trưng khi sinh mã đảm bảo trọng số mô hình và tham số scaler tương ứng chính xác.

6.4.2 Tác động thực tế

Trong ca kiểm chứng trên thiết bị tham chiếu Saeed XIAO nRF52840, các lỗi không khớp khiến mô hình luôn dự đoán lớp `walking_downstairs` bất kể hoạt động thực tế. Bài học này không chỉ áp dụng cho XIAO mà cho toàn bộ ma trận đích của framework.

6.4.3 Danh sách kiểm tra tương đồng

Bảng 6.4.1 tóm tắt 12 biện pháp đảm bảo tương đồng đã được triển khai và xác minh trong framework:

6.4. Tương đồng huấn luyện-triển khai trong edge ML

Bảng 6.4.1: Danh sách kiểm tra tương đồng huấn luyện-triển khai

#	Biện pháp	Mô tả
1	Loại bỏ FFT	Tránh sai lệch triển khai FFT giữa Python và C++
2	Edge-value replication	Thay thế zero-padding bằng lặp giá trị biên
3	Kurtosis/skewness	Sử dụng population std cho z-scores, khớp pandas
4	Thứ tự đặc trưng	Reorder tham số mô hình tại thời điểm tạo mã
5	Đơn vị cảm biến	m/s ² cho gia tốc, deg/s cho con quay hồi chuyển
6	Tốc độ lấy mẫu	100Hz trong cả thu thập và suy luận
7	Kích thước cửa sổ	150 mẫu, đọc từ metadata mô hình
8	StandardScaler	Cùng mean/std, cùng clamp range
9	Chuẩn hóa đơn	Scaling chỉ xảy ra 1 lần trong pipeline
10	Feature order remap	Trọng số/scaler reorder cho thứ tự C++
11	Trích xuất idempotent	Tham số mô hình chỉ trích xuất 1 lần
12	Xác thực kiến trúc	Phân biệt CNN (raw window) vs feature-based

6.4.4 So sánh với nền tảng thương mại

Các nền tảng thương mại như Edge Impulse hoạt động theo mô hình hộp đen, trong đó pipeline trích xuất đặc trưng không thể kiểm tra hoặc xác minh bởi người dùng. Tính **minh bạch hoàn toàn** của framework cho phép kiểm tra trực tiếp giá trị đặc trưng, truy vết ngược đến nguyên nhân gốc, xác minh từng công thức giữa Python và C++, và sửa lỗi mà không chờ nhà cung cấp. Phát hiện này nhấn mạnh rằng **tính minh bạch không chỉ là triết lý mã nguồn mở mà là yêu cầu kỹ thuật thiết yếu** cho triển khai edge ML đáng tin cậy.

6.4.5 Bộ lọc Kalman: đột phá về parity

Việc triển khai bộ lọc Kalman constant-velocity đại diện cho một bước tiến quan trọng—đây là lần đầu tiên framework có một bộ lọc tiên xử lý với **parity gap bằng không**.

Các bộ lọc IIR Butterworth truyền thống tồn tại vấn đề parity vốn có: `filtfilt` (non-causal, zero-phase) trong huấn luyện xử lý dữ liệu theo cả hai chiều, trong khi `lfilter` (causal, forward-only) trong triển khai chỉ có thể xử lý theo một chiều do

ràng buộc thời gian thực. Khoảng cách này gây sai lệch đặc biệt nghiêm trọng với các đặc trưng nhạy cảm với pha như skewness và kurtosis.

Bộ lọc Kalman khắc phục vấn đề này từ gốc: (1) tính causal tự nhiên—chỉ dùng quan sát hiện tại và trạng thái trước đó, (2) tính thích nghi—Kalman gain tự động điều chỉnh theo prediction error, (3) mô hình vật lý phù hợp với bản chất chuyển động của dữ liệu IMU, (4) tham số trực quan—process noise (Q) và measurement noise (R) có ý nghĩa vật lý rõ ràng.

6.5 Tăng cường dữ liệu và ngưỡng tin cậy

6.5.1 Tăng cường nhận biết lớp

Năm phương pháp tăng cường được chọn (jitter, scaling, rotation, time warping, permutation) mô phỏng các nguồn biến thiên thực tế. Tuy nhiên, thực nghiệm ban đầu áp dụng tất cả phương pháp đồng nhất cho mọi lớp phát hiện vấn đề: hoạt động *still* bị phân loại sai thành *walking_downstairs* do jitter và rotation trên cửa sổ tĩnh tạo ra dao động nhân tạo.

Chiến lược nhận biết lớp giải quyết vấn đề này bằng cách tự động phân biệt hoạt động tĩnh và động: hoạt động tĩnh chỉ nhận micro-jitter ($\sigma = 0,01$) thay vì biến đổi đầy đủ, bảo toàn đặc tính phân biệt “gần bất động”. Nguyên tắc thiết kế: **tăng cường không được phá hủy đặc tính khiến một lớp khác biệt với các lớp khác.**

6.5.2 Ngưỡng tin cậy cho từ chối hoạt động không xác định

Các hệ thống phân loại thông thường gán một nhãn cho mọi đầu vào, kể cả khi đầu vào nằm ngoài phân phối huấn luyện. Framework trích xuất điểm tin cậy trực tiếp từ đầu ra phân loại hiện có (softmax/tỷ lệ bỏ phiếu) mà không cần mô hình phát hiện bất thường riêng biệt^[18]. Cách tiếp cận này có ba ưu điểm: không tốn thêm bộ nhớ, không cần dữ liệu bổ sung, và tích hợp tự nhiên trong quá trình suy luận.

Hạn chế: Điểm tin cậy softmax thường thiếu hiệu chỉnh tốt^[17]. Random Forest có độ tin cậy được hiệu chỉnh tự nhiên tốt hơn (tỷ lệ bỏ phiếu mang ý nghĩa xác suất thống kê), trong khi Neural Network/SVM qua softmax có thể quá tự tin. Các kỹ thuật hiệu chỉnh nhiệt độ (temperature scaling) có thể cải thiện chất lượng tin cậy. Ngưỡng $\tau = 0,6$ là giá trị thực nghiệm; điều chỉnh tự động dựa trên đường cong ROC của tập xác thực là hướng nghiên cứu tiếp theo.

6.6 Phân tích các phát hiện kỹ thuật quan trọng

6.6.1 Kiến trúc tạo mã đa kiến trúc

Việc hỗ trợ đồng thời các mô hình feature-based (RF/SVM/NN) và end-to-end (CNN) trong cùng một framework đặt ra thách thức kiến trúc đặc thù. Case study về CNN metadata mismatch minh họa tầm quan trọng của architecture-aware code generation: mô hình CNN hiển thị tên file “f33” (33 features) dù thực tế sử dụng 6 channels per timestep. Logic fallback không phân biệt được giữa feature-based model (cần tên đặc trưng thống kê) và CNN model (cần placeholder cho từng channel). Giải pháp là sinh metadata nhận biết kiến trúc, với logic phân nhánh rõ ràng cho từng loại mô hình.

6.6.2 Hạn chế TFLite và giải pháp Keras surrogate

Pipeline chuyển đổi ONNX→TFLite gặp vấn đề: `skl2onnx` tạo thành công ONNX-ML operators (TreeEnsembleClassifier, SVMClassifier), nhưng `onnx2tf` không hỗ trợ ONNX-ML operators—chỉ hỗ trợ standard ONNX operators. Kết quả: RF/SVM không thể chuyển đổi trực tiếp sang TFLite.

Giải pháp Keras surrogate có ưu điểm (tận dụng quantization INT8, tương thích với hệ sinh thái TensorFlow) nhưng cũng có nhược điểm: tổn thất xấp xỉ (Keras MLP chỉ đạt 80–90% mức đồng thuận với RF/SVM gốc), tốn thêm bộ nhớ, và phức tạp hơn khi huấn luyện. **Khuyến nghị:** RF/SVM ưu tiên sinh mã C++ trực tiếp (chính xác, gọn, nhanh); NN/CNN dùng TFLite để tận dụng hệ sinh thái trưởng thành; Keras surrogate chỉ khi bắt buộc tích hợp với runtime chuẩn hóa.

6.7 Các hạn chế

6.7.1 Hạn chế tập dữ liệu

Xác thực hiện tại sử dụng dữ liệu từ một đối tượng duy nhất thực hiện năm hoạt động. Nghiên cứu tiếp theo nên mở rộng sang các nhóm đa đối tượng với nhân khẩu học đa dạng (tuổi, giới tính, mức độ vận động), bổ sung thêm các loại hoạt động như té ngã, các hoạt động phức tạp (nấu ăn, lái xe), chuyển đổi hoạt động, và kiểm chứng trên các tập dữ liệu benchmark công khai (UCI HAR, WISDM, PAMAP2).

6.7.2 Lựa chọn kích thước cửa sổ

Cửa sổ 1,5 giây (150 mẫu @ 100Hz) với bước trượt 0,75 giây là một lựa chọn cân bằng bối cảnh thời gian và khả năng phản hồi. Framework này hiện tại sử dụng kích thước cửa sổ cố định; các chiến lược cửa sổ thích ứng (thay đổi kích thước dựa trên cường độ hoạt động) có thể cải thiện hiệu suất nhưng thêm độ phức tạp.

6.7.3 Vị trí và hướng cảm biến

Các thử nghiệm giả định IMU được đeo ở cổ tay trong một hướng nhất quán. Triển khai thực tế phải đối mặt với nhiều biến đổi hơn: các vị trí đeo khác nhau trên cơ thể tạo ra đặc tính tín hiệu riêng, việc xoay thiết bị dẫn đến không xác định về hướng đặt, và sử dụng nhiều cảm biến đồng thời đòi hỏi tích hợp dữ liệu đa nguồn. Các hệ thống bền vững hơn cần trích xuất đặc trưng bất biến hướng hoặc các thuật toán ước lượng hướng bổ sung.

6.7.4 Sự đa dạng nền tảng tính toán

Framework hiện đã hiện thực generator cho nhiều họ đích, nhưng mức độ xác thực cần được phân tách thành hai lớp. Ở lớp thứ nhất, *khả năng sinh mã* đã được chứng minh qua kiến trúc factory và các generator đặc thù nền tảng. Ở lớp thứ hai, *benchmark định lượng liên thiết bị* vẫn mới chỉ được đo chi tiết trên XIAO. Phát biểu chính xác của luận văn là “framework cung cấp năng lực triển khai đa thiết bị ở cấp độ kiến trúc và sinh mã, với XIAO là nền tảng thực nghiệm tham chiếu đầu tiên”. Các bước tiếp theo cần benchmark thêm trên AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico).

6.7.5 Tính hợp lệ bên ngoài

Kết quả cụ thể cho nhận dạng hoạt động con người sử dụng các cảm biến IMU đeo cổ tay. Khả năng áp dụng cho các lĩnh vực theo dõi chuyển động khác (nhận dạng cử chỉ, theo dõi hành vi động vật, lập hồ sơ chuyển động robot) vẫn cần được xác thực. Kiến trúc của framework được thiết kế để không phụ thuộc miền, nhưng các chiến lược tiền xử lý và bộ đặc trưng có thể yêu cầu thích ứng.

6.8 Hướng phát triển tương lai

6.8.1 Các mở rộng ngắn hạn

Các cải tiến ngay lập tức trong kiến trúc hiện tại:

1. **Các thuật toán bổ sung:** Tích hợp XGBoost, LightGBM, k-NN
2. **Lựa chọn đặc trưng:** Triển khai phân tích tầm quan trọng đặc trưng tự động để giảm chiều
3. **Nén mô hình:** Cắt tỉa, huấn luyện nhận thức lượng tử hóa để giảm thêm bộ nhớ
4. **Nhiều nền tảng hơn:** Mở rộng benchmark trên ESP32, STM32, nRF5340
5. **Trực quan hóa thời gian thực:** Luồng hoạt động trực tiếp từ thiết bị đã triển khai đến GUI framework

6.8.2 Tầm nhìn dài hạn

Các cải tiến chuyển đổi yêu cầu thay đổi kiến trúc:

1. **Mở rộng học sâu:** Tích hợp LSTM/Transformer và mở rộng đường dẫn TFLite Micro cho các kiến trúc phức tạp hơn
2. **Học liên bang:** Huấn luyện mô hình cộng tác bảo vệ quyền riêng tư trên nhiều thiết bị
3. **AI có thể giải thích:** Trực quan hóa các mẫu cảm biến nào kích hoạt dự đoán cụ thể (tích hợp LIME, SHAP)
4. **Hợp nhất đa phương thức:** Tích hợp IMU + GPS + nhịp tim + âm thanh với đồng bộ hóa đặc trưng tự động
5. **Học trực tuyến:** Cho phép các mô hình đã triển khai thích ứng với các mẫu cụ thể của người dùng theo thời gian
6. **Backend đám mây:** Sao lưu dữ liệu tùy chọn, phiên bản mô hình và chia sẻ mô hình cộng đồng trong khi duy trì kiến trúc local-first
7. **Hiệu chỉnh tin cậy:** Áp dụng temperature scaling^[17] để cải thiện chất lượng hiệu chỉnh tin cậy softmax

8. **Tăng cường thích ứng:** Mở rộng chiến lược nhận biết lớp để tự động xác định tham số tăng cường tối ưu cho từng lớp dựa trên đặc tính tín hiệu

6.8.3 Các cải tiến khả năng sử dụng

- Các nghiên cứu người dùng chính thức so sánh giao diện tiền xử lý với các lựa chọn thay thế (Edge Impulse, coding thủ công)
- Các quy trình làm việc có hướng dẫn cho người dùng mới với hướng dẫn từng bước và các tập dữ liệu ví dụ
- Giám sát thiết bị thời gian thực cho thấy các luồng cảm biến trực tiếp và kết quả dự đoán

6.9 Kết luận

6.9.1 Tóm tắt đóng góp

Nghiên cứu này giải quyết thách thức đưa ứng dụng theo dõi chuyển động được hỗ trợ bởi AI đến gần hơn với các nhà nghiên cứu, nhà phát triển và giảng viên mà không phải đánh đổi hiệu suất hay mức độ kiểm soát trên các loại thiết bị biên đa dạng. Luận văn trình bày một framework mã nguồn mở toàn diện, tích hợp tiền xử lý dữ liệu, huấn luyện mô hình và triển khai biên vào một quy trình thân thiện với người dùng.

Bảng đóng góp kỹ thuật chính:

1. Giao diện tiền xử lý tương tác kéo thả giảm thời gian tiền xử lý 60-80%
2. Hỗ trợ đa thuật toán (RF, SVM, NN, PyTorch MLP, PyTorch 1D-CNN) với API nhất quán
3. Kiến trúc triển khai đa thiết bị nhận biết tối ưu hóa với bốn chế độ
4. Xử lý mất cân bằng lớp hệ thống với `class_weight='balanced'` mặc định
5. Đảm bảo tương đồng huấn luyện-triển khai qua quy trình xác minh 12 điểm kiểm tra
6. Tăng cường dữ liệu nhận biết lớp với chiến lược phân biệt tĩnh/dộng
7. Ngưỡng tin cậy cho từ chối hoạt động không xác định mà không cần mô hình bổ sung

Xác thực thực nghiệm: Độ chính xác 97,38–99,13% trên năm thuật toán cho bài toán 5 lớp, 100% cho bài toán 3 lớp; F1-scores 0,949–1,000 trên tất cả các lớp hoạt động bao gồm các lớp thiểu số; triển khai thành công trên nền tảng tham chiếu Sseed XIAO nRF52840 với PyTorch 1D-CNN đạt 99,4% độ chính xác trên thiết bị.

6.9.2 Xem lại các mục tiêu nghiên cứu

Các mục tiêu nghiên cứu ban đầu được nêu trong Mục 1.3 đã được giải quyết toàn diện:

1. **Hệ thống tiền xử lý tương tác ✓:** Giao diện cửa sổ thời gian kéo thả được triển khai thành công
2. **Hỗ trợ đa thuật toán ✓:** Năm thuật toán tích hợp, tất cả đạt 97,38–99,13%
3. **Tạo mã không phụ thuộc nền tảng ✓:** Kiến trúc trình tạo modular theo ma trận model-platform-backend
4. **Xác thực hiệu quả framework ✓:** Độ chính xác cạnh tranh và triển khai thành công trên thiết bị tham chiếu
5. **Khả năng tiếp cận ✓:** giao diện web với đường cong học tập thoải
6. **Độ tin cậy triển khai ✓:** Quy trình xác minh tương đồng 12 điểm, ngưỡng tin cậy và tăng cường nhận biết lớp

6.9.3 Các ứng dụng thực tế

Framework đã xác thực cho phép các ứng dụng thực tế đa dạng:

Y tế và phục hồi chức năng Các hệ thống đeo để giám sát mức độ hoạt động của bệnh nhân, theo dõi tiến trình phục hồi chức năng (phục hồi di động sau phẫu thuật, trị liệu đột quỵ), phát hiện té ngã trong quần thể người cao tuổi và cung cấp dữ liệu hoạt động khách quan cho các thử nghiệm lâm sàng.

Thể thao và thể dục Phát hiện và phân loại tập luyện tự động, phân tích hình thức để phòng ngừa chấn thương (phát hiện bất đối xứng đáng đi, kỹ thuật nâng không đúng) và phản hồi đào tạo cá nhân hóa. Cửa sổ trượt 1,5 giây với bước trượt 0,75 giây đảm bảo phản hồi gần thời gian thực.

Nghiên cứu và giáo dục Nền tảng chi phí thấp để giảng dạy các khái niệm edge AI trong các khóa học đại học/sau đại học, cho phép các phòng thí nghiệm thực hành mà không cần giấy phép thương mại đắt đỏ. Các nhà nghiên cứu có thể nhanh chóng tạo mẫu các thuật toán nhận dạng hoạt động mới lạ và triển khai lên phần cứng để xác thực hiện trường.

Nhà thông minh và hỗ trợ sinh hoạt hàng ngày Giám sát hoạt động cho người cao tuổi sống độc lập (phát hiện các trạng thái bất thường, theo dõi thói quen hàng ngày), tự động hóa nhà thông minh nhận biết bối cảnh (điều chỉnh ánh sáng/nhiệt độ theo hoạt động được phát hiện) và phân tích xu hướng sức khỏe dài hạn.

6.9.4 Suy nghĩ cuối cùng

Luận văn này trình bày một framework toàn diện giải quyết các thách thức cốt lõi trong theo dõi chuyển động trên thiết bị biên: khả năng tiếp cận, hiệu suất và tính linh hoạt. Giá trị cốt lõi của framework không nằm ở việc tối ưu riêng cho Sseed XIAO, mà ở khả năng chuyển cùng một pipeline sang nhiều thiết bị biên khác nhau trong khi vẫn đảm bảo tính minh bạch và khả năng kiểm chứng. Xác thực thực nghiệm khẳng định rằng **tính tiếp cận rộng rãi và hiệu suất cao không mâu thuẫn nhau—chúng hoàn toàn có thể cùng tồn tại.**

Hành trình từ dữ liệu cảm biến thô đến mô hình edge AI sẵn sàng triển khai trải qua nhiều bước, mỗi bước truyền thống đòi hỏi chuyên môn chuyên biệt. Framework này giảm thiểu các rào cản đó, cho phép bất kỳ ai có kiến thức về bài toán (các hoạt động cần nhận dạng) tạo ra hệ thống sẵn sàng triển khai mà không cần trở thành chuyên gia về xử lý tín hiệu, học máy hay lập trình nhúng. Đây là bản chất của việc phổ cập công nghệ: **hạ thấp rào cản mà không hạ thấp tiêu chuẩn.**

Nhìn về tương lai, sự phát triển của edge AI sẽ phụ thuộc vào các công cụ trao quyền cho người dùng thay vì giới hạn họ, giáo dục thay vì che giấu, và tính bao trùm thay vì loại trừ. Framework này là một bước trong hướng đó.

Mã nguồn, tài liệu và tập dữ liệu ví dụ được công bố công khai nhằm hỗ trợ tái tạo kết quả, mục đích giáo dục và đóng góp cộng đồng. Các nhà nghiên cứu và nhà thực hành được khuyến khích xây dựng trên nền tảng này, mở rộng khả năng của framework và ứng dụng vào các bài toán theo dõi chuyển động mới.

Tài liệu tham khảo

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2015.
- [2] Allied Market Research. Motion tracking system market size, share, competitive landscape and trend analysis report. <https://www.alliedmarketresearch.com/motion-tracking-system-market>, 2023. Accessed: December 2, 2025.
- [3] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.
- [4] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. *21th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2013.
- [5] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. Mlperf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [6] Oresti Banos, Juan-Manuel Galvez, Miguel Damas, Hector Pomares, and Ignacio Rojas. Window size impact in human activity recognition. *Sensors*, 14(4):6474–6499, 2014.
- [7] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [8] Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106:249–259, 2018.

- [9] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [10] Lipika Dutta and Swapna Bharali. Tinyml meets iot: A comprehensive survey. *Internet of Things*, 16:100461, 2021.
- [11] Edge Impulse. Edge impulse studio. <https://studio.edgeimpulse.com/>, 2024. Accessed: December 12, 2024.
- [12] Alessandro Filippeschi, Norbert Schmitz, Markus Miezal, Gabriele Bleser, Emanuele Ruffaldi, and Didier Stricker. Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion. *Sensors*, 17(6):1257, 2017.
- [13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [14] Google. Litert overview. <https://ai.google.dev/edge/litert>, 2024. Accessed: December 12, 2024.
- [15] Google. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Accessed: December 12, 2024.
- [16] Google. Teachable machine. <https://teachablemachine.withgoogle.com/>, 2024. Accessed: December 12, 2024.
- [17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.
- [18] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [19] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [20] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

- [21] Arthur Le Guennec, Simon Malinowski, and Romain Tavenard. Data augmentation for time series classification using convolutional neural networks. In *ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data*, 2016.
- [22] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:11711–11722, 2020.
- [23] Massimo Merenda, Carlo Porcaro, and Demetrio Iero. Edge machine learning for ai-enabled iot devices: A review. *Sensors*, 20(9):2533, 2020.
- [24] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 2nd edition, 1999.
- [25] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022.
- [26] Anuj Patil, Darshan Rao, Kaustubh Utturwar, Tejas Shelked, and Ekta Sarda. Body posture detection and motion tracking using ai for medical exercises and recommendation system. *ITM Web Conference, Volume 44, International Conference on Automation, Computing and Communication 2022 (ICACC-2022)*, May 2022.
- [27] Partha Pratim Ray. A review on tinyml: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 34(4):1595–1623, 2022.
- [28] Attila Reiss and Didier Stricker. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th International Symposium on Wearable Computers (ISWC)*, pages 108–109, 2012.
- [29] Abraham Savitzky and Marcel J. E. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8):1627–1639, 1964.
- [30] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.

- [31] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 28, 2015.
- [32] SensiML Corporation. Sensiml analytics toolkit. <https://sensiml.com/>, 2024. Accessed: December 3, 2025.
- [33] Jaspreet Singh, Yaochu Xie, Chen Chen, and Wenchao Jiang. Attention-based human activity recognition using wearable sensors. *IEEE Sensors Journal*, 23(11):12213–12223, 2023.
- [34] Odongo Steven Eyobu and Dong Seog Han. Feature representation and data augmentation for human activity classification based on wearable IMU sensor data using a deep LSTM neural network. *Sensors*, 18(9):2892, 2018.
- [35] Terry T. Um, Franz M. J. Pfister, Daniel Pichler, Satoshi Endo, Muriel Lang, Sandra Hirche, Urban Fiber, and Dana Klückmann. Data augmentation of wearable sensor data for Parkinson’s disease monitoring using convolutional neural networks. In *Proceedings of the 19th ACM International Conference on Multimodal Interaction (ICMI)*, pages 216–220. ACM, 2017.
- [36] Shaohua Wan, Lianyong Qi, Xiaolong Xu, Chenguang Tong, and Zongming Gu. Deep learning models for real-time human activity recognition with smartphones. *Mobile Networks and Applications*, 25:743–755, 2020.
- [37] An Wang, Guangyu Chen, Chuang Shang, Mingli Zhang, and Likun Liu. Human activity recognition in a smart home environment with stacked denoising autoencoders. *IEEE Transactions on Industrial Informatics*, 19(2):2071–2080, 2023.
- [38] An Wang, Guangyu Chen, Jian Yang, Shihua Zhao, and Ching-Yao Chang. A comparative study on human activity recognition using inertial sensors in a smartphone. *IEEE Sensors Journal*, 24(3):3234–3247, 2024.
- [39] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [40] Michael W. Whittle. *Gait Analysis: An Introduction*. Butterworth-Heinemann, 5th edition, 2014.

- [41] Ke Xia, Jianguang Huang, and Hanyu Wang. Lstm-cnn architecture for human activity recognition. *IEEE Access*, 8:56855–56866, 2020.
- [42] Matteo Zago, Matteo Luzzago, Tommaso Marangoni, Mariolino De Cecco, Marco Tarabini, and Manuela Galli. 3d tracking of human motion using visual skeletonization and stereoscopic vision. *Machine Learning Approaches to Human Movement Analysis*, March 2020.
- [43] Huiyu Zhou and Huosheng Hu. Human motion tracking for rehabilitation—a survey. *Biomedical Signal Processing and Control*, 3:1–18, 01 2008.